

Assessment 1: Analysing historical data with system performance - Phase 2

Student ID: 35721588

Unit: ITO5202

Teaching Period: 5, 2026

Dataset: Brazilian E-Commerce Public Dataset by Olist

Source: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Contents

Setup

- Environment and configuration
- Data loading
- Data quality assessment

Part A: Analytical query design and implementation

1. Business query design
2. DataFrame API implementation, including Catalyst optimisation
3. Spark SQL implementation and API comparison
4. Result validation

Part B: System perspective and performance analysis

1. Partitioning strategy
 2. Execution time benchmarking
 3. Execution plan interpretation
 4. DAG analysis via the Spark Web UI
-

Environment and configuration

Execution environment

For this project, we plan to run Spark in **local mode** on a single machine. In this setup, Spark does not create separate executor JVMs. Instead, the driver process handles the computation itself and uses the machine's available logical CPU cores to run tasks in parallel. Because of this, the main memory setting that matters for our setup is `spark.driver.memory`, so we do not need to configure executor memory separately.

Property	Value
Machine	MacBook Air (Retina, 13-inch, 2018)
Processor	1.6 GHz dual-core Intel Core i5
Logical cores	4
Physical memory	8 GB
Operating system	macOS Sonoma 14.7.8
Java	Eclipse Temurin JDK 17 (x64)
Python	3.11.9
PySpark	3.5.1
Spark master	local[*]

The environment details are generated directly in the notebook rather than written in manually. This means the values referred to later in the Part B benchmarking discussion can be checked against the notebook output.

```
In [1]: # Import packages
import os
import time
from statistics import median
import pandas as pd

from pyspark.sql import SparkSession, Window
from pyspark.sql import functions as F
from pyspark.sql.types import *

# Set up data directory folder path
DATA_DIR = "data"
```

SparkSession configuration

For this project, there are three key Spark settings which we changed from their default values due to how they affect the behaviour we want to examine later in Part B.

spark.driver.memory = 3g. Our machine has 8 GB of RAM, which also needs to support the computer's other processes. Allocating too much memory to Spark could slow down our execution significantly. This matters to us beyond a speed perspective, since Part B.2 compares execution times. More specifically, this would make our results less useful because they could reflect memory pressure rather than Spark's actual processing behaviour.

spark.sql.shuffle.partitions = 4. This setting determines how many partitions Spark creates after a shuffle. The default is 200, which makes more sense for a much larger cluster than for the local environment we are using here. With four available task slots and a working set of roughly 110,000 rows after joining and

filtering, using 200 partitions would create many very small tasks and add unnecessary scheduling overhead.

spark.sql.adaptive.enabled = false. Spark's Adaptive Query Execution (AQE) can change the physical execution plan while a query is running. On one hand, this can improve performance, but on the other hand, it makes the execution harder to compare with the plan shown by `explain(extended=True)`. Because Parts B.3 and B.4 require us to examine the physical plan and its corresponding DAG, AQE is turned off so that the printed plan and the executed plan remain consistent. This also means that the partition counts discussed in Part B.1 reflect the values we set ourselves rather than values Spark changes during execution.

```
In [2]: # Spark session build with local mode, 4 task slots, AQE off (as explained)
spark = SparkSession.builder \
    .appName("IT05202-A1-olist-Freight") \
    .master("local[*]") \
    .config("spark.driver.memory", "3g") \
    .config("spark.sql.shuffle.partitions", 4) \
    .config("spark.sql.adaptive.enabled", False) \
    .config("spark.sql.session.timeZone", "UTC") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")
spark
```

```
26/09/22 22:10:46 WARN Utils: Your hostname, Mounishas-MacBook-Air.local r
esolves to a loopback address: 127.0.0.1; using 192.168.0.137 instead (on
interface en0)
26/09/22 22:10:46 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to an
other address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setL
ogLevel(newLevel).
26/09/22 22:10:47 WARN NativeCodeLoader: Unable to load native-hadoop libr
ary for your platform... using builtin-java classes where applicable
```

Out [2]: **SparkSession - in-memory**

SparkContext

Spark UI

Version	v3.5.1
Master	local[*]
AppName	IT05202-A1-olist-Freight

```
In [3]: sc = spark.sparkContext

# Print the main Spark environment settings
print("Spark version:", spark.version)
print("Master:", sc.master)
print("Available task slots:", sc.defaultParallelism)
print("Driver memory:", spark.conf.get("spark.driver.memory"))
print("Shuffle partitions:", spark.conf.get("spark.sql.shuffle.partitions"))
print("AQE enabled:", spark.conf.get("spark.sql.adaptive.enabled"))
print("Broadcast join threshold (bytes):", spark.conf.get("spark.sql.auto
```

```
# Web UI link  
print("Web UI:", sc.uiWebUrl)
```

```
Spark version: 3.5.1  
Master: local[*]  
Available task slots: 4  
Driver memory: 3g  
Shuffle partitions: 4  
AQE enabled: false  
Broadcast join threshold (bytes): 10485760b  
Web UI: http://192.168.0.137:4040
```

```
In [4]: # Confirms the session can run a job end to end  
spark.range(10).count()
```

```
Out[4]: 10
```

Data Loading

Defining schemas

Our seven in-scope source files are loaded in using manually defined schemas instead of `inferSchema=True`. This is because by us using schema inference, Spark has to inspect the data before it is able to load it, thus adding extra work (this is particularly important for the geolocation file, which contains over one million rows).

Furthermore, if we were to allow Spark to infer the schema automatically, the same column in different data files could be interpreted as being of different data types, which can have downstream impacts on analysis when we try to do joins. Instead, by defining the schema ourselves, we are able to ensure consistency in data types.

We also note that as per our proposal, we are looking to only use seven of the nine available data files, as do not wish to include the payments and reviews dataset as they fall outside the scope of our analysis. As such, we do not read in these datasets such as to avoid unnecessary extra work.

Note on the approved proposal

Given a heading error in Section 3 of our original proposal, we need to make a correction in our dataset list. The first table in Section 3 of our approved proposal is labelled `olist_geolocation_dataset.csv`, but the columns listed underneath it actually belong to `olist_order_items_dataset.csv`. The geolocation dataset then appears again later in the sme section of our proposal with the correct columns.

We note that the approved proposal has been kept unchanged in `proposal/proposal.md` for consistency. However, the schemas defined below

use the actual, correct columns from each source file, which were checked against the downloaded dataset.

Note on source column names

The source Kaggle file uses the column names `product_name_lenght` and `product_description_lenght`. However, in our approved proposal, we listed these names with the correct spelling of "length".

Now, for the purposes of our analysis, the schema needs to match the actual CSV headers exactly, thus creating a mismatch with our earlier proposal. To keep the rest of the notebook easier to read, these two columns are renamed immediately after loading.

```
In [5]: # Define the schemas manually so Spark does not have to infer them

order_items_schema = StructType([
    StructField("order_id", StringType(), False),
    StructField("order_item_id", IntegerType(), False),
    StructField("product_id", StringType(), True),
    StructField("seller_id", StringType(), True),
    StructField("shipping_limit_date", TimestampType(), True),
    StructField("price", DoubleType(), True),
    StructField("freight_value", DoubleType(), True),
])

orders_schema = StructType([
    StructField("order_id", StringType(), False),
    StructField("customer_id", StringType(), False),
    StructField("order_status", StringType(), True),
    StructField("order_purchase_timestamp", TimestampType(), True),
    StructField("order_approved_at", TimestampType(), True),
    StructField("order_delivered_carrier_date", TimestampType(), True),
    StructField("order_delivered_customer_date", TimestampType(), True),
    StructField("order_estimated_delivery_date", TimestampType(), True),
])

customers_schema = StructType([
    StructField("customer_id", StringType(), False),
    StructField("customer_unique_id", StringType(), True),
    StructField("customer_zip_code_prefix", IntegerType(), True),
    StructField("customer_city", StringType(), True),
    StructField("customer_state", StringType(), True),
])

sellers_schema = StructType([
    StructField("seller_id", StringType(), False),
    StructField("seller_zip_code_prefix", IntegerType(), True),
    StructField("seller_city", StringType(), True),
    StructField("seller_state", StringType(), True),
])

geolocation_schema = StructType([
    StructField("geolocation_zip_code_prefix", IntegerType(), True),
    StructField("geolocation_lat", DoubleType(), True),
    StructField("geolocation_lng", DoubleType(), True),
    StructField("geolocation_city", StringType(), True),
])
```

```

    StructField("geolocation_state", StringType(), True),
])

products_schema = StructType([
    StructField("product_id", StringType(), False),
    StructField("product_category_name", StringType(), True),
    StructField("product_name_lenght", IntegerType(), True),
    StructField("product_description_lenght", IntegerType(), True),
    StructField("product_photos_qty", IntegerType(), True),
    StructField("product_weight_g", IntegerType(), True),
    StructField("product_length_cm", IntegerType(), True),
    StructField("product_height_cm", IntegerType(), True),
    StructField("product_width_cm", IntegerType(), True),
])

category_schema = StructType([
    StructField("product_category_name", StringType(), True),
    StructField("product_category_name_english", StringType(), True),
])

```

```

In [6]: # Helper function to read a CSV
def load_csv(filename, schema):
    return (spark.read
            .option("header", True)
            .schema(schema)
            .csv(f"{DATA_DIR}/{filename}"))

order_items = load_csv("olist_order_items_dataset.csv", order_items_schema)
orders = load_csv("olist_orders_dataset.csv", orders_schema)
customers = load_csv("olist_customers_dataset.csv", customers_schema)
sellers = load_csv("olist_sellers_dataset.csv", sellers_schema)
geolocation = load_csv("olist_geolocation_dataset.csv", geolocation_schema)
categories = load_csv("product_category_name_translation.csv", category_schema)

# Correct the misspelled column names present in the source file headers
products = (load_csv("olist_products_dataset.csv", products_schema)
            .withColumnRenamed("product_name_lenght", "product_name_length")
            .withColumnRenamed("product_description_lenght", "product_description_length"))

```

```

In [7]: # Check row counts against the expected values
order_items_count = order_items.count()
orders_count = orders.count()
customers_count = customers.count()
sellers_count = sellers.count()
geolocation_count = geolocation.count()
products_count = products.count()
categories_count = categories.count()

rows = [
    {
        "dataset": "order_items",
        "expected": 112650,
        "actual": order_items_count,
        "match": order_items_count == 112650,
        "columns": len(order_items.columns)
    },
    {
        "dataset": "orders",
        "expected": 99441,

```

```

        "actual": orders_count,
        "match": orders_count == 99441,
        "columns": len(orders.columns)
    },
    {
        "dataset": "customers",
        "expected": 99441,
        "actual": customers_count,
        "match": customers_count == 99441,
        "columns": len(customers.columns)
    },
    {
        "dataset": "sellers",
        "expected": 3095,
        "actual": sellers_count,
        "match": sellers_count == 3095,
        "columns": len(sellers.columns)
    },
    {
        "dataset": "geolocation",
        "expected": 1000163,
        "actual": geolocation_count,
        "match": geolocation_count == 1000163,
        "columns": len(geolocation.columns)
    },
    {
        "dataset": "products",
        "expected": 32951,
        "actual": products_count,
        "match": products_count == 32951,
        "columns": len(products.columns)
    },
    {
        "dataset": "categories",
        "expected": 71,
        "actual": categories_count,
        "match": categories_count == 71,
        "columns": len(categories.columns)
    }
]

pd.DataFrame(rows)

```

Out[7]:

	dataset	expected	actual	match	columns
0	order_items	112650	112650	True	7
1	orders	99441	99441	True	8
2	customers	99441	99441	True	5
3	sellers	3095	3095	True	4
4	geolocation	1000163	1000163	True	5
5	products	32951	32951	True	9
6	categories	71	71	True	2

Data quality assessment

Our approved proposal identified a few data quality issues that we will need to check before we can safely begin our analysis.

As such, it is important for us to now look at these issues in the loaded data, measure how much of the data is affected, and consider the cleaning and aggregation choices we will need to make later when building our main analysis dataset.

```
In [8]: # Check the different order statuses
orders.groupBy("order_status") \
    .count() \
    .orderBy(F.desc("count")) \
    .show()
```

[Stage 16:=====>
3) / 4]

(1 +

order_status	count
delivered	96478
shipped	1107
canceled	625
unavailable	609
invoiced	314
processing	301
created	5
approved	2

```
In [9]: # Check missing delivery-related timestamps
orders.select(
    F.sum(F.col("order_approved_at").isNull().cast("int")).alias("missing_approved"),
    F.sum(F.col("order_delivered_carrier_date").isNull().cast("int")).alias("missing_carrier"),
    F.sum(F.col("order_delivered_customer_date").isNull().cast("int")).alias("missing_delivered"),
    F.sum(F.col("order_estimated_delivery_date").isNull().cast("int")).alias("missing_estimated")
).show()
```

[Stage 18:=====>
3) / 4]

(1 +

missing_approved	missing_carrier	missing_delivered	missing_estimated
160	1783	2965	0

```
In [10]: # Check how many geolocation rows there are for each postcode prefix
geo_stats = geolocation.groupBy("geolocation_zip_code_prefix").count()

total_geo_rows = geolocation.count()
distinct_postcodes = geo_stats.count()
```

```
print("Total geolocation rows:", total_geo_rows)
print("Distinct postcode prefixes:", distinct_postcodes)

geo_stats.agg(
    F.avg("count").alias("average rows per postcode"),
    F.max("count").alias("maximum rows per postcode")
).show()
```

Total geolocation rows: 1000163
Distinct postcode prefixes: 19015

[Stage 25:=====> (1 + 3) / 4]

average rows per postcode	maximum rows per postcode
52.598632658427555	1146

```
In [11]: # Number of different customer postcode prefixes
customer_postcodes = customers.select("customer_zip_code_prefix").distinct()

print("Distinct customer postcode prefixes:", customer_postcodes)

# Check how customers are distributed across states
customers.groupBy("customer_state") \
    .count() \
    .orderBy(F.desc("count")) \
    .show(10)
```

Distinct customer postcode prefixes: 14994

customer_state	count
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140
ES	2033
GO	2020

only showing top 10 rows

```
In [12]: # Check for missing product information used later in the analysis

products.select(
    F.sum(F.col("product_category_name").isNull().cast("int")).alias("missing_category"),
    F.sum(F.col("product_weight_g").isNull().cast("int")).alias("missing_weight"),
    F.sum(F.col("product_length_cm").isNull().cast("int")).alias("missing_length"),
    F.sum(F.col("product_height_cm").isNull().cast("int")).alias("missing_height"),
    F.sum(F.col("product_width_cm").isNull().cast("int")).alias("missing_width")
).show()
```

missing_category	missing_weight	missing_length	missing_height	missing_width
610	2	2	2	

Data quality findings

Order completeness: There are 99,441 orders in total, with 96,478 marked as delivered (97.0%). However, 2,965 orders have missing `order_delivered_customer_date`, which exceeds the number of non-delivered orders by eight. This means that in our data, there are eight orders which were marked as delivered even though there was no recorded delivery timestamp. Furthermore, there is no indication from the data as to what could be causing this (e.g. delivery error, system issue, , etc.). Since we do not have a clear cause, we should treat this as a data integrity issue, and thus want to filter our analysis for both `oder_status = 'delivered'`, as well as a non-null delivery timestamp.

Geolocation duplication: The geolocation dataset has 1,000,163 rows, however it only contains 19,015 unique postcode prefixes. That means that on average, each unique postcode appears ~52.6 times in the dataset, with the most common postcode appearing 1,146 times. Given this, if we were to join this dataset directly, it would result in a large number of duplicate matches. Thus, we want to reduce our geolocation data to one row per postcode prefix before conducting any joins. As an additional benefit, at this smaller size, it is also small enough to be used as a broadcast lookup rather than requiring both sides of the join to be shuffled.

Cardinality of partitioning column: `customer_zip_code_prefix` contains 14,994 unique values, which we can see is very close to the ~15,000 expected values we mentioned in the proposal. Another thing we briefly mentioned in the proposal was the uneven distribution of customers across states. From our above data exploration, we can see a much clearer picture of the customer distribution, with São Paulo containing 41,746 customers, which accounts for ~42% of the total, while SP, RJ and MG together account for aother ~66.6%. This uneven distribution is important for us later when we consider the hash and range partitioning comparison in Part B.1, because range partitioning may produce less balanced partitions when the values themselves are unevenly distributed.

Product attribute completeness: We can see that there are 610 products (1.9%) with no category name, while two products are missing physical dimension values. This means measures that depend on package volume or weight cannot be calculated for those rows. As such, instead of filling in estimated values, we want to exclude these rows from the relevant weight-based calculations, with the number of affected records being reported with the results.

Category translation coverage. The translation table contains 71 category names, but the products table contains two categories that are missing from it: `pc_gamer` and `portateis_cozinha_e_preparadores_de_alimentos`. Rather than leaving these untranslated, we opted to add the two missing mappings to the lookup before joining, so the English category is complete for all products. The 610 products with no category name will thus remain null across both fields.

Part A.1: Business query design

Analytical question

When considering any analysis, it is important for us to first consider the business context of the data.

We know that Olist operates as a marketplace intermediary; meaning that sellers are able to set their own prices and Olist charges the customer a freight amount per item. However, what we aren't able to immediately ascertain is how to compare these values, any indication of freight profit, cost recovery or whether a shipment was priced correctly. That is to say, the same freight charge can represent something quite different for a light product and a heavy product, or for shipments travelling between different states.

Thus, for our analysis, freight is therefore compared relative to product weight and shipping lane.

From here, we seek to understand the answer to one key question through our query:

For each shipping lane (seller state to customer state) and each calendar quarter, what is the mean freight charged per kilogram, how does it compare to other lanes active in the same quarter, and how has it changed from the same lane's previous period?

In order for us to answer this, our analysis first groups order items by shipping lane and quarter so that we have final output contains one row for each qualifying lane and quarter. We then want to calculate freight charged per kilogram, rank the qualifying lanes within each quarter, and compare each lane with its own result from the previous quarter.

How we can interpret the results

Our query can provide us with two different types of comparison:

- The **within-quarter** ranking, which compares shipping lanes with one another. We note that this needs to be interpreted carefully because freight charges are affected by distance and other shipment characteristics. For example, a shipment within one state is not directly comparable with one travelling across

Brazil. The ranking is therefore used mainly to show the distribution of freight per kilogram and identify lanes at the higher or lower ends of that distribution.

- The **within-lane** comparison, which is useful for looking at change over time. Because the seller and customer states remain the same with lanes, geography is more consistent when a lane is compared with its own earlier result. A large change may therefore suggest a change in factors such as the products being shipped, package size, carrier arrangements or freight pricing.

Importantly, we need to keep in mind that a higher or lower value is not treated as automatically "good" or "bad". Rather, our main purpose should be the identification of patterns that may be worth investigating further. We should also note that `distance_km` is calculated from the postcode-prefix centroid values, so it utilises straight-line distance rather than the actual shipment distance (Which is unknown from our dataset). However, we have elected to retain it as supporting context even though it does not form part of our primary measure.

What the analysis can tell us

The result of our analysis can be leverage as a potential screening tool which can highlight lanes that:

- Have relatively high or low freight per kilogram within a quarter; or
- Show a noticeable change compared with their previous observed quarter.

The main purpose of the analysis is therefore to reduce a large number of individual order items into a smaller set of lane-quarter results that can be investigated more closely.

Operations used

From the above, it becomes clear that the results we need cannot be produced using a single aggregation. This is due to the fact that the required information is spread across several datasets and the analysis includes both group-level and time-based comparisons.

Operation	Use in the analysis
Joins	<code>order_items</code> is combined with <code>orders</code> for order dates and status, <code>customers</code> and <code>sellers</code> for the shipping lane, and <code>products</code> for product weight.
Derived columns	Shipping lane, weight in kilograms, freight per kilogram and purchase quarter are calculated from the source columns.
Aggregation	Order items are grouped by shipping lane and quarter to produce the final lane-quarter level measures.
Post-aggregation filtering	Lane-quarter groups with too few items are removed after aggregations due to the fact that the minimum count applies to the group rather than an individual row.
Window functions	<code>RANK()</code> compares lanes within the same quarter, while <code>LAG()</code> retrieves the previous available result for each lane.

Operation	Use in the analysis
Time-based analysis	Calendar quarters are derived from <code>order_purchase_timestamp</code> and used to compare results over time.

We can see from the above that our query plans to use six of the operation types listed in the assessment requirements, which is above the required minimum.

Why Spark is appropriate for the analysis

It is obvious that our chosen dataset is small enough to run on one machine, and thus distributed computing is not necessarily a requirement purely driven by dataset size.

In saying that, Spark is still useful because the query contains several operations that are important in distributed data processing. Our analysis combines datasets with different sizes, performs joins on high-cardinality keys, aggregates the resulting records, and then applies window functions that require the data to be organised in different ways.

Some of these operations can require Spark to redistribute data between partitions. The value of Spark here is therefore not that the dataset could not be processed without it, but that the query demonstrates the types of operations and execution decisions that become important with larger distributed datasets.

Filtering decisions

- **Delivered orders only:** Our data quality checks in the previous section of this notebook showed that our dataset contained eight orders which were marked as `delivered` despite having no recorded customer delivery timestamp. Thus, our analysis requires both `order_status = 'delivered'` and a non-null `order_delivered_customer_date`.
- **Recorded product weight:** Freight per kilogram cannot be calculated when `product_weight_g` is missing. As such, records where `product_weight_g` is missing are excluded from calculations that require weight rather than simply estimating a value for them.
- **Minimum lane-quarter volume:** Lane-quarter groups containing only a small number of items can produce unstable or extreme results. Thus, a minimum item count is therefore applied after aggregation so the ranking is based on groups with a more meaningful amount of data.

Part A.2: DataFrame API implementation

Building the analytical base view

The information we need for our analysis is spread across several source files, so the first step for us is to combine them into a single row-level DataFrame.

This base view brings together the order, customer, seller and product information needed for our query, and also applies the filtering decisions we identified earlier.

The remaining derived columns, such as shipping lane, product weight in kilograms and purchase quarter, are also created at this stage so that we can leverage them later in our analysis.

The completed base view is cached because it is used repeatedly in later sections, including the DataFrame and SQL versions of the query, the equivalence check, partitioning experiments and execution-plan analysis. This avoids rebuilding the same joins and derived columns each time.

Join approach

Since our source tables are not all the same size, the joins need to be handled differently depending on the data involved.

The larger joins, particularly those involving `order_items`, `orders` and `customers`, use high-cardinality keys and may require Spark to redistribute records between partitions. This makes them more expensive than joins involving the smaller lookup tables.

The smaller datasets on the other hand, such as sellers, products and the reduced geolocation lookup, are small enough to be candidates for broadcast joins, which avoids shuffling both sides.

In particular, we had earlier discussed the pre-processing work required for the geolocation dataset and outlined the potential for duplicate matching due to multiple rows containing the same postcode prefix. In the context of our join approach, we can see that this additional processing step has the added benefit of making the lookup much smaller, and thus making it suitable for a broadcast join as well.

```
In [13]: # Reduce the geolocation data to one row for each postcode prefix
geo_lookup = geolocation.groupBy("geolocation_zip_code_prefix").agg(
    F.avg("geolocation_lat").alias("lat"),
    F.avg("geolocation_lng").alias("lng")
)

# Cache the geolocation data
geo_lookup.cache()

geo_count = geo_lookup.count()
print("Geolocation lookup rows:", geo_count)
```

```
[Stage 35:=====>
2) / 4]
```

```
(2 +
```

```
Geolocation lookup rows: 19015
```

```
In [14]: # Keep orders that were delivered and have a delivery date
orders_delivered = orders.filter(
    (F.col("order_status") == "delivered") &
    (F.col("order_delivered_customer_date").isNotNull())
```

```
)  
  
delivered_count = orders_delivered.count()  
print("Delivered orders retained:", delivered_count)
```

Delivered orders retained: 96470

```
In [15]: # Add the two category translations missing from the source file  
missing_categories = spark.createDataFrame(  
    [  
        ("pc_gamer", "pc_gamer"),  
        ("portateis_cozinha_e_preparadores_de_alimentos",  
         "kitchen_portables_and_food_preparers")  
    ],  
    ["product_category_name", "product_category_name_english"]  
)  
  
categories_complete = categories.unionByName(missing_categories)
```

```
In [16]: # Only keep the columns needed from each table for the joins  
orders_join = orders_delivered.select(  
    "order_id",  
    "customer_id",  
    "order_purchase_timestamp",  
    "order_delivered_customer_date",  
    "order_estimated_delivery_date"  
)  
  
customers_join = customers.select(  
    "customer_id",  
    "customer_state",  
    "customer_zip_code_prefix"  
)  
  
sellers_join = sellers.select(  
    "seller_id",  
    "seller_state",  
    "seller_zip_code_prefix"  
)  
  
products_join = products.select(  
    "product_id",  
    "product_category_name",  
    "product_weight_g",  
    "product_length_cm",  
    "product_height_cm",  
    "product_width_cm"  
)  
  
categories_join = categories_complete.select(  
    "product_category_name",  
    "product_category_name_english"  
)
```

```
In [17]: # Build the main row-level dataset  
base = order_items.join(  
    orders_join,  
    on="order_id",  
    how="inner"  
)
```

```

base = base.join(
    customers_join,
    on="customer_id",
    how="inner"
)

# These tables are small, so request broadcast joins
base = base.join(
    F.broadcast(sellers_join),
    on="seller_id",
    how="inner"
)

base = base.join(
    F.broadcast(products_join),
    on="product_id",
    how="left"
)

base = base.join(
    F.broadcast(categories_join),
    on="product_category_name",
    how="left"
)

print("Base view rows:", base.count())

```

Base view rows: 110189

```

In [18]: # Separate postcode lookups for customers and sellers
customer_geo = geo_lookup.select(
    F.col("geolocation_zip_code_prefix").alias("customer_zip_code_prefix"),
    F.col("lat").alias("cust_lat"),
    F.col("lng").alias("cust_lng")
)

seller_geo = geo_lookup.select(
    F.col("geolocation_zip_code_prefix").alias("seller_zip_code_prefix"),
    F.col("lat").alias("sell_lat"),
    F.col("lng").alias("sell_lng")
)

```

```

In [19]: # Add approximate coordinates for both ends of the shipping lane
base = base.join(
    F.broadcast(customer_geo),
    on="customer_zip_code_prefix",
    how="left"
)

base = base.join(
    F.broadcast(seller_geo),
    on="seller_zip_code_prefix",
    how="left"
)

```

```

In [20]: # Create the columns needed for analysis
base_view = base.withColumn(

```

```

        "lane",
        F.concat_ws(" -> ", F.col("seller_state"), F.col("customer_state"))
    )

base_view = base_view.withColumn(
    "purchase_quarter",
    F.concat_ws(
        "-",
        F.year("order_purchase_timestamp"),
        F.concat(F.lit("Q"), F.quarter("order_purchase_timestamp"))
    )
)

base_view = base_view.withColumn(
    "weight_kg",
    F.col("product_weight_g") / 1000.0
)

base_view = base_view.withColumn(
    "freight_per_kg",
    F.when(
        F.col("weight_kg") > 0,
        F.col("freight_value") / F.col("weight_kg")
    )
)

base_view = base_view.withColumn(
    "freight_to_price_ratio",
    F.when(
        F.col("price") > 0,
        F.col("freight_value") / F.col("price")
    )
)

base_view = base_view.withColumn(
    "package_volume_cm3",
    F.col("product_length_cm") *
    F.col("product_height_cm") *
    F.col("product_width_cm")
)

```

```

In [21]: # Approximate straight-line distance between the seller and customer
base_view = base_view.withColumn(
    "distance_km",
    F.lit(6371.0) * 2 * F.asin(
        F.sqrt(
            F.pow(
                F.sin(
                    F.radians(F.col("cust_lat") - F.col("sell_lat")) / 2
                ),
                2
            )
            +
            F.cos(F.radians(F.col("sell_lat"))) *
            F.cos(F.radians(F.col("cust_lat"))) *
            F.pow(
                F.sin(
                    F.radians(F.col("cust_lng") - F.col("sell_lng")) / 2
                ),
                2
            )
        )
    )
)

```

```

    )
  )
)

base_view = base_view.withColumn(
  "is_intrastate",
  F.col("seller_state") == F.col("customer_state")
)

```

```

In [22]: # Keep the columns that we will later
base_view = base_view.select(
  "order_id",
  "order_item_id",
  "lane",
  "seller_state",
  "customer_state",
  "customer_zip_code_prefix",
  "seller_zip_code_prefix",
  "purchase_quarter",
  "order_purchase_timestamp",
  "price",
  "freight_value",
  "weight_kg",
  "freight_per_kg",
  "freight_to_price_ratio",
  "package_volume_cm3",
  "distance_km",
  "is_intrastate",
  "product_category_name",
  "product_category_name_english"
)

```

Caching decision

Here, `base_view` is cached because given that we need to reuse it throughout the rest of our analysis, and it would take several steps to construct each time.

Since Spark transformations are not actually executed until an action is called, without caching, repeated actions on `base_view` would result in Spark having to work through the same upstream processing multiple times, including reading the source data, reducing the geolocation dataset, performing the joins and calculating the derived columns.

Also, caching is reasonable here because the final base view is much smaller than the raw source data. It contains ~110,000 rows, and has already been reduced to only the columns we likely need later in the notebook. By comparison, if we were to cache the original source datasets, this would provide considerably less benefit because most of the columns there are only used once when building the base view.

Furthermore, `geo_lookup` is also cached because the same postcode lookup is used twice: once for the customer location and once for the seller location. This avoids repeating the aggregation of the original geolocation dataset for both joins.

```
In [23]: # Confirm the joins preserved one row per order item before caching
print("Base view rows:", base_view.count())
print("Distinct order item keys:",
      base_view.select("order_id", "order_item_id").distinct().count())
```

Base view rows: 110189

[Stage 59:=====> (3 + 1) / 4]

Distinct order item keys: 110189

```
In [24]: # Cache the finished base view
base_view.cache()

# Run count() so the cache is actually populated
base_count = base_view.count()

print("Base view rows:", base_count)
print("Number of columns:", len(base_view.columns))
```

[Stage 69:=====> (1 + 3) / 4]

Base view rows: 110189

Number of columns: 19

```
In [25]: # Main analysis columns
base_view.select(
    "lane",
    "purchase_quarter",
    "price",
    "freight_value",
    "weight_kg",
    "freight_per_kg",
    "distance_km"
).show(10, truncate=False)
```

```

+-----+-----+-----+-----+-----+-----+
--+-----+
|lane      |purchase_quarter|price |freight_value|weight_kg|freight_per_kg
|distance_km      |
+-----+-----+-----+-----+-----+-----+
--+-----+
|SP -> RJ|2017-Q3          |58.9  |13.29          |0.65     |20.4461538461538
45|301.50468067509604|
|SP -> SP|2017-Q2          |239.9 |19.93          |30.0     |0.664333333333333
33|585.5639365089672 |
|MG -> MG|2018-Q1          |199.0  |17.87          |3.05     |5.85901639344262
4 |312.34351136818617|
|SP -> SP|2018-Q3          |12.99 |12.79          |0.2      |63.9499999999999
96|293.16841989111373|
|PR -> SP|2017-Q1          |199.9 |18.14          |3.75     |4.83733333333333
35|646.1634625711988 |
|SP -> MG|2017-Q2          |21.9  |12.69          |0.45     |28.2
|161.85424084700483|
|SP -> SP|2017-Q4          |19.9  |11.85          |0.2      |59.2499999999999
9 |484.860164492648 |
|SP -> SP|2018-Q3          |810.0 |70.75          |13.805   |5.12495472654835
2 |548.1063089157578 |
|SP -> SP|2018-Q1          |145.95|11.65          |2.0      |5.825
|53.23650744571914 |
|SP -> SP|2018-Q3          |53.99 |11.4           |0.85     |13.4117647058823
53|33.24946825936782 |
+-----+-----+-----+-----+-----+-----+
--+-----+

```

only showing top 10 rows

```

In [26]: # Check how many rows are missing values
base_view.select(
    F.count("*").alias("total_rows"),
    F.sum(
        F.col("freight_per_kg").isNull().cast("int")
    ).alias("missing_freight_per_kg"),
    F.sum(
        F.col("distance_km").isNull().cast("int")
    ).alias("missing_distance"),
    F.sum(
        F.col("product_category_name").isNull().cast("int")
    ).alias("missing_category"),
    F.sum(
        F.col("product_category_name_english").isNull().cast("int")
    ).alias("missing_english_category")
).show()

```

```

+-----+-----+-----+-----+-----+-----+
--+-----+
|total_rows|missing_freight_per_kg|missing_distance|missing_category|missi
ng_english_category|
+-----+-----+-----+-----+-----+-----+
--+-----+
|    110189|                26|            536|            1537|
1537|
+-----+-----+-----+-----+-----+-----+
--+-----+

```

Defining the freight per kilogram measure

Before aggregating the data, we still need to define what our "freight per kilogram" measure is actually measuring so that our resulting analysis and interpretation is an accurate reflection of the data itself. Fundamentally, there are two reasonable ways this measure could be defined, and given that they can produce quite different results, it is important for us to carefully step through them before making a decision.

The first option is to average the `freight_per_kg` value calculated for each individual item. The key issue here is that very light items can produce extremely high ratios. For example, in the sample above, a 0.2 kg item has freight of 63.95 per kg, while a 30 kg item has a value of only 0.66 per kg. If these item-level ratios are averaged, both items have the same influence on the result even though their weights are very different.

The second option is to divide the **total freight charged by the total weight shipped** within each lane-quarter. This gives an overall freight charge per kilogram across everything shipped in that group.

It is clear from the above that the second definition provides a much more robust analysis. However, we do need to note that by using the total freight divided by total weight definition, it also impacts how missing values should be handled. More specifically, rows with a missing or zero product weight must be removed before aggregation. Otherwise, their freight value would still contribute to total freight while adding no weight to the denominator, which would overstate freight per kilogram for that group.

We know already from our earlier data quality check that there are 26 rows in the base view where a valid freight-per-kilogram value could not be calculated, so these rows need to be excluded before the lane-quarter aggregation.

NOTE: Even though it is not used in this instance as a "freight per kilogram" definition, we still want to retain the average of the individual item-level ratios as a secondary measure due to the fact that comparing the two can help show where differences in item weights are having a strong effect on the result. We should be careful however, to clearly distinguish between the two measures so as to not conflate the two measure so as to not lead to incorrect conclusions.

```
In [27]: # Keep rows where weight can be used in the calculation
base_weighted = base_view.filter(
    F.col("weight_kg") > 0
)

weighted_count = base_weighted.count()
base_count = base_view.count()

print("Rows retained for weight-based measures:", weighted_count)
print("Rows excluded:", base_count - weighted_count)
```

Rows retained for weight-based measures: 110163

Rows excluded: 26

Establishing a minimum lane-quarter volume

Now that we have confirmed the number of rows being retained, it is important to consider reasonable threshold for the number of groups we are retaining. Rather than choosing a threshold arbitrarily, we first want to examine how the data is actually distributed across quarters and across lane-quarter combinations.

In [28]: *# Check how many rows are in each quarter*

```
base_weighted.groupBy(
  "purchase_quarter"
).count().orderBy(
  "purchase_quarter"
).show()
```

```
+-----+-----+
|purchase_quarter|count|
+-----+-----+
|      2016-Q3   |    3|
|      2016-Q4   |   314|
|      2017-Q1   |  5664|
|      2017-Q2   |10051|
|      2017-Q3   |13946|
|      2017-Q4   |19875|
|      2018-Q1   |23572|
|      2018-Q2   |22644|
|      2018-Q3   |14094|
+-----+-----+
```

In [29]: *# Count how many items are in each lane and quarter*

```
lane_quarter_sizes = base_weighted.groupBy(
  "lane",
  "purchase_quarter"
).count()

print("Total lane-quarter groups:", lane_quarter_sizes.count())
```

Total lane-quarter groups: 1856

In [30]: *# Look at the range of group sizes before choosing a minimum*

```
lane_quarter_sizes.select(
  F.min("count").alias("minimum"),
  F.expr("percentile_approx(count, 0.25)").alias("25th percentile"),
  F.expr("percentile_approx(count, 0.50)").alias("median"),
  F.expr("percentile_approx(count, 0.75)").alias("75th percentile"),
  F.max("count").alias("maximum")
).show()
```

```
+-----+-----+-----+-----+-----+
|minimum|25th percentile|median|75th percentile|maximum|
+-----+-----+-----+-----+-----+
|      1|              2|      4|              18|    8053|
+-----+-----+-----+-----+-----+
```

In [31]: *# See how many lane-quarter groups would remain at different minimum size*

```
groups_10 = lane_quarter_sizes.filter(F.col("count") >= 10).count()
groups_20 = lane_quarter_sizes.filter(F.col("count") >= 20).count()
groups_30 = lane_quarter_sizes.filter(F.col("count") >= 30).count()
groups_50 = lane_quarter_sizes.filter(F.col("count") >= 50).count()
groups_100 = lane_quarter_sizes.filter(F.col("count") >= 100).count()

print("Groups with at least 10 items:", groups_10)
print("Groups with at least 20 items:", groups_20)
print("Groups with at least 30 items:", groups_30)
print("Groups with at least 50 items:", groups_50)
print("Groups with at least 100 items:", groups_100)
```

```
Groups with at least 10 items: 632
Groups with at least 20 items: 445
Groups with at least 30 items: 348
Groups with at least 50 items: 260
Groups with at least 100 items: 159
```

Choosing the minimum lane-quarter volume

We can see from the above checks that the lane-quarter group sizes are quite uneven. Across the 1,856 combinations, the median group contains only 4 items the 25th percentile contains 2, but the maximum number of items is 8,053. This means that many lane-quarter combinations are based on only a small number of shipments.

This is particularly important for our analysis because very small groups can produce very high or very low freight-per-kilogram values based on only a few items. Since we are ranking the lane-quarter results, these small groups could appear near the extremes of the ranking and can heavily skew our results.

We can also see the number of groups that are retained at various thresholds in the checks above. If we convert these figures to being percentages of the total, we have:

Minimum items	Groups retained	Share of total
10	632	34.1%
20	445	24.0%
30	348	18.8%
50	260	14.0%
100	159	8.6%

For this analysis, we want to use a minimum of **30 items per lane-quarter**. This is above the 75th percentile group size, so it removes most of the very small combinations while still retaining 348 groups for comparison.

We believe that this is a practical choice to reduce the influence of groups based on only a handful of shipments while keeping enough lane-quarter combinations for the ranking and time-based analysis.

Importantly, this threshold also helps with the `LAG()` comparison. `LAG()` returns the previous **observed** result for each lane, which may not always be the immediately preceding calendar quarter. Requiring at least 30 items helps us to retain more active lanes, although gaps between observed quarters can still occur.

Furthermore, we can also see from our above checks that the earliest periods contain much less data than the later quarters. `2016-Q3` contains only 3 order items and `2016-Q4` contains 314, so it is likely that very few or no lane-quarter groups from this period will meet the 30-item threshold. As a result, most of our final analysis will be based on activity during 2017 and 2018.

```
In [32]: MIN_ITEMS = 30

# Summarise the data by shipping lane and quarter
lane_quarter = base_weighted.groupBy(
    "lane",
    "seller_state",
    "customer_state",
    "purchase_quarter"
).agg(
    F.count("*").alias("item_count"),
    F.countDistinct("order_id").alias("order_count"),
    F.sum("freight_value").alias("total_freight"),
    F.sum("weight_kg").alias("total_weight_kg"),
    F.avg("freight_per_kg").alias("mean_item_freight_per_kg"),
    F.avg("freight_to_price_ratio").alias("mean_freight_to_price"),
    F.avg("distance_km").alias("mean_distance_km")
)

# Main freight per kg measure: total freight divided by total weight
lane_quarter = lane_quarter.withColumn(
    "freight_per_kg",
    F.col("total_freight") / F.col("total_weight_kg")
)

# Remove lane-quarter groups that are too small
lane_quarter = lane_quarter.filter(
    F.col("item_count") >= MIN_ITEMS
)

print("Qualifying lane-quarters:", lane_quarter.count())
```

Qualifying lane-quarters: 348

Checking for a platform-wide trend

Before we can compare each lane with its own previous quarter, we need to first know whether freight per kilogram was stable across the platform as a whole over the period.

```
In [33]: # Look at the overall freight per kg and average item weight for each qua
quarter_summary = base_weighted.groupBy(
    "purchase_quarter"
).agg(
    F.count("*").alias("items"),
```

```

(F.sum("freight_value") / F.sum("weight_kg")).alias("quarter_baseline")
F.avg("weight_kg").alias("mean_weight_kg")
)

quarter_summary.select(
  "purchase_quarter",
  "items",
  F.round("quarter_baseline_fpk", 2).alias("overall_freight_per_kg"),
  F.round("mean_weight_kg", 2).alias("mean_weight_kg")
).orderBy(
  "purchase_quarter"
).show()

```

purchase_quarter	items	overall_freight_per_kg	mean_weight_kg
2016-Q3	3	2.83	1.0
2016-Q4	314	9.01	2.18
2017-Q1	5664	8.31	2.29
2017-Q2	10051	8.54	2.28
2017-Q3	13946	8.62	2.24
2017-Q4	19875	9.3	2.09
2018-Q1	23572	9.39	2.08
2018-Q2	22644	9.92	2.08
2018-Q3	14094	12.3	1.77

Adding a relative measure

We can see from the above that the platform-wide freight-per-kilogram figure increased from 8.31 in 2017-Q1 to 12.30 in 2018-Q3, which is an increase of roughly 48%. Additionally, over the same period, the mean item weight fell from 2.29 kg to 1.77 kg, signifying a decrease of about 23%.

We should note that these movements are very likely to be related. More specifically, while freight charges do not necessarily decrease in direct proportion to parcel weight, some parts of a freight charge may remain similar regardless of how heavy an item is. Thus, as the average item weight falls, freight per kilogram can therefore increase even without an equivalent change in the underlying freight charge.

In the context of our analysis, this likely means that the raw change for an individual lane should not be interpreted without considering what was happening across the platform at the same time. That is to say, if freight per kilogram are increasing across the board, part of a lane's increase could simply be attributed to that broader movement.

In order for us to account for this, we can add two relative measures alongside the raw change:

- **relative_to_quarter** : which shows a lane's freight per kilogram as a multiple of the platform-wide freight-per-kilogram figure for the same quarter.
- **relative_change** : which measures how that relative position changes between the lane's consecutive observed results.

Through the above, we are able to then compare each lane against the platform figure from the same quarter in order to control for the overall shift in freight per kilogram over time, and ultimately provide a mechanism for us to distinguish a broad platform-wide movement from a change that is more specific to an individual lane.

```
In [34]: # Add the platform-wide freight per kg for each quarter
lane_quarter = lane_quarter.join(
    F.broadcast(
        quarter_summary.select(
            "purchase_quarter",
            "quarter_baseline_fpk"
        )
    ),
    on="purchase_quarter",
    how="left"
)

lane_quarter = lane_quarter.withColumn(
    "relative_to_quarter",
    F.col("freight_per_kg") / F.col("quarter_baseline_fpk")
)

In [35]: # Rank lanes within the same quarter
quarter_window = Window.partitionBy(
    "purchase_quarter"
).orderBy(
    F.desc("freight_per_kg")
)

# Used to count how many qualifying lanes are in each quarter
quarter_totals = Window.partitionBy(
    "purchase_quarter"
)

# Compare each lane with its previous observed result
lane_window = Window.partitionBy(
    "lane"
).orderBy(
    "purchase_quarter"
)

In [36]: lane_quarter_ranked = lane_quarter.withColumn(
    "rank_in_quarter",
    F.rank().over(quarter_window)
)

lane_quarter_ranked = lane_quarter_ranked.withColumn(
    "lanes_in_quarter",
    F.count("*").over(quarter_totals)
)

lane_quarter_ranked = lane_quarter_ranked.withColumn(
    "prev_quarter",
    F.lag("purchase_quarter").over(lane_window)
)

lane_quarter_ranked = lane_quarter_ranked.withColumn(
    "prev_freight_per_kg",
```

```

    F.lag("freight_per_kg").over(lane_window)
  )

lane_quarter_ranked = lane_quarter_ranked.withColumn(
  "prev_relative",
  F.lag("relative_to_quarter").over(lane_window)
)

lane_quarter_ranked = lane_quarter_ranked.withColumn(
  "change_pct",
  F.round(
    100 * (
      F.col("freight_per_kg") - F.col("prev_freight_per_kg")
    ) / F.col("prev_freight_per_kg"),
    1
  )
)

lane_quarter_ranked = lane_quarter_ranked.withColumn(
  "relative_change",
  F.col("relative_to_quarter") - F.col("prev_relative")
)

lane_quarter_ranked.cache()

print("Qualifying lane-quarters:", lane_quarter_ranked.count())

```

[Stage 109:>

(0 +

4) / 4]

Qualifying lane-quarters: 348

```

In [37]: # Look at the highest freight per kg lanes in the most recent full quarter
q2_2018 = lane_quarter_ranked.filter(
  F.col("purchase_quarter") == "2018-Q2"
)

print("=== 2018-Q2: lanes ranked by freight charged per kilogram ===")

q2_2018.select(
  "rank_in_quarter",
  "lane",
  "item_count",
  F.round("freight_per_kg", 2).alias("freight_per_kg"),
  F.round("mean_item_freight_per_kg", 2).alias("mean_item_ratio"),
  F.round("relative_to_quarter", 2).alias("relative_to_quarter"),
  F.round("mean_distance_km", 0).alias("avg_dist_km")
).orderBy(
  "rank_in_quarter"
).show(15, truncate=False)

```

=== 2018-Q2: lanes ranked by freight charged per kilogram ===

rank_in_quarter	lane	item_count	freight_per_kg	mean_item_ratio	relative_to_quarter
	avg_dist_km				
1	MA -> SP	53	61.07	67.54	6.16
2329.0					
2	PE -> MG	30	49.21	55.39	4.96
1700.0					
3	PE -> SP	35	41.06	50.92	4.14
2118.0					
4	RJ -> PE	30	36.17	62.69	3.65
1849.0					
5	SP -> RN	70	33.34	116.88	3.36
2271.0					
6	SP -> MA	85	24.4	78.59	2.46
2208.0					
7	RJ -> RS	45	23.97	44.39	2.42
1198.0					
8	SP -> PB	62	19.82	78.01	2.0
2177.0					
9	SP -> AL	45	19.6	69.58	1.98
1915.0					
10	RS -> RJ	50	19.51	55.37	1.97
1122.0					
11	RS -> MG	40	18.85	44.63	1.9
1321.0					
12	RJ -> MG	131	18.6	58.76	1.88
385.0					
13	RS -> PR	38	18.45	51.84	1.86
575.0					
14	SP -> CE	197	18.24	69.36	1.84
2251.0					
15	SP -> TO	47	18.11	75.59	1.83
1485.0					

only showing top 15 rows

```
In [38]: # Largest changes from each lane's previous observed quarter
previous_results = lane_quarter_ranked.filter(
    F.col("prev_freight_per_kg").isNotNull()
)

print("=== Largest raw changes from previous observed quarter ===")

previous_results.select(
    "lane",
    "prev_quarter",
    "purchase_quarter",
    "item_count",
    F.round("prev_freight_per_kg", 2).alias("prev_fpk"),
    F.round("freight_per_kg", 2).alias("fpk"),
    "change_pct",
    "rank_in_quarter",
    "lanes_in_quarter"
).orderBy(
```

```
F.desc(F.abs(F.col("change_pct")))
).show(15, truncate=False)
```

=== Largest raw changes from previous observed quarter ===

lane	prev_quarter	purchase_quarter	item_count	prev_fpk	fpk	change_p
ct	rank_in_quarter	lanes_in_quarter				
SP -> T0	2017-Q2	2017-Q4	32	10.18	32.81	222.4
1	58					
RJ -> PR	2018-Q2	2018-Q3	38	15.18	33.31	119.4
2	46					
MG -> PR	2017-Q1	2017-Q2	50	7.87	15.55	97.6
7	44					
RS -> RJ	2018-Q1	2018-Q2	50	10.14	19.51	92.4
10	60					
SP -> RN	2018-Q1	2018-Q2	70	17.52	33.34	90.2
5	60					
RS -> MG	2018-Q1	2018-Q2	40	10.06	18.85	87.3
11	60					
MG -> PR	2018-Q2	2018-Q3	45	9.71	18.1	86.4
14	46					
MG -> ES	2017-Q4	2018-Q2	33	7.55	13.89	84.1
24	60					
SC -> PR	2017-Q4	2018-Q1	68	6.02	10.96	82.1
32	59					
SC -> RJ	2017-Q4	2018-Q1	84	6.57	11.85	80.2
28	59					
PR -> SC	2018-Q2	2018-Q3	47	11.64	20.85	79.2
10	46					
RJ -> RJ	2017-Q3	2017-Q4	204	5.1	9.0	76.6
41	58					
RJ -> RS	2018-Q1	2018-Q2	45	13.83	23.97	73.3
7	60					
RJ -> MG	2017-Q3	2017-Q4	136	7.92	13.69	72.7
15	58					
SP -> RN	2017-Q2	2017-Q3	59	15.38	26.15	70.0
1	50					

only showing top 15 rows

```
In [39]: # Compare changes after accounting for the platform baseline in each quar
relative_results = lane_quarter_ranked.filter(
    F.col("prev_relative").isNull()
)

print("=== Largest movements relative to the quarterly platform baseline

relative_results.select(
    "lane",
    "prev_quarter",
    "purchase_quarter",
    "item_count",
    F.round("prev_relative", 2).alias("prev_relative"),
    F.round("relative_to_quarter", 2).alias("relative_to_quarter"),
    F.round("relative_change", 2).alias("relative_change"),
```

```

    "change_pct",
    "rank_in_quarter",
    "lanes_in_quarter"
  ).orderBy(
    F.desc(F.abs(F.col("relative_change")))
  ).show(15, truncate=False)

```

=== Largest movements relative to the quarterly platform baseline ===

lane	prev_quarter	purchase_quarter	item_count	prev_relative	relative_to_quarter	relative_change	change_pct	rank_in_quarter	lanes_in_quarter
SP -> T0	2017-Q2	2017-Q4	32	1.19	3.53				
2.34	222.4	1	58						
SP -> T0	2017-Q4	2018-Q1	48	3.53	1.41				
-2.12	-59.7	19	59						
SP -> RN	2018-Q1	2018-Q2	70	1.87	3.36				
1.5	90.2	5	60						
SP -> MA	2017-Q2	2017-Q3	98	2.99	1.59				
-1.4	-46.3	11	50						
SP -> RN	2017-Q3	2017-Q4	71	3.04	1.72				
-1.32	-38.8	9	58						
SP -> AL	2017-Q2	2017-Q3	38	2.64	1.34				
-1.3	-48.8	16	50						
SP -> R0	2017-Q3	2018-Q1	38	2.25	0.98				
-1.27	-52.6	45	59						
PE -> SP	2018-Q1	2018-Q2	35	5.39	4.14				
-1.25	-18.9	3	60						
SP -> RN	2017-Q2	2017-Q3	59	1.8	3.04				
1.24	70.0	1	50						
RS -> RJ	2017-Q3	2017-Q4	51	2.23	1.01				
-1.23	-51.3	36	58						
RJ -> PR	2018-Q2	2018-Q3	38	1.53	2.71				
1.18	119.4	2	46						
SP -> PI	2017-Q4	2018-Q1	87	2.36	1.18				
-1.17	-49.3	31	59						
RS -> SP	2017-Q1	2017-Q2	86	3.08	2.04				
-1.04	-31.9	5	44						
RJ -> MG	2017-Q1	2017-Q2	46	2.1	1.07				
-1.03	-47.6	26	44						
MA -> SP	2018-Q2	2018-Q3	55	6.16	5.14				
-1.02	3.6	1	46						

only showing top 15 rows

Reading the results

Based on our analysis above, there are four key findings that we're able to identify:

1. The within-quarter ranking is strongly related to distance, as expected. We can see that in 2018-Q2, the eleven highest-ranked lanes all have mean distances between 1,100 km - 2,300 km, while RJ to MG, which is at rank 12 has a mean distance of just 385km. This supports our previous discussion around how a high ranking does not, by itself, mean that a lane is priced unusually.

2. The two freight-per-kilogram measures can produce quite different results.

In the above analysis, for the SP to RN lane in 2018-Q2, the freight per kilogram is 33.34, while the mean of the individual item ratios is 116.88. This shows how very light items can have a large effect on the average of the individual ratios.

3. The raw changes also need to be read against the broader platform trend.

Across the 280 comparisons with an observed result for previous periods, 170 are increases and 110 are decreases, with a median change of +5.0%. We note that freight per kilogram was also increasing across the platform over this period, so a positive raw change does not necessarily mean that a lane moved differently from the overall pattern.

4. The relative measure gives additional context to these changes.

`relative_to_quarter` compares each lane with the platform-wide freight-per-kilogram figure from within the same quarter, while `relative_change` shows how that relative position changes over time. This helps us to account for the broader movement in the platform baseline.

Overall, we can see that the outputs above have helped us to narrow the analysis from 110,163 individual order items to a much smaller set of lane-quarter results worth examining more closely. These include lanes that sit well above or below the platform baseline for a particular period and lanes whose relative position changes noticeably between observed quarters.

How Spark executes and optimises our query

We can see from the above that our analysis moves through several key stages:

- The source files are joined to create `base_view`,
- The data is filtered and aggregated to lane-quarter level, and
- Window functions are used for the ranking and previous-period comparisons.

An important feature of Spark is that it does not simply execute each line in the order it appears in the notebook. Instead, it leverages Catalyst to first build a logical plan, optimising it before producing a physical plan, which is then finally actually executed. Importantly, the optimisations below can therefore be linked directly to the different stages of our analysis, with the resulting physical plan being something we examine closely in Part B.3 later.

Building the base view

If we now turn to stepping through our analysis one step at a time, the first stage of our analysis involved combining information from the order items, orders, customers, sellers, products, category translation and geolocation datasets.

Before these joins could be carried out, we used a `select()` statement to keep only the columns that were needed later. For example, only five of the eight order columns and six of the product columns are taken into the joins.

This is consistent with the *projection pruning* feature of Spark, where Catalyst works backwards from the required output to determine which columns are actually needed and avoids carrying unnecessary columns further through the query. Our manual `select()` statements make this explicit in the notebook, although Catalyst can also remove unused columns when it optimises the plan.

Our delivered-order filter also reduces the data before the main joins for a similar reason. We used the `order_status = 'delivered'` filter together with the non-null delivery timestamp removes incomplete orders before they enter the rest of the pipeline.

We can see that Catalyst operates in a somewhat similar way through *predicate pushdown*, where it attempts to move filters as close to the source as possible without changing the result, but still reducing the rows passed into later joins.

How the joins are handled

The next stage to look at is the join stage. While there are several join algorithms available in Spark, the two most relevant to our query are broadcast joins and shuffle-based joins.

For smaller tables, such as the sellers, products, the category translation, and the reduced geolocation lookup tables, they can be explicitly wrapped in a broadcast join through `F.broadcast()`. Broadcasting sends a copy of the smaller table to the tasks working on the larger table, which avoids us having to redistribute both sides of the join by key.

On the other hand, larger joins such as the `order_items`, `orders` and `customers` ones are different. Since these tables contain more rows and use high-cardinality identifiers such as `order_id`, they are more likely to require Spark to use a shuffle-based strategy that redistributes rows by the join key.

Our geolocation lookup in particular is a useful example of preprocessing affecting the join strategy. The original geolocation file contains 1,000,163 rows because postcode prefixes appear repeatedly. We reduce this to 19,015 postcode-level records before joining. This step is necessary first to avoid duplicate matches, but it also makes the lookup much smaller. The reduced version can then be broadcast separately for the customer and seller postcode joins instead of joining the full geolocation table twice.

Aggregation and filtering

After the base view is created, we were then able to group the data by shipping lane and quarter.

For aggregates such as total freight, total weight and item count, Spark is able to calculate partial results within each partition before combining them after a shuffle. That is to say that the majority of the aggregation work can be completed locally

before data is moved between partitions, thereby reducing the amount of information that needs to be redistributed.

On the other hand, `countDistinct("order_id")` requires us to do some additional work because Spark needs to make sure an order is not counted more than once across partitions. It is retained because order count provides useful context alongside item count.

The minimum volume rule (`item_count >= 30`) is only applied after aggregation due to the fact that `item_count` does not exist until each lane-quarter group has been formed. It is therefore equivalent to a SQL `HAVING` condition rather than a row-level filter.

Adding the quarterly baseline

Our next step was to calculate the platform-wide quarterly baseline `base_weighted`, using the total freight divided by total weight for each quarter.

In our above analysis, we saw that this gave us a small quarter-level table, which we could then join back to the qualifying lane-quarter results. Because the baseline contains only one row per quarter, it could be explicitly broadcast when joined.

Importantly, keeping the baseline calculation separate is useful to us for two reasons:

1. It ensures the platform figure is calculated from all valid weighted rows rather than only from lane-quarter groups that pass the 30-item threshold.
2. The same quarterly values used in our descriptive output are also used to calculate `relative_to_quarter`, so the definition remains consistent throughout the analysis.

Applying the window functions

Once we reduce our data to just the qualifying lane-quarter rows, we can calculate the remaining comparisons using window functions.

In order for us to do thus, there are two different types of groupings which are required:

- `purchase_quarter` : this is used to rank lanes against other lanes active in the same quarter and to count the number of qualifying lanes in that period.
- `lane` : which is used to retrieve each lane's previous observed result using `LAG()`.

These operations require Spark to organise the data according to the relevant window key, and the ordered windows also require rows to be sorted within those groups. Because the ranking and temporal comparison use different partitioning keys, Spark may need to redistribute the aggregated data again after the earlier `groupBy`.

Effect of caching

As part of our analysis, we made the decision to cache the `base_view` table given that we needed to reuse it in several later sections of the notebook. A key feature of Spark is that once the cache has been set, Spark can simply read the prepared analytical data from memory rather than repeating the upstream joins, geolocation aggregation and derived-column calculations.

We note that this is likely to become particularly important for the DataFrame and SQL comparison and for the timed runs in Part B.2, where both implementations should begin from the same prepared input.

Because `base_view` is cached, the execution plan for later queries starts from the cached DataFrame rather than showing all of the joins used to create it. This means the downstream plan mainly shows the aggregation and window operations. The join behaviour used to build `base_view` therefore needs to be examined separately when interpreting the physical plan in Part B.3.

Part A.3: Spark SQL implementation

We note that in our earlier sections of this notebook, we have already built the analysis using the DataFrame API. Now, we want to recreate the same result using Spark SQL so that the two approaches can be compared directly in Part A.4.

In order for us to use SQL, the DataFrames we require need to be registered as temporary views first. In doing so, they are given a table name that can be referenced within SQL queries without creating or saving a separate dataset.

Both versions of our analysis use the same cached `base_view` that we defined earlier, which means they are able to both start from the same prepared data.

Here, the SQL query is split into several Common Table Expressions (CTEs) using `WITH`. Then, each CTE is able to handle one stage of the analysis, following the same general order as our previous DataFrame version:

- filter to rows with valid weight
- aggregate to lane-quarter level
- apply the 30-item minimum
- add the quarterly platform baseline
- calculate the ranking and previous-period comparisons

Keeping the SQL in these steps makes it easier for us to follow and also makes the logic easier to compare with our DataFrame implementation.

```
In [40]: # Make base_view available to Spark SQL
base_view.createOrReplaceTempView("base_view")

spark.sql("SHOW TABLES").show()
```

```

+-----+-----+-----+
|namespace|tableName|isTemporary|
+-----+-----+-----+
|          |base_view|          true|
+-----+-----+-----+

```

```

In [41]: sql_query = """
WITH weighted_items AS (
    SELECT *
    FROM base_view
    WHERE weight_kg > 0
),

quarter_baseline AS (
    SELECT
        purchase_quarter,
        SUM(freight_value) / SUM(weight_kg) AS quarter_baseline_fpk
    FROM weighted_items
    GROUP BY purchase_quarter
),

lane_quarter AS (
    SELECT
        lane,
        seller_state,
        customer_state,
        purchase_quarter,
        COUNT(*) AS item_count,
        COUNT(DISTINCT order_id) AS order_count,
        SUM(freight_value) AS total_freight,
        SUM(weight_kg) AS total_weight_kg,
        SUM(freight_value) / SUM(weight_kg) AS freight_per_kg,
        AVG(freight_per_kg) AS mean_item_freight_per_kg,
        AVG(freight_to_price_ratio) AS mean_freight_to_price,
        AVG(distance_km) AS mean_distance_km
    FROM weighted_items
    GROUP BY
        lane,
        seller_state,
        customer_state,
        purchase_quarter
    HAVING COUNT(*) >= 30
),

with_baseline AS (
    SELECT
        lq.*,
        qb.quarter_baseline_fpk,
        lq.freight_per_kg / qb.quarter_baseline_fpk AS relative_to_quarte
    FROM lane_quarter lq
    LEFT JOIN quarter_baseline qb
        ON lq.purchase_quarter = qb.purchase_quarter
)

SELECT
    lane,
    seller_state,
    customer_state,
    purchase_quarter,

```

```

item_count,
order_count,
total_freight,
total_weight_kg,
freight_per_kg,
mean_item_freight_per_kg,
mean_freight_to_price,
mean_distance_km,
quarter_baseline_fpk,
relative_to_quarter,

RANK() OVER (
    PARTITION BY purchase_quarter
    ORDER BY freight_per_kg DESC
) AS rank_in_quarter,

COUNT(*) OVER (
    PARTITION BY purchase_quarter
) AS lanes_in_quarter,

LAG(purchase_quarter) OVER (
    PARTITION BY lane
    ORDER BY purchase_quarter
) AS prev_quarter,

LAG(freight_per_kg) OVER (
    PARTITION BY lane
    ORDER BY purchase_quarter
) AS prev_freight_per_kg,

LAG(relative_to_quarter) OVER (
    PARTITION BY lane
    ORDER BY purchase_quarter
) AS prev_relative,

ROUND(
    100 * (
        freight_per_kg -
        LAG(freight_per_kg) OVER (
            PARTITION BY lane
            ORDER BY purchase_quarter
        )
    )
    /
    LAG(freight_per_kg) OVER (
        PARTITION BY lane
        ORDER BY purchase_quarter
    ),
    1
) AS change_pct,

relative_to_quarter -
LAG(relative_to_quarter) OVER (
    PARTITION BY lane
    ORDER BY purchase_quarter
) AS relative_change

FROM with_baseline

```

```
lane_quarter_sql = spark.sql(sql_query)

print("Qualifying lane-quarters (SQL):", lane_quarter_sql.count())
```

Qualifying lane-quarters (SQL): 348

```
In [42]: # Save the SQL result as a temporary view so it is easier to query
lane_quarter_sql.createOrReplaceTempView("lane_quarter_sql")

print("=== 2018-Q2: lanes ranked by freight charged per kilogram (SQL) ==")

spark.sql("""
    SELECT
        rank_in_quarter,
        lane,
        item_count,
        ROUND(freight_per_kg, 2) AS freight_per_kg,
        ROUND(mean_item_freight_per_kg, 2) AS mean_item_ratio,
        ROUND(relative_to_quarter, 2) AS relative_to_quarter,
        ROUND(mean_distance_km, 0) AS avg_dist_km
    FROM lane_quarter_sql
    WHERE purchase_quarter = '2018-Q2'
    ORDER BY rank_in_quarter
    LIMIT 15
""").show(truncate=False)
```

=== 2018-Q2: lanes ranked by freight charged per kilogram (SQL) ===

rank_in_quarter	lane	item_count	freight_per_kg	mean_item_ratio	relative_to_quarter
	avg_dist_km				
1	MA -> SP	53	61.07	67.54	6.16
2329.0					
2	PE -> MG	30	49.21	55.39	4.96
1700.0					
3	PE -> SP	35	41.06	50.92	4.14
2118.0					
4	RJ -> PE	30	36.17	62.69	3.65
1849.0					
5	SP -> RN	70	33.34	116.88	3.36
2271.0					
6	SP -> MA	85	24.4	78.59	2.46
2208.0					
7	RJ -> RS	45	23.97	44.39	2.42
1198.0					
8	SP -> PB	62	19.82	78.01	2.0
2177.0					
9	SP -> AL	45	19.6	69.58	1.98
1915.0					
10	RS -> RJ	50	19.51	55.37	1.97
1122.0					
11	RS -> MG	40	18.85	44.63	1.9
1321.0					
12	RJ -> MG	131	18.6	58.76	1.88
385.0					
13	RS -> PR	38	18.45	51.84	1.86
575.0					
14	SP -> CE	197	18.24	69.36	1.84
2251.0					
15	SP -> TO	47	18.11	75.59	1.83
1485.0					

```
In [43]: print("=== Largest movements relative to the quarterly platform baseline")

spark.sql("""
    SELECT
        lane,
        prev_quarter,
        purchase_quarter,
        item_count,
        ROUND(prev_relative, 2) AS prev_relative,
        ROUND(relative_to_quarter, 2) AS relative_to_quarter,
        ROUND(relative_change, 2) AS relative_change,
        change_pct,
        rank_in_quarter,
        lanes_in_quarter
    FROM lane_quarter_sql
    WHERE prev_relative IS NOT NULL
    ORDER BY ABS(relative_change) DESC
    LIMIT 15
""").show(truncate=False)
```

```

=== Largest movements relative to the quarterly platform baseline (SQL) ==
=
+-----+-----+-----+-----+-----+-----+
|lane      |prev_quarter|purchase_quarter|item_count|prev_relative|relative_
to_quarter|relative_change|change_pct|rank_in_quarter|lanes_in_quarter|
+-----+-----+-----+-----+-----+-----+
|SP -> T0|2017-Q2      |2017-Q4      |32      |1.19      |3.53
|2.34      |222.4      |1      |58      |
|SP -> T0|2017-Q4      |2018-Q1      |48      |3.53      |1.41
|-2.12      |-59.7      |19      |59      |
|SP -> RN|2018-Q1      |2018-Q2      |70      |1.87      |3.36
|1.5      |90.2      |5      |60      |
|SP -> MA|2017-Q2      |2017-Q3      |98      |2.99      |1.59
|-1.4      |-46.3      |11      |50      |
|SP -> RN|2017-Q3      |2017-Q4      |71      |3.04      |1.72
|-1.32      |-38.8      |9      |58      |
|SP -> AL|2017-Q2      |2017-Q3      |38      |2.64      |1.34
|-1.3      |-48.8      |16      |50      |
|SP -> R0|2017-Q3      |2018-Q1      |38      |2.25      |0.98
|-1.27      |-52.6      |45      |59      |
|PE -> SP|2018-Q1      |2018-Q2      |35      |5.39      |4.14
|-1.25      |-18.9      |3      |60      |
|SP -> RN|2017-Q2      |2017-Q3      |59      |1.8      |3.04
|1.24      |70.0      |1      |50      |
|RS -> RJ|2017-Q3      |2017-Q4      |51      |2.23      |1.01
|-1.23      |-51.3      |36      |58      |
|RJ -> PR|2018-Q2      |2018-Q3      |38      |1.53      |2.71
|1.18      |119.4      |2      |46      |
|SP -> PI|2017-Q4      |2018-Q1      |87      |2.36      |1.18
|-1.17      |-49.3      |31      |59      |
|RS -> SP|2017-Q1      |2017-Q2      |86      |3.08      |2.04
|-1.04      |-31.9      |5      |44      |
|RJ -> MG|2017-Q1      |2017-Q2      |46      |2.1      |1.07
|-1.03      |-47.6      |26      |44      |
|MA -> SP|2018-Q2      |2018-Q3      |55      |6.16      |5.14
|-1.02      |3.6      |1      |46      |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+

```

Notes on the SQL implementation

We can see immediately that there are a few key differences in the SQL version when comparing it to the DataFrame approach.

The CTEs follow the same steps as the analysis. Each **WITH** block handles one distinct part of the process:

- Filtering to usable rows
- Calculating the quarterly baseline
- Aggregating to lane-quarter level
- Applying the minimum volume rule
- Joining the baseline back on

The final `SELECT` then adds the ranking and previous-period comparisons, making the SQL version easier to compare directly with the DataFrame implementation.

There are 3 key aspects in our comparison of the DataFrame and SQL versions that we should highlight:

1. The post-aggregation filter: In the DataFrame version, the 30-item minimum is applied using `.filter()` after `.agg()`. However, in SQL, we are able to implement the same rule using `HAVING`, which makes it immediately clear to a reader that the condition applies to the grouped result rather than to individual rows.

2. Repeated window expression: In the final `SELECT` call, `LAG(freight_per_kg)` is used both as an output column and then again when calculating `change_pct`. This is due to the fact that the alias created in the same `SELECT` cannot be reused immediately, and so the window expression appears more than once. In this regard, the DataFrame version is a little easier to read.

3. Minimum threshold definition: The DataFrame version uses the `MIN_ITEMS` variable, while the SQL query contains the value `30` in the `HAVING` clause. This works fine for our current analysis, but changing the threshold would require editing the SQL text unless the query was built using a variable.

Comparing the DataFrame API and Spark SQL

Now that we have implemented the same analysis using both approaches, our next step is to consider a side by side comparison of how they align, and also how they differ in practice.

The fundamental difference we need to consider here is not what they are capable of doing, since it is clear that both the DataFrame API and Spark SQL are capable of the analysis (and also, they are processed by the same Catalyst optimiser). Instead, we are concerned with the key differences in how the analysis is written, followed and changed.

Where the DataFrame API works well

Looking first at the DataFrame version, it is important to note the relative ease at which intermediate columns could be created and reused at later points. For example, `prev_freight_per_kg` was created once and then we could use it again later for `change_pct`. On the other hand, in our SQL query, the `LAG()` expression needed to be repeated in the final `SELECT`, which makes that section longer and means the same logic needs to be redefined in several places.

Parameters are also easier to keep separate from the analysis under the DataFrame version. We were able to define `MIN_ITEMS = 30` once and then leverage that defined reference, which is important since by having a named variable, it makes it easier to later test different thresholds.

The DataFrame approach was also useful while building our analysis. We were able to separately inspect objects such as `base_weighted`, `lane_quarter_sizes` and `lane_quarter`, which made it straightforward to check intermediate results and decide on the minimum group size.

Where Spark SQL works well

While the DataFrame version certainly offers key advantages, it important to also note that the SQL version gives a clearer view of the finished analysis as a whole, which can be a significant benefit when conducting complex queries.

In particular, the CTEs divide the query into named stages: `weighted_items`, `quarter_baseline`, `lane_quarter` and `with_baseline`, before the window calculations are applied. This makes it possible to follow most of the final analytical pipeline from top to bottom in one place.

Furthermore, the distinction between row-level and group-level filtering is also clearer. SQL uses `WHERE` for filtering individual rows and `HAVING` for filtering after aggregation. In the DataFrame version both are written using `.filter()`, with their position in the transformation sequence determining when they are applied.

SQL also utilises codig syntax and language that is arguably a much more intuitive to learn, making it a very accessible option for newer developers.

Which approach suited this analysis?

For our specific use case, the DataFrame API was more useful while we were developing and testing the analysis. We were able to build the pipeline gradually, inspect intermediate results and adjust individual steps without rewriting the whole query.

Once our logic had been settled, the SQL version provided a more compact view of the finished process because the CTEs keep each stage together in a single query.

Given this, it arguably makes little sense for us to choose one approach purely on the basis of performance. This is due to that fact that when considering the question of equivalency, both use Spark's underlying execution engine, so readability and the type of work being done are more useful reasons for choosing between them.

However, one downside of maintaining both versions is that any future change to the analysis needs to be made twice. Therefore, the equivalence check in Part A.4 of our analysis is critical when testing because it confirms that the two implementations currently produce the same result.

Part A.4: Results validation

Now that we have implemented our analysis using both the DataFrame API and Spark SQL, we need to check that they actually produce the same result.

Since differences could exist anywhere in the outputs or could even be hidden by rounding differences, simply looking at the summary statistics, or even just the top few rows is not sufficient. We therefore want to compare the full results rather than relying on a simple visual check.

As such, our validation aims to look at four things:

1. Both outputs have the same number of rows, the same lane-quarter keys and matching column types.
2. Rows from one result can be matched to the other using the full, unrounded values.
3. Numeric columns are comparable, with some small allowances made for a small floating-point difference in case tiny calculation differences appear.
4. After applying the same ordering, the two complete outputs match.

Before running these checks, we need to first ensure that both results are reduced to the same set of columns in the same order so that we are comparing like with like.

```
In [44]: # Use the same columns from both results
compare_cols = [
    "lane",
    "purchase_quarter",
    "item_count",
    "order_count",
    "total_freight",
    "total_weight_kg",
    "freight_per_kg",
    "mean_item_freight_per_kg",
    "quarter_baseline_fpk",
    "relative_to_quarter",
    "rank_in_quarter",
    "lanes_in_quarter",
    "prev_quarter",
    "prev_freight_per_kg",
    "prev_relative",
    "change_pct",
    "relative_change"
]

df_result = lane_quarter_ranked.select(compare_cols)
sql_result = lane_quarter_sql.select(compare_cols)
```

```
In [45]: # Check the basic shape of both outputs
df_rows = df_result.count()
sql_rows = sql_result.count()

print("DataFrame rows:", df_rows)
print("SQL rows:", sql_rows)
print("Column types match:", df_result.dtypes == sql_result.dtypes)

# Check that both contain the same lane-quarter keys
```

```

df_keys = df_result.select(
    "lane",
    "purchase_quarter"
).distinct()

sql_keys = sql_result.select(
    "lane",
    "purchase_quarter"
).distinct()

print("DataFrame keys:", df_keys.count())
print("SQL keys:", sql_keys.count())

print(
    "Keys only in DataFrame:",
    df_keys.exceptAll(sql_keys).count()
)

print(
    "Keys only in SQL:",
    sql_keys.exceptAll(df_keys).count()
)

```

DataFrame rows: 348
 SQL rows: 348
 Column types match: True
 DataFrame keys: 348
 SQL keys: 348
 Keys only in DataFrame: 0
 Keys only in SQL: 0

```

In [46]: # Check whether any complete rows differ
df_only = df_result.exceptAll(sql_result)
sql_only = sql_result.exceptAll(df_result)

print(
    "Rows in DataFrame result but not SQL:",
    df_only.count()
)

print(
    "Rows in SQL result but not DataFrame:",
    sql_only.count()
)

```

Rows in DataFrame result but not SQL: 0
 Rows in SQL result but not DataFrame: 0

```

In [47]: # Match rows by lane and quarter so the numeric values can be compared
matched = df_result.alias("d").join(
    sql_result.alias("s"),
    on=["lane", "purchase_quarter"],
    how="inner"
)

# Compare the numeric columns for each matched lane-quarter
diffs = matched.select(
    F.max(
        F.abs(F.col("d.total_freight") - F.col("s.total_freight"))
    ).alias("total_freight"),

```

```
F.max(
  F.abs(F.col("d.total_weight_kg") - F.col("s.total_weight_kg"))
).alias("total_weight_kg"),

F.max(
  F.abs(F.col("d.freight_per_kg") - F.col("s.freight_per_kg"))
).alias("freight_per_kg"),

F.max(
  F.abs(
    F.col("d.mean_item_freight_per_kg")
    - F.col("s.mean_item_freight_per_kg")
  )
).alias("mean_item_freight_per_kg"),

F.max(
  F.abs(
    F.col("d.quarter_baseline_fpk")
    - F.col("s.quarter_baseline_fpk")
  )
).alias("quarter_baseline_fpk"),

F.max(
  F.abs(
    F.col("d.relative_to_quarter")
    - F.col("s.relative_to_quarter")
  )
).alias("relative_to_quarter"),

F.max(
  F.abs(
    F.col("d.prev_freight_per_kg")
    - F.col("s.prev_freight_per_kg")
  )
).alias("prev_freight_per_kg"),

F.max(
  F.abs(
    F.col("d.prev_relative")
    - F.col("s.prev_relative")
  )
).alias("prev_relative"),

F.max(
  F.abs(F.col("d.change_pct") - F.col("s.change_pct"))
).alias("change_pct"),

F.max(
  F.abs(
    F.col("d.relative_change")
    - F.col("s.relative_change")
  )
).alias("relative_change")
)

diffs.show(vertical=True, truncate=False)
```

```

--RECORD 0-----
total_freight          | 2.764863893389702E-10
total_weight_kg        | 3.2741809263825417E-11
freight_per_kg         | 3.108624468950438E-14
mean_item_freight_per_kg | 5.6843418860808015E-14
quarter_baseline_fpk   | 0.0
relative_to_quarter    | 3.3306690738754696E-15
prev_freight_per_kg    | 3.108624468950438E-14
prev_relative          | 3.3306690738754696E-15
change_pct             | 0.0
relative_change        | 3.4416913763379853E-15

```

```

In [48]: # Find the largest difference across all numeric columns
diff_values = diffs.collect()[0]

max_diff = max(
    value for value in diff_values
    if value is not None
)

print("Largest difference in any numeric column:", max_diff)
print("Within tolerance of 1e-9:", max_diff < 1e-9)

```

```

Largest difference in any numeric column: 2.764863893389702e-10
Within tolerance of 1e-9: True

```

```

In [49]: # Final check after putting both results in the same order
df_ordered = df_result.orderBy(
    "purchase_quarter",
    "lane"
)

sql_ordered = sql_result.orderBy(
    "purchase_quarter",
    "lane"
)

same_results = df_ordered.collect() == sql_ordered.collect()

print("Ordered results identical:", same_results)

```

```

Ordered results identical: True

```

Results validation

In order to validate where our results from the DataFrame and SQL queries align as expected, there were four key checks that we performed:

Shape (check 1). We wanted to ensure that both results contained the same number of rows. And from our checks, we can see that this is the case, with both containing 348 rows that covered the same 348 lane-quarter keys. So we can confirm from this check that there were no keys which appeared in only one result, and the column types also matched exactly.

Exact comparison (check 2). We can see here that `exceptAll` returned zero rows in both directions, meaning that the two implementations produced the same

complete rows. Because `exceptAll` also preserves duplicates, it also checked both the values and the number of times each row appears. Again, this result shows complete alignment, as we were expecting.

Numeric tolerance (check 3). Our third check was to do a numeric comparison. In this check, we found very small differences in several columns, with the largest being approximately 2.8×10^{-10} in `total_freight`.

As we discussed earlier, these small differences are consistent with normal floating-point behaviour in Spark. More specifically, aggregations such as `SUM()` are calculated in parts across partitions and then combined. Also, floating-point addition cannot be perfectly associative as it will always contain some small level of rounding, so combining the same values in a slightly different order can produce differences in the final decimal places.

This also explains why the exact comparison and tolerance comparison can appear inconsistent. `lane_quarter_ranked` is cached, so its values remained fixed after materialisation, while the SQL result is recalculated when a new action is called. Different actions can therefore produce tiny floating-point differences even though the underlying calculation is unchanged.

Ordered comparison (check 4). We know that Spark does not guarantee that rows will be returned in the same order each time a DataFrame is evaluated. This means that the DataFrame and SQL versions could contain exactly the same rows but list them in a different sequence. So that we can avoid treating a difference in row order as a difference in the actual results, we want to first sort both of the by `purchase_quarter` and `lane`.

In this check, we can see that once the same ordering was applied, the collected rows from the two implementations are identical.

Part B.1: Partitioning strategy

In this section, we want to compare hash and range partitioning using `customer_zip_code_prefix`, which is the high-cardinality column that we identified in our dataset proposal, and we want to apply both approaches to the same `base_view` that we used in our earlier analysis.

In **hash partitioning**, we look to use the postcode prefix to determine which partition each row belongs to. Here, the rows with the same postcode prefix are kept together, but similar or neighbouring postcode values can be spread across different partitions.

On the other hand, **range partitioning** looks to instead divide the postcode values into ranges. This means rows with similar postcode prefixes are more likely to be placed in the same partition.

Choosing the number of partitions

For both approaches, we want to use **8 partitions** so that the comparison is consistent.

Since our Spark environment has four available task slots, eight partitions gives Spark more tasks than it can run at once while still keeping each partition reasonably large. With approximately 110,000 rows, this works out to roughly 13,800 rows per partition on average.

Furthermore, eight partitions also gives us more detail when comparing the two strategies than if we were to only use four. With four partitions, each partition would cover a much larger share of the data and differences in how hash and range partitioning distribute the rows would likely be much harder to see. At the same time, eight is still small enough that the resulting partition sizes are easy to inspect and compare.

```
In [50]: N_PARTITIONS = 8

# Partition the same data in two different ways
hash_df = base_view.repartition(
    N_PARTITIONS,
    "customer_zip_code_prefix"
)

range_df = base_view.repartitionByRange(
    N_PARTITIONS,
    "customer_zip_code_prefix"
)
```

```
In [51]: # Get some information about each partition
def partition_profile(df):

    profile = df.withColumn(
        "partition_id",
        F.spark_partition_id()
    )

    profile = profile.groupBy(
        "partition_id"
    ).agg(
        F.count("*").alias("records"),
        F.countDistinct("customer_zip_code_prefix").alias("distinct_prefix"),
        F.min("customer_zip_code_prefix").alias("min_prefix"),
        F.max("customer_zip_code_prefix").alias("max_prefix"),
        F.countDistinct("customer_state").alias("states"),
        F.round(
            100 * F.avg(
                (F.col("customer_state") == "SP").cast("int")
            ),
            1
        ).alias("pct_SP")
    )

    return profile.orderBy("partition_id").toPandas()
```

```
In [52]: hash_profile = partition_profile(hash_df)
range_profile = partition_profile(range_df)

hash_profile["pct_of_rows"] = (
    100 * hash_profile["records"] / hash_profile["records"].sum()
).round(1)

range_profile["pct_of_rows"] = (
    100 * range_profile["records"] / range_profile["records"].sum()
).round(1)
```

```
In [53]: print("=== Hash partitioning ===")
display(hash_profile)

print("=== Range partitioning ===")
display(range_profile)
```

=== Hash partitioning ===

	partition_id	records	distinct_prefixes	min_prefix	max_prefix	states	pct_SP	
0	0	14141	1842	1003	99970	27	42.8	
1	1	14436	1895	1004	99960	26	39.3	
2	2	14223	1849	1007	99925	27	43.7	
3	3	13721	1889	1008	99980	25	41.3	
4	4	13239	1847	1017	99965	27	40.8	
5	5	13767	1878	1006	99855	27	41.8	
6	6	13760	1859	1013	99955	27	43.5	
7	7	12902	1830	1005	99940	27	44.1	

=== Range partitioning ===

	partition_id	records	distinct_prefixes	min_prefix	max_prefix	states	pct_SP	
0	0	13222	2263	1003	5136	1	100.0	
1	1	14267	1603	5138	11095	1	100.0	
2	2	14247	1123	11250	15910	1	100.0	
3	3	14539	1202	15915	25561	2	32.4	
4	4	12253	1330	25565	35530	3	0.0	
5	5	14418	2513	35534	59518	7	0.0	
6	6	14490	3012	59528	85301	16	0.0	
7	7	12753	1843	85303	99980	3	0.0	

```
In [54]: # Compare how balanced the partition sizes are
hash_records = hash_profile["records"]
```

```

range_records = range_profile["records"]

hash_mean = hash_records.mean()
range_mean = range_records.mean()

summary = pd.DataFrame([
    {
        "strategy": "hash",
        "min_records": hash_records.min(),
        "max_records": hash_records.max(),
        "mean_records": round(hash_mean, 1),
        "std_dev": round(hash_records.std(), 1),
        "max_to_mean": round(hash_records.max() / hash_mean, 2),
        "coeff_of_variation": round(hash_records.std() / hash_mean, 3)
    },
    {
        "strategy": "range",
        "min_records": range_records.min(),
        "max_records": range_records.max(),
        "mean_records": round(range_mean, 1),
        "std_dev": round(range_records.std(), 1),
        "max_to_mean": round(range_records.max() / range_mean, 2),
        "coeff_of_variation": round(range_records.std() / range_mean, 3)
    }
])

summary

```

Out [54]:

	strategy	min_records	max_records	mean_records	std_dev	max_to_mean	cc
0	hash	12902	14436	13773.6	510.5	1.05	
1	range	12253	14539	13773.6	897.6	1.06	

Discussion

Distribution shape

From our analysis, we can see that both approaches produce fairly even partition sizes. The hash partitions contain between 12,902 and 14,436 rows, while the range partitions contain between 12,253 and 14,539. They also have the similar maximum-to-mean ratio of 1.06 and 1.05, respectively. Furthermore, the range partitioning has coefficient of variation of 0.065 compared with 0.037 for hash partitioning.

What the partitions contain

With range partitioning, nearby postcode prefixes are kept together, making the concentration of São Paulo more visible, with partitions 0 to 2 containing only São Paulo records, while later partitions cover much wider postcode ranges.

Hash partitioning on the other hand produces a different pattern. Each partition contains postcode prefixes from almost the full range. So, the geographic concentration in the original data is spread across the hash partitions rather than kept together.

Which strategy is more suitable

Hash partitioning is particularly useful when rows need to be grouped using an exact key. Range partitioning however, is more useful when we might need to preserve the ordering or locality of the key, because similar postcode values remain together.

For our analysis, the final aggregation is based on shipping lane and quarter rather than `customer_zip_code_prefix`. This means neither strategy matches our needs exactly.

Skew and performance

The main concentration in our dataset is at state level. Thus, if a heavily skewed field such as `customer_state` were used as a partitioning key, a large share of the records could end up assigned to the same task and slow the overall stage.

One approach we could consider to address this is *key salting*, where a large key is temporarily split across several values so that its records can be processed by multiple tasks and then combined afterwards.

Part B.2: Execution time benchmarking

In this section, we want to compare the execution time of the DataFrame and Spark SQL versions of our analysis.

Execution environment

All timings were recorded in Spark *local mode* on a 2018 MacBook Air with a 1.6 GHz dual-core Intel Core i5 and four logical cores. As we mentioned at the start of our notebook, Spark uses the local machine rather than separate executors, with `spark.driver.memory` set to 3 GB. Adaptive Query Execution is disabled and `spark.sql.shuffle.partitions` is set to 4.

As such, these timings are specific to this machine and would likely differ when replicated on other hardware or on a Spark cluster.

What is being timed

Each run starts from the same cached `base_view` we created earlier, and executes the analysis through to the final result.

Importantly, that means that this excludes CSV loading and the joins used to construct the base view from both implementations. The benchmark instead focuses on the steps written separately in the DataFrame and SQL versions: filtering, lane-quarter aggregation, the minimum-volume rule, the quarterly baseline join and the window calculations.

Each run also ends with `.collect()` to force Spark to evaluate the complete result. And for the DataFrame version, our Part A.2 logic is placed inside a function so that a new query is created for each timed run.

Controlling the benchmark

The cached `lane_quarter_ranked` result from Part A.2 is released before timing so that it cannot be reused during the benchmark. `base_view` remains cached so that both implementations begin from the same prepared input.

Additionally, the DataFrame and SQL runs are alternated to reduce the chance that changes in machine conditions affect only one implementation. Finally, wall-clock time is used for the comparison because most of the Spark processing occurs in the JVM rather than in Python. Three runs are recorded for each implementation, and the **median** is reported so that one unusually slow run has less influence on the final result.

```
In [55]: # Remove the cached final result before benchmarking
lane_quarter_ranked.unpersist(blocking=True)

# Reset these if the benchmark needs to be run again
timings = {
    "DataFrame": [],
    "Spark SQL": []
}
```

```
In [56]: def build_dataframe_query():

    # Start with rows that have a valid weight
    weighted = base_view.filter(
        F.col("weight_kg") > 0
    )

    # Platform baseline for each quarter
    baseline = weighted.groupBy(
        "purchase_quarter"
    ).agg(
        (
            F.sum("freight_value") / F.sum("weight_kg")
        ).alias("quarter_baseline_fpk")
    )

    # Aggregate to lane-quarter level
    result = weighted.groupBy(
        "lane",
        "seller_state",
        "customer_state",
        "purchase_quarter"
    ).agg(
        F.count("*").alias("item_count"),
        F.countDistinct("order_id").alias("order_count"),
        F.sum("freight_value").alias("total_freight"),
        F.sum("weight_kg").alias("total_weight_kg"),
        F.avg("freight_per_kg").alias("mean_item_freight_per_kg"),
        F.avg("freight_to_price_ratio").alias("mean_freight_to_price"),
        F.avg("distance_km").alias("mean_distance_km")
    )
```

```
)

result = result.withColumn(
    "freight_per_kg",
    F.col("total_freight") / F.col("total_weight_kg")
)

result = result.filter(
    F.col("item_count") >= MIN_ITEMS
)

# Add the platform baseline
result = result.join(
    F.broadcast(baseline),
    on="purchase_quarter",
    how="left"
)

result = result.withColumn(
    "relative_to_quarter",
    F.col("freight_per_kg") / F.col("quarter_baseline_fpk")
)

# Ranking and previous-period values
result = result.withColumn(
    "rank_in_quarter",
    F.rank().over(quarter_window)
)

result = result.withColumn(
    "lanes_in_quarter",
    F.count("*").over(quarter_totals)
)

result = result.withColumn(
    "prev_quarter",
    F.lag("purchase_quarter").over(lane_window)
)

result = result.withColumn(
    "prev_freight_per_kg",
    F.lag("freight_per_kg").over(lane_window)
)

result = result.withColumn(
    "prev_relative",
    F.lag("relative_to_quarter").over(lane_window)
)

result = result.withColumn(
    "change_pct",
    F.round(
        100 * (
            F.col("freight_per_kg") - F.col("prev_freight_per_kg")
        ) / F.col("prev_freight_per_kg"),
        1
    )
)

result = result.withColumn(
```

```

        "relative_change",
        F.col("relative_to_quarter") - F.col("prev_relative")
    )

    return result

```

```

In [57]: def build_sql_query():
        return spark.sql(sql_query)

```

```

In [58]: %%time

# DataFrame - run 1
start = time.perf_counter()

rows = build_dataframe_query().collect()

elapsed = time.perf_counter() - start
timings["DataFrame"].append(round(elapsed, 3))

print("Rows returned:", len(rows))
print("Recorded time:", round(elapsed, 3), "seconds")

```

```

[Stage 274:>
4) / 4] (0 +
Rows returned: 348
Recorded time: 2.508 seconds
CPU times: user 36.9 ms, sys: 14.2 ms, total: 51.1 ms
Wall time: 2.51 s

```

```

In [59]: %%time

# Spark SQL - run 1
start = time.perf_counter()

rows = build_sql_query().collect()

elapsed = time.perf_counter() - start
timings["Spark SQL"].append(round(elapsed, 3))

print("Rows returned:", len(rows))
print("Recorded time:", round(elapsed, 3), "seconds")

```

```

Rows returned: 348
Recorded time: 1.774 seconds
CPU times: user 9.67 ms, sys: 4.38 ms, total: 14.1 ms
Wall time: 1.77 s

```

```

In [60]: %%time

# DataFrame - run 2
start = time.perf_counter()

rows = build_dataframe_query().collect()

elapsed = time.perf_counter() - start
timings["DataFrame"].append(round(elapsed, 3))

```

```
print("Rows returned:", len(rows))
print("Recorded time:", round(elapsed, 3), "seconds")
```

Rows returned: 348
Recorded time: 1.976 seconds
CPU times: user 38 ms, sys: 15.4 ms, total: 53.4 ms
Wall time: 1.98 s

```
In [61]: %%time

# Spark SQL - run 2
start = time.perf_counter()

rows = build_sql_query().collect()

elapsed = time.perf_counter() - start
timings["Spark SQL"].append(round(elapsed, 3))

print("Rows returned:", len(rows))
print("Recorded time:", round(elapsed, 3), "seconds")
```

Rows returned: 348
Recorded time: 1.387 seconds
CPU times: user 9.78 ms, sys: 4.16 ms, total: 13.9 ms
Wall time: 1.39 s

```
In [62]: %%time

# DataFrame - run 3
start = time.perf_counter()

rows = build_dataframe_query().collect()

elapsed = time.perf_counter() - start
timings["DataFrame"].append(round(elapsed, 3))

print("Rows returned:", len(rows))
print("Recorded time:", round(elapsed, 3), "seconds")
```

Rows returned: 348
Recorded time: 1.596 seconds
CPU times: user 34.7 ms, sys: 13.9 ms, total: 48.6 ms
Wall time: 1.6 s

```
In [63]: %%time

# Spark SQL - run 3
start = time.perf_counter()

rows = build_sql_query().collect()

elapsed = time.perf_counter() - start
timings["Spark SQL"].append(round(elapsed, 3))

print("Rows returned:", len(rows))
print("Recorded time:", round(elapsed, 3), "seconds")
```

Rows returned: 348
Recorded time: 1.379 seconds
CPU times: user 8.66 ms, sys: 3.66 ms, total: 12.3 ms
Wall time: 1.38 s

```
In [64]: df_times = timings["DataFrame"]
sql_times = timings["Spark SQL"]

benchmark = pd.DataFrame([
    {
        "API": "DataFrame",
        "Run 1": df_times[0],
        "Run 2": df_times[1],
        "Run 3": df_times[2],
        "Median": round(median(df_times), 3)
    },
    {
        "API": "Spark SQL",
        "Run 1": sql_times[0],
        "Run 2": sql_times[1],
        "Run 3": sql_times[2],
        "Median": round(median(sql_times), 3)
    }
])

benchmark
```

```
Out [64]:
```

	API	Run 1	Run 2	Run 3	Median
0	DataFrame	2.508	1.976	1.596	1.976
1	Spark SQL	1.774	1.387	1.379	1.387

Discussion

We can see that the two implementations perform very similarly. The median execution time was *1.976 seconds* for the DataFrame API and *1.387 seconds* for Spark SQL, a difference of less than 0.6 seconds. Importantly, this is smaller than the variation between repeated runs, so we should not treat it as a meaningful performance difference.

This is consistent with Spark's execution model. Both interfaces are passed through the **Catalyst optimiser**, which converts them into optimised logical and physical plans before execution. As a result, equivalent DataFrame and SQL queries should perform essentially the same joins, aggregations, exchanges and window operations regardless of how the query was written.

Both versions also start from the same cached `base_view`, so they inherit the same upstream data and avoid repeating the earlier join lineage. As such, the most complex work in the benchmark is therefore the shuffle and sort activity which is required by the aggregation and window functions. Since both implementations process the same rows using the same partition settings, their shuffle volume and JVM-side serialisation costs should also be very similar.

We should also note that the first DataFrame run at *2.508 seconds* is the main outlier here. However, this is likely to be a reflection of one-off costs such as JVM warm-up, Spark code generation or task initialisation rather than a consistent API difference.

However, by reporting the median, we can account for the impact of this single slower run.

We note that the DataFrame runs also used slightly more Python CPU time, likely due to the fact that constructing the query requires several Python-to-JVM calls.

However, no Python UDFs were used, so the row-level processing remains inside Spark's JVM engine and this overhead is very small compared with the overall execution time.

Part B.3: Execution plan interpretation

In this section, we want to examine the execution plan for the lane-quarter aggregation that we did earlier in Part A.2, including the join to the quarterly platform baseline.

We want to use this part of the analysis because it contains both aggregation and join operations, and it also allows us to better understand how some of the Spark behaviour that we discussed earlier works in practice. In particular, we want to see how Spark handles `countDistinct("order_id")` and also whether the small quarterly baseline is broadcast as expected rather than being redistributed.

By calling `explain(extended=True)`, we are able to see four versions of the plan:

- The **parsed logical plan**, which represents the operations that were written by us
- The **analysed logical plan**, where Spark had resolved the columns and data types
- The **optimised logical plan**, after Catalyst had simplified and rearranged the query
- The **physical plan**, which shows the specific algorithms Spark plans to use when the query actually gets executed

We should also note that since Adaptive Query Execution is disabled in our Spark configuration, Spark will not revise the physical plan during execution based on runtime statistics. This makes the printed plan easier to compare with the execution behaviour discussed in this section and also the DAG that we will see in Part B.4.

Before printing the plan, we want to first increase

`spark.sql.debug.maxToStringFields` so that Spark does not shorten any of the longer column lists, thereby ensuring that the plan is shown in full.

```
In [65]: # Stop Spark abbreviating long column lists in the plan output
spark.conf.set("spark.sql.debug.maxToStringFields", 200)

lane_quarter.explain(extended=True)
```

== Parsed Logical Plan ==

```
'Project [purchase_quarter#775, lane#748, seller_state#51, customer_state#42, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_distance_km#8005, freight_per_kg#8018, quarter_baseline_fpk#8548, ('freight_per_kg / 'quarter_baseline_fpk) AS relative_to_quarter#9137]
+- Project [purchase_quarter#775, lane#748, seller_state#51, customer_state#42, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_distance_km#8005, freight_per_kg#8018, quarter_baseline_fpk#8548]
  +- Join LeftOuter, (purchase_quarter#775 = purchase_quarter#9116)
    :- Filter (item_count#7994L >= cast(30 as bigint))
      : +- Project [lane#748, seller_state#51, customer_state#42, purchase_quarter#775, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_distance_km#8005, (total_freight#7997 / total_weight_kg#7999) AS freight_per_kg#8018]
        : +- Aggregate [lane#748, seller_state#51, customer_state#42, purchase_quarter#775], [lane#748, seller_state#51, customer_state#42, purchase_quarter#775, count(1) AS item_count#7994L, count(distinct order_id#8) AS order_count#7995L, sum(freight_value#14) AS total_freight#7997, sum(weight_kg#803) AS total_weight_kg#7999, avg(freight_per_kg#832) AS mean_item_freight_per_kg#8001, avg(freight_to_price_ratio#862) AS mean_freight_to_price#8003, avg(distance_km#925) AS mean_distance_km#8005]
          : +- Filter (weight_kg#803 > cast(0 as double))
            : +- Project [order_id#8, order_item_id#9, lane#748, seller_state#51, customer_state#42, customer_zip_code_prefix#40, seller_zip_code_prefix#49, purchase_quarter#775, order_purchase_timestamp#25, price#13, freight_value#14, weight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, is_intrastate#958, product_category_name#71, product_category_name_english#67]
              : +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, weight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, (seller_state#51 = customer_state#42) AS is_intrastate#958]
                : +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, weight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, package_volume_cm3#893, ((6371.0 * cast(2 as double)) * ASIN(SQRT((POWER(SIN((RADIANS((cust_lat#683 - sell_lat#689)) / cast(2 as double))), cast(2 as double)) + ((COS(RADIANS(sell_lat#689)) * COS(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS((cust_lng#684 - sell_lng#690)) / cast(2 as double))), cast(2 as double)))))) AS distance_km#925]
                  : +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, weight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, (seller_state#51 = customer_state#42) AS is_intrastate#958]
```

```

tomer_date#28, order_estimated_delivery_date#29, customer_state#42, seller
_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#7
7, product_width_cm#78, product_category_name_english#67, cust_lat#683, cu
st_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, we
ight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, ((product_len
gth_cm#76 * product_height_cm#77) * product_width_cm#78) AS package_volume
_cm3#893]
:
+- Project [seller_zip_code_prefix#49, custo
mer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id
#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12,
price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_c
ustomer_date#28, order_estimated_delivery_date#29, customer_state#42, sell
er_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#
77, product_width_cm#78, product_category_name_english#67, cust_lat#683, c
ust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, w
eight_kg#803, freight_per_kg#832, CASE WHEN (price#13 > cast(0 as double))
THEN (freight_value#14 / price#13) END AS freight_to_price_ratio#862]
:
+- Project [seller_zip_code_prefix#49, cu
stomer_zip_code_prefix#40, product_category_name#71, product_id#10, seller
_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#1
2, price#13, freight_value#14, order_purchase_timestamp#25, order_delivere
d_customer_date#28, order_estimated_delivery_date#29, customer_state#42, s
eller_state#51, product_weight_g#75, product_length_cm#76, product_height_
cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#68
3, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#77
5, weight_kg#803, CASE WHEN (weight_kg#803 > cast(0 as double)) THEN (frei
ght_value#14 / weight_kg#803) END AS freight_per_kg#832]
:
+- Project [seller_zip_code_prefix#49,
customer_zip_code_prefix#40, product_category_name#71, product_id#10, sell
er_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date
#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delive
red_customer_date#28, order_estimated_delivery_date#29, customer_state#42,
seller_state#51, product_weight_g#75, product_length_cm#76, product_height
_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#68
3, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#77
5, (cast(product_weight_g#75 as double) / cast(1000.0 as double)) AS weigh
t_kg#803]
:
+- Project [seller_zip_code_prefix#49,
customer_zip_code_prefix#40, product_category_name#71, product_id#10,
seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_
date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_de
livered_customer_date#28, order_estimated_delivery_date#29, customer_state
#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_h
eight_cm#77, product_width_cm#78, product_category_name_english#67, cust_l
at#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, concat_ws(-, c
ast(year(cast(order_purchase_timestamp#25 as date)) as string), concat(0,
cast(quarter(cast(order_purchase_timestamp#25 as date)) as string))) AS pu
rchase_quarter#775]
:
+- Project [seller_zip_code_pref
ix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#1
0, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_lim
it_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order
_delivered_customer_date#28, order_estimated_delivery_date#29, customer_st
ate#42, seller_state#51, product_weight_g#75, product_length_cm#76, produc
t_height_cm#77, product_width_cm#78, product_category_name_english#67, cus
t_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, concat_ws(->, selle
r_state#51, customer_state#42) AS lane#748]
:
+- Project [seller_zip_code_p
refix#49, customer_zip_code_prefix#40, product_category_name#71, product_i
d#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_

```

```

limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, or
der_delivered_customer_date#28, order_estimated_delivery_date#29, customer
_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, pro
duct_height_cm#77, product_width_cm#78, product_category_name_english#67,
cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690]
:                                     +- Join LeftOuter, (seller
_zip_code_prefix#49 = seller_zip_code_prefix#688)
:                                     :- Project [customer_zi
p_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, c
ustomer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#
13, freight_value#14, order_purchase_timestamp#25, order_delivered_custome
r_date#28, order_estimated_delivery_date#29, customer_state#42, seller_sta
te#51, seller_zip_code_prefix#49, product_weight_g#75, product_length_cm#7
6, product_height_cm#77, product_width_cm#78, product_category_name_englis
h#67, cust_lat#683, cust_lng#684]
:                                     : +- Join LeftOuter,
(customer_zip_code_prefix#40 = customer_zip_code_prefix#682)
:                                     : :- Project [produ
ct_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id
#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, o
rder_purchase_timestamp#25, order_delivered_customer_date#28, order_estima
ted_delivery_date#29, customer_state#42, customer_zip_code_prefix#40, sell
er_state#51, seller_zip_code_prefix#49, product_weight_g#75, product_lengt
h_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_
english#67]
:                                     : : +- Join LeftOu
ter, (product_category_name#71 = product_category_name#66)
:                                     : : :- Project
[product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9,
shipping_limit_date#12, price#13, freight_value#14, order_purchase_timesta
mp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29,
customer_state#42, customer_zip_code_prefix#40, seller_state#51, seller_zi
p_code_prefix#49, product_category_name#71, product_weight_g#75, product_l
ength_cm#76, product_height_cm#77, product_width_cm#78]
:                                     : : : +- Join
LeftOuter, (product_id#10 = product_id#70)
:                                     : : : :- Pr
oject [seller_id#11, customer_id#23, order_id#8, order_item_id#9, product_
id#10, shipping_limit_date#12, price#13, freight_value#14, order_purchase_
timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_d
ate#29, customer_state#42, customer_zip_code_prefix#40, seller_state#51, s
eller_zip_code_prefix#49]
:                                     : : : : +-
Join Inner, (seller_id#11 = seller_id#48)
:                                     : : : : :- Project [customer_id#23, order_id#8, order_item_id#9, product_id#10, se
ller_id#11, shipping_limit_date#12, price#13, freight_value#14, order_purc
hase_timestamp#25, order_delivered_customer_date#28, order_estimated_deliv
ery_date#29, customer_state#42, customer_zip_code_prefix#40]
:                                     : : : : :
: +- Join Inner, (customer_id#23 = customer_id#38)
:                                     : : : : :
: :- Project [order_id#8, order_item_id#9, product_id#10, seller_id#1
1, shipping_limit_date#12, price#13, freight_value#14, customer_id#23, ord
er_purchase_timestamp#25, order_delivered_customer_date#28, order_estimate
d_delivery_date#29]
:                                     : : : : :
: : +- Join Inner, (order_id#8 = order_id#22)
:                                     : : : : :
: : :- Relation [order_id#8, order_item_id#9, product_id#10, seller_i

```

```

d#11,shipping_limit_date#12,price#13,freight_value#14] csv
:
:      +- Project [order_id#22, customer_id#23, order_purchase_timest
amp#25, order_delivered_customer_date#28, order_estimated_delivery_date#2
9]
:
:      +- Filter ((order_status#24 = delivered) AND isnotnull(orde
r_delivered_customer_date#28))
:
:      +- Relation [order_id#22,customer_id#23,order_status#24,
order_purchase_timestamp#25,order_approved_at#26,order_delivered_carrier_d
ate#27,order_delivered_customer_date#28,order_estimated_delivery_date#29]
csv
:
:      +- Project [customer_id#38, customer_state#42, customer_zip_code_pre
fix#40]
:
:      +- Relation [customer_id#38,customer_unique_id#39,customer_zip_co
de_prefix#40,customer_city#41,customer_state#42] csv
:
+- ResolvedHint (strategy=broadcast)
:
+- Project [seller_id#48, seller_state#51, seller_zip_code_prefix#49]
:
+- Relation [seller_id#48,seller_zip_code_prefix#49,seller_city#50,seller_
state#51] csv
:
:      +- Re
solvedHint (strategy=broadcast)
:
:      +-
Project [product_id#70, product_category_name#71, product_weight_g#75, pro
duct_length_cm#76, product_height_cm#77, product_width_cm#78]
:
+- Project [product_id#70, product_category_name#71, product_name_length#8
8, product_description_lenght#73 AS product_description_length#99, product
_photos_qty#74, product_weight_g#75, product_length_cm#76, product_height_
cm#77, product_width_cm#78]
:
:
+- Project [product_id#70, product_category_name#71, product_name_lenght#7
2 AS product_name_length#88, product_description_lenght#73, product_photos
_qty#74, product_weight_g#75, product_length_cm#76, product_height_cm#77,
product_width_cm#78]
:
:
+- Relation [product_id#70,product_category_name#71,product_name_lenght#7
2,product_description_lenght#73,product_photos_qty#74,product_weight_g#75,
product_length_cm#76,product_height_cm#77,product_width_cm#78] csv
:
:      +- Resolved
Hint (strategy=broadcast)
:
:      +- Proje
ct [product_category_name#66, product_category_name_english#67]
:
:      +- Un
ion false, false
:
:
Relation [product_category_name#66,product_category_name_english#67] csv
:
:      +-
Project [product_category_name#537, product_category_name_english#538]
:
:
+- LogicalRDD [product_category_name#537, product_category_name_english#53
8], false
:
:      +- ResolvedHint
(strategy=broadcast)

```

```

:                                     :      +- Project [ge
olocation_zip_code_prefix#56 AS customer_zip_code_prefix#682, lat#403 AS c
ust_lat#683, lng#405 AS cust_lng#684]

:                                     :      +- Aggregat
e [geolocation_zip_code_prefix#56], [geolocation_zip_code_prefix#56, avg(g
eolocation_lat#57) AS lat#403, avg(geolocation_lng#58) AS lng#405]

:                                     :      +- Relat
ion [geolocation_zip_code_prefix#56,geolocation_lat#57,geolocation_lng#58,
geolocation_city#59,geolocation_state#60] csv

:                                     +- ResolvedHint (strate
gy=broadcast)

:                                     +- Project [geolocat
ion_zip_code_prefix#717 AS seller_zip_code_prefix#688, lat#403 AS sell_lat
#689, lng#405 AS sell_lng#690]

:                                     +- Aggregate [geo
location_zip_code_prefix#717], [geolocation_zip_code_prefix#717, avg(geolo
cation_lat#718) AS lat#403, avg(geolocation_lng#719) AS lng#405]

:                                     +- Relation [g
eolocation_zip_code_prefix#717,geolocation_lat#718,geolocation_lng#719,geo
location_city#720,geolocation_state#721] csv
+- ResolvedHint (strategy=broadcast)
+- Project [purchase_quarter#9116, quarter_baseline_fpk#8548]
+- Aggregate [purchase_quarter#9116], [purchase_quarter#9116,
count(1) AS items#8545L, (sum(freight_value#9075) / sum(weight_kg#803)) AS
quarter_baseline_fpk#8548, avg(weight_kg#803) AS mean_weight_kg#8550]
+- Filter (weight_kg#803 > cast(0 as double))
+- Project [order_id#9069, order_item_id#9070, lane#748,
seller_state#9092, customer_state#9088, customer_zip_code_prefix#9086, sel
ler_zip_code_prefix#9090, purchase_quarter#9116, order_purchase_timestamp#
9079, price#9074, freight_value#9075, weight_kg#803, freight_per_kg#832, f
reight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, is_int
rastate#958, product_category_name#9094, product_category_name_english#910
3]

+- Project [seller_zip_code_prefix#9090, customer_zip
_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#
9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit
_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079,
order_delivered_customer_date#9082, order_estimated_delivery_date#9083, cu
stomer_state#9088, seller_state#9092, product_weight_g#9098, product_lengt
h_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category
_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#69
0, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#832, fre
ight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, (seller_
state#9092 = customer_state#9088) AS is_intrastate#958]

+- Project [seller_zip_code_prefix#9090, customer_
zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_
id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_lim
it_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#907
9, order_delivered_customer_date#9082, order_estimated_delivery_date#9083,
customer_state#9088, seller_state#9092, product_weight_g#9098, product_len
gth_cm#9099, product_height_cm#9100, product_width_cm#9101, product_catego
ry_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#6
90, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#832, fr
eight_to_price_ratio#862, package_volume_cm3#893, ((6371.0 * cast(2 as dou
ble)) * ASIN(SQRT((POWER(SIN((RADIANS((cust_lat#683 - sell_lat#689)) / cas
t(2 as double))), cast(2 as double)) + ((COS(RADIANS(sell_lat#689)) * COS
(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS((cust_lng#684 - sell_lng#69
0)) / cast(2 as double))), cast(2 as double)))))) AS distance_km#925]

+- Project [seller_zip_code_prefix#9090, custom
er_zip_code_prefix#9086, product_category_name#9094, product_id#9071, sell

```

```

er_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_
limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#
9079, order_delivered_customer_date#9082, order_estimated_delivery_date#90
83, customer_state#9088, seller_state#9092, product_weight_g#9098, product
_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_ca
tegory_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_l
ng#690, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#83
2, freight_to_price_ratio#862, ((product_length_cm#9099 * product_height_c
m#9100) * product_width_cm#9101) AS package_volume_cm3#893]
    +- Project [seller_zip_code_prefix#9090, cus
tomer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, s
eller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shippi
ng_limit_date#9073, price#9074, freight_value#9075, order_purchase_timesta
mp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date
#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, prod
uct_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product
_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sel
l_lng#690, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#
832, CASE WHEN (price#9074 > cast(0 as double)) THEN (freight_value#9075 /
price#9074) END AS freight_to_price_ratio#862]
    +- Project [seller_zip_code_prefix#9090,
customer_zip_code_prefix#9086, product_category_name#9094, product_id#907
1, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, sh
ipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_tim
estamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_
date#9083, customer_state#9088, seller_state#9092, product_weight_g#9098,
product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, pro
duct_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689,
sell_lng#690, lane#748, purchase_quarter#9116, weight_kg#803, CASE WHEN (w
eight_kg#803 > cast(0 as double)) THEN (freight_value#9075 / weight_kg#80
3) END AS freight_per_kg#832]
    +- Project [seller_zip_code_prefix#909
0, customer_zip_code_prefix#9086, product_category_name#9094, product_id#9
071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070,
shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_t
imestamp#9079, order_delivered_customer_date#9082, order_estimated_deliver
y_date#9083, customer_state#9088, seller_state#9092, product_weight_g#909
8, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101,
product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#6
89, sell_lng#690, lane#748, purchase_quarter#9116, (cast(product_weight_g#
9098 as double) / cast(1000.0 as double)) AS weight_kg#803]
    +- Project [seller_zip_code_prefix#
9090, customer_zip_code_prefix#9086, product_category_name#9094, product_i
d#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#907
0, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchas
e_timestamp#9079, order_delivered_customer_date#9082, order_estimated_deli
very_date#9083, customer_state#9088, seller_state#9092, product_weight_g#9
098, product_length_cm#9099, product_height_cm#9100, product_width_cm#910
1, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_la
t#689, sell_lng#690, lane#748, concat_ws(-, cast(year(cast(order_purchase_
timestamp#9079 as date)) as string), concat(0, cast(quarter(cast(order_pur
chase_timestamp#9079 as date)) as string))) AS purchase_quarter#9116]
    +- Project [seller_zip_code_pref
ix#9090, customer_zip_code_prefix#9086, product_category_name#9094, produc
t_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#
9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purc
hase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_d
elivery_date#9083, customer_state#9088, seller_state#9092, product_weight_
g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9
101, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell

```

```

lat#689, sell_lng#690, concat_ws( -> , seller_state#9092, customer_state#9
088) AS lane#748]

+- Project [seller_zip_code_p
refix#9090, customer_zip_code_prefix#9086, product_category_name#9094, pro
duct_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_
id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_p
urchase_timestamp#9079, order_delivered_customer_date#9082, order_estimate
d_delivery_date#9083, customer_state#9088, seller_state#9092, product_weig
ht_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_c
m#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684, se
ll_lat#689, sell_lng#690]

+- Join LeftOuter, (seller
_zip_code_prefix#9090 = seller_zip_code_prefix#688)

:- Project [customer_zi
p_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id
#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit
_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079,
order_delivered_customer_date#9082, order_estimated_delivery_date#9083, cu
stomer_state#9088, seller_state#9092, seller_zip_code_prefix#9090, product
_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_wi
dth_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#68
4]

: +- Join LeftOuter,
(customer_zip_code_prefix#9086 = customer_zip_code_prefix#682)

: :- Project [produ
ct_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077,
order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, f
reight_value#9075, order_purchase_timestamp#9079, order_delivered_customer
_date#9082, order_estimated_delivery_date#9083, customer_state#9088, custo
mer_zip_code_prefix#9086, seller_state#9092, seller_zip_code_prefix#9090,
product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, pro
duct_width_cm#9101, product_category_name_english#9103]

: +- Join LeftOu
ter, (product_category_name#9094 = product_category_name#9102)

: :- Project
[product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_i
tem_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, ord
er_purchase_timestamp#9079, order_delivered_customer_date#9082, order_esti
mated_delivery_date#9083, customer_state#9088, customer_zip_code_prefix#90
86, seller_state#9092, seller_zip_code_prefix#9090, product_category_name#
9094, product_weight_g#9098, product_length_cm#9099, product_height_cm#910
0, product_width_cm#9101]

: +- Join
LeftOuter, (product_id#9071 = product_id#9093)

: :- Pr
oject [seller_id#9072, customer_id#9077, order_id#9069, order_item_id#907
0, product_id#9071, shipping_limit_date#9073, price#9074, freight_value#90
75, order_purchase_timestamp#9079, order_delivered_customer_date#9082, ord
er_estimated_delivery_date#9083, customer_state#9088, customer_zip_code_pr
efix#9086, seller_state#9092, seller_zip_code_prefix#9090]

: +-
Join Inner, (seller_id#9072 = seller_id#9089)

: :- Project [customer_id#9077, order_id#9069, order_item_id#9070, product_i
d#9071, seller_id#9072, shipping_limit_date#9073, price#9074, freight_valu
e#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082,
order_estimated_delivery_date#9083, customer_state#9088, customer_zip_code
_prefix#9086]

: +- Join Inner, (customer_id#9077 = customer_id#9084)

```

```

:      :- Project [order_id#9069, order_item_id#9070, product_id#9071, sell
er_id#9072, shipping_limit_date#9073, price#9074, freight_value#9075, cust
omer_id#9077, order_purchase_timestamp#9079, order_delivered_customer_date
#9082, order_estimated_delivery_date#9083]
:      :      +- Join Inner, (order_id#9069 = order_id#9076)
:      :      :- Relation [order_id#9069,order_item_id#9070,product_id#9071,
seller_id#9072,shipping_limit_date#9073,price#9074,freight_value#9075] csv
:      :      +- Project [order_id#9076, customer_id#9077, order_purchase_ti
mestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery
_date#9083]
:      :      +- Filter ((order_status#9078 = delivered) AND isnotnull(or
der_delivered_customer_date#9082))
:      :      +- Relation [order_id#9076,customer_id#9077,order_status
#9078,order_purchase_timestamp#9079,order_approved_at#9080,order_delivered
_carrier_date#9081,order_delivered_customer_date#9082,order_estimated_deli
very_date#9083] csv
:      +- Project [customer_id#9084, customer_state#9088, customer_zip_code
_prefix#9086]
:      +- Relation [customer_id#9084,customer_unique_id#9085,customer_zi
p_code_prefix#9086,customer_city#9087,customer_state#9088] csv
+- ResolvedHint (strategy=broadcast)
+- Project [seller_id#9089, seller_state#9092, seller_zip_code_prefix#909
0]
+- Relation [seller_id#9089,seller_zip_code_prefix#9090,seller_city#9091,s
eller_state#9092] csv
:      :      +- Re
solvedHint (strategy=broadcast)
:      :      +-
Project [product_id#9093, product_category_name#9094, product_weight_g#909
8, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101]
+- Project [product_id#9093, product_category_name#9094, product_name_leng
th#88, product_description_lenght#9096 AS product_description_length#99, p
roduct_photos_qty#9097, product_weight_g#9098, product_length_cm#9099, pro
duct_height_cm#9100, product_width_cm#9101]
+- Project [product_id#9093, product_category_name#9094, product_name_leng
ht#9095 AS product_name_length#88, product_description_lenght#9096, produc
t_photos_qty#9097, product_weight_g#9098, product_length_cm#9099, product_
height_cm#9100, product_width_cm#9101]
+- Relation [product_id#9093,product_category_name#9094,product_name_lengh
t#9095,product_description_lenght#9096,product_photos_qty#9097,product_wei
ght_g#9098,product_length_cm#9099,product_height_cm#9100,product_width_cm#
9101] csv
:      :      +- Resolved
Hint (strategy=broadcast)
:      :      +- Proje
ct [product_category_name#9102, product_category_name_english#9103]

```

```

:      :      +- Un
ion false, false
:      :      :-
Relation [product_category_name#9102,product_category_name_english#9103] c
sv
:      :      +-
Project [product_category_name#9104, product_category_name_english#9105]
:      :
+- LogicalRDD [product_category_name#9104, product_category_name_english#9
105], false
:      +- ResolvedHint
(strategy=broadcast)
:      +- Project [ge
olocation_zip_code_prefix#9106 AS customer_zip_code_prefix#682, lat#403 AS
cust_lat#683, lng#405 AS cust_lng#684]
:      +- Aggregat
e [geolocation_zip_code_prefix#9106], [geolocation_zip_code_prefix#9106, a
vg(geolocation_lat#9107) AS lat#403, avg(geolocation_lng#9108) AS lng#405]
:      +- Relat
ion [geolocation_zip_code_prefix#9106,geolocation_lat#9107,geolocation_lng
#9108,geolocation_city#9109,geolocation_state#9110] csv
+- ResolvedHint (strate
gy=broadcast)
+- Project [geolocat
ion_zip_code_prefix#9111 AS seller_zip_code_prefix#688, lat#403 AS sell_la
t#689, lng#405 AS sell_lng#690]
+- Aggregate [geo
location_zip_code_prefix#9111], [geolocation_zip_code_prefix#9111, avg(geo
location_lat#9112) AS lat#403, avg(geolocation_lng#9113) AS lng#405]
+- Relation [g
eolocation_zip_code_prefix#9111,geolocation_lat#9112,geolocation_lng#9113,
geolocation_city#9114,geolocation_state#9115] csv

== Analyzed Logical Plan ==
purchase_quarter: string, lane: string, seller_state: string, customer_sta
te: string, item_count: bigint, order_count: bigint, total_freight: doubl
e, total_weight_kg: double, mean_item_freight_per_kg: double, mean_freight
_to_price: double, mean_distance_km: double, freight_per_kg: double, quart
er_baseline_fpk: double, relative_to_quarter: double
Project [purchase_quarter#775, lane#748, seller_state#51, customer_state#4
2, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_k
g#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_di
stance_km#8005, freight_per_kg#8018, quarter_baseline_fpk#8548, (freight_p
er_kg#8018 / quarter_baseline_fpk#8548) AS relative_to_quarter#9137]
+- Project [purchase_quarter#775, lane#748, seller_state#51, customer_stat
e#42, item_count#7994L, order_count#7995L, total_freight#7997, total_weigh
t_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean
_distance_km#8005, freight_per_kg#8018, quarter_baseline_fpk#8548]
+- Join LeftOuter, (purchase_quarter#775 = purchase_quarter#9116)
:- Filter (item_count#7994L >= cast(30 as bigint))
: +- Project [lane#748, seller_state#51, customer_state#42, purchas
e_quarter#775, item_count#7994L, order_count#7995L, total_freight#7997, to
tal_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8
003, mean_distance_km#8005, (total_freight#7997 / total_weight_kg#7999) AS
freight_per_kg#8018]
: +- Aggregate [lane#748, seller_state#51, customer_state#42, pu
rchase_quarter#775], [lane#748, seller_state#51, customer_state#42, purcha
se_quarter#775, count(1) AS item_count#7994L, count(distinct order_id#8) A
S order_count#7995L, sum(freight_value#14) AS total_freight#7997, sum(weig
ht_kg#803) AS total_weight_kg#7999, avg(freight_per_kg#832) AS mean_item_f

```

```

reight_per_kg#8001, avg(freight_to_price_ratio#862) AS mean_freight_to_pri
ce#8003, avg(distance_km#925) AS mean_distance_km#8005]
:
    +- Filter (weight_kg#803 > cast(0 as double))
:
    +- Project [order_id#8, order_item_id#9, lane#748, selle
r_state#51, customer_state#42, customer_zip_code_prefix#40, seller_zip_cod
e_prefix#49, purchase_quarter#775, order_purchase_timestamp#25, price#13,
freight_value#14, weight_kg#803, freight_per_kg#832, freight_to_price_rati
o#862, package_volume_cm3#893, distance_km#925, is_intrastate#958, product
_category_name#71, product_category_name_english#67]
:
    +- Project [seller_zip_code_prefix#49, customer_zip_c
ode_prefix#40, product_category_name#71, product_id#10, seller_id#11, cust
omer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13,
freight_value#14, order_purchase_timestamp#25, order_delivered_customer_da
te#28, order_estimated_delivery_date#29, customer_state#42, seller_state#5
1, product_weight_g#75, product_length_cm#76, product_height_cm#77, produc
t_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#68
4, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, weight_kg#8
03, freight_per_kg#832, freight_to_price_ratio#862, package_volume_cm3#89
3, distance_km#925, (seller_state#51 = customer_state#42) AS is_intrastate
#958]
:
    +- Project [seller_zip_code_prefix#49, customer_zi
p_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, c
ustomer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#
13, freight_value#14, order_purchase_timestamp#25, order_delivered_custome
r_date#28, order_estimated_delivery_date#29, customer_state#42, seller_sta
te#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, pr
oduct_width_cm#78, product_category_name_english#67, cust_lat#683, cust_ln
g#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, weight_
kg#803, freight_per_kg#832, freight_to_price_ratio#862, package_volume_cm3
#893, ((6371.0 * cast(2 as double)) * ASIN(SQRT((POWER(SIN((RADIANS((cust_
lat#683 - sell_lat#689)) / cast(2 as double))), cast(2 as double)) + ((COS
(RADIANS(sell_lat#689)) * COS(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS
((cust_lng#684 - sell_lng#690)) / cast(2 as double))), cast(2 as doubl
e)))))) AS distance_km#925]
:
    +- Project [seller_zip_code_prefix#49, customer
_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#1
1, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, pr
ice#13, freight_value#14, order_purchase_timestamp#25, order_delivered_cus
tomer_date#28, order_estimated_delivery_date#29, customer_state#42, seller
_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#7
7, product_width_cm#78, product_category_name_english#67, cust_lat#683, cu
st_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, we
ight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, ((product_len
gth_cm#76 * product_height_cm#77) * product_width_cm#78) AS package_volume
_cm3#893]
:
    +- Project [seller_zip_code_prefix#49, custo
mer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id
#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12,
price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_c
ustomer_date#28, order_estimated_delivery_date#29, customer_state#42, sell
er_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#
77, product_width_cm#78, product_category_name_english#67, cust_lat#683, c
ust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, w
eight_kg#803, freight_per_kg#832, CASE WHEN (price#13 > cast(0 as double))
THEN (freight_value#14 / price#13) END AS freight_to_price_ratio#862]
:
    +- Project [seller_zip_code_prefix#49, cu
stomer_zip_code_prefix#40, product_category_name#71, product_id#10, seller
_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#1
2, price#13, freight_value#14, order_purchase_timestamp#25, order_delivere
d_customer_date#28, order_estimated_delivery_date#29, customer_state#42, s

```

```

    eller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, weight_kg#803, CASE WHEN (weight_kg#803 > cast(0 as double)) THEN (freight_value#14 / weight_kg#803) END AS freight_per_kg#832]
    :
    +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#775, (cast(product_weight_g#75 as double) / cast(1000.0 as double)) AS weight_kg#803]
    :
    +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, concat_ws(-, cast(year(cast(order_purchase_timestamp#25 as date)) as string), concat(Q, cast(quarter(cast(order_purchase_timestamp#25 as date)) as string))) AS purchase_quarter#775]
    :
    +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, concat_ws(->, seller_state#51, customer_state#42) AS lane#748]
    :
    +- Project [seller_zip_code_prefix#49, customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690]
    :
    +- Join LeftOuter, (seller_zip_code_prefix#49 = seller_zip_code_prefix#688)
    :
    :- Project [customer_zip_code_prefix#40, product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, seller_state#51, seller_zip_code_prefix#49, product_weight_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684]
    :
    : +- Join LeftOuter, (customer_zip_code_prefix#40 = customer_zip_code_prefix#682)
    :
    :- Project [product_category_name#71, product_id#10, seller_id#11, customer_id#23, order_id#8, order_item_id#9, shipping_limit_date#12, price#13, freight_value#14, order_purchase_timestamp#25, order_delivered_customer_date#28, order_estimated_delivery_date#29, customer_state#42, customer_zip_code_prefix#40, sell

```

68/98

```

+- Project [seller_id#48, seller_state#51, seller_zip_code_prefix#49]
      :
+- Relation [seller_id#48,seller_zip_code_prefix#49,seller_city#50,seller_
state#51] csv
      :
      :      :      +- Re
solvedHint (strategy=broadcast)
      :
      :      :      +-
Project [product_id#70, product_category_name#71, product_weight_g#75, pro
duct_length_cm#76, product_height_cm#77, product_width_cm#78]
      :
+- Project [product_id#70, product_category_name#71, product_name_length#8
8, product_description_lenght#73 AS product_description_length#99, product
_photos_qty#74, product_weight_g#75, product_length_cm#76, product_height_
cm#77, product_width_cm#78]
      :
+- Project [product_id#70, product_category_name#71, product_name_lenght#7
2 AS product_name_length#88, product_description_lenght#73, product_photos
_qty#74, product_weight_g#75, product_length_cm#76, product_height_cm#77,
product_width_cm#78]
      :
+- Relation [product_id#70,product_category_name#71,product_name_lenght#7
2,product_description_lenght#73,product_photos_qty#74,product_weight_g#75,
product_length_cm#76,product_height_cm#77,product_width_cm#78] csv
      :
      :      +- Resolved
Hint (strategy=broadcast)
      :
      :      +- Proje
ct [product_category_name#66, product_category_name_english#67]
      :
      :      +- Un
ion false, false
      :
      :      :      :-
Relation [product_category_name#66,product_category_name_english#67] csv
      :
      :      +-
Project [product_category_name#537, product_category_name_english#538]
      :
+- LogicalRDD [product_category_name#537, product_category_name_english#53
8], false
      :
      :      +- ResolvedHint
(strategy=broadcast)
      :
      :      +- Project [ge
olocation_zip_code_prefix#56 AS customer_zip_code_prefix#682, lat#403 AS c
ust_lat#683, lng#405 AS cust_lng#684]
      :
      :      +- Aggregat
e [geolocation_zip_code_prefix#56], [geolocation_zip_code_prefix#56, avg(g
eolocation_lat#57) AS lat#403, avg(geolocation_lng#58) AS lng#405]
      :
      :      +- Relat
ion [geolocation_zip_code_prefix#56,geolocation_lat#57,geolocation_lng#58,
geolocation_city#59,geolocation_state#60] csv
      :
      :      +- ResolvedHint (strate
gy=broadcast)
      :
      :      +- Project [geolocat
ion_zip_code_prefix#717 AS seller_zip_code_prefix#688, lat#403 AS sell_lat
#689, lng#405 AS sell_lng#690]
      :
      :      +- Aggregate [geo
location_zip_code_prefix#717], [geolocation_zip_code_prefix#717, avg(geolo
cation_lat#718) AS lat#403, avg(geolocation_lng#719) AS lng#405]
      :
      :      +- Relation [g
eolocation_zip_code_prefix#717,geolocation_lat#718,geolocation_lng#719,geo
location_city#720,geolocation_state#721] csv
      +- ResolvedHint (strategy=broadcast)
      +- Project [purchase_quarter#9116, quarter_baseline_fpk#8548]

```

```

+- Aggregate [purchase_quarter#9116], [purchase_quarter#9116,
count(1) AS items#8545L, (sum(freight_value#9075) / sum(weight_kg#803)) AS
quarter_baseline_fpk#8548, avg(weight_kg#803) AS mean_weight_kg#8550]
+- Filter (weight_kg#803 > cast(0 as double))
+- Project [order_id#9069, order_item_id#9070, lane#748,
seller_state#9092, customer_state#9088, customer_zip_code_prefix#9086, sel
ler_zip_code_prefix#9090, purchase_quarter#9116, order_purchase_timestamp#
9079, price#9074, freight_value#9075, weight_kg#803, freight_per_kg#832, f
reight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, is_int
rastate#958, product_category_name#9094, product_category_name_english#910
3]
+- Project [seller_zip_code_prefix#9090, customer_zip
_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#
9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit
_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079,
order_delivered_customer_date#9082, order_estimated_delivery_date#9083, cu
stomer_state#9088, seller_state#9092, product_weight_g#9098, product_lengt
h_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category
_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#69
0, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#832, fre
ight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, (seller_
state#9092 = customer_state#9088) AS is_intrastate#958]
+- Project [seller_zip_code_prefix#9090, customer_
zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_
id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_lim
it_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#907
9, order_delivered_customer_date#9082, order_estimated_delivery_date#9083,
customer_state#9088, seller_state#9092, product_weight_g#9098, product_len
gth_cm#9099, product_height_cm#9100, product_width_cm#9101, product_catego
ry_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#6
90, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#832, fr
eight_to_price_ratio#862, package_volume_cm3#893, ((6371.0 * cast(2 as dou
ble)) * ASIN(SQRT((POWER(SIN((RADIANS((cust_lat#683 - sell_lat#689)) / cas
t(2 as double))), cast(2 as double)) + ((COS(RADIANS(sell_lat#689)) * COS
(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS((cust_lng#684 - sell_lng#69
0)) / cast(2 as double))), cast(2 as double)))))) AS distance_km#925]
+- Project [seller_zip_code_prefix#9090, custom
er_zip_code_prefix#9086, product_category_name#9094, product_id#9071, sell
er_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_
limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#
9079, order_delivered_customer_date#9082, order_estimated_delivery_date#90
83, customer_state#9088, seller_state#9092, product_weight_g#9098, product
_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_ca
tegory_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_l
ng#690, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#83
2, freight_to_price_ratio#862, ((product_length_cm#9099 * product_height_c
m#9100) * product_width_cm#9101) AS package_volume_cm3#893]
+- Project [seller_zip_code_prefix#9090, cus
tomer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, s
eller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shippi
ng_limit_date#9073, price#9074, freight_value#9075, order_purchase_timesta
mp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date
#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, prod
uct_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product
_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sel
l_lng#690, lane#748, purchase_quarter#9116, weight_kg#803, freight_per_kg#
832, CASE WHEN (price#9074 > cast(0 as double)) THEN (freight_value#9075 /
price#9074) END AS freight_to_price_ratio#862]
+- Project [seller_zip_code_prefix#9090,
customer_zip_code_prefix#9086, product_category_name#9094, product_id#907

```

```
1, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#9116, weight_kg#803, CASE WHEN (weight_kg#803 > cast(0 as double)) THEN (freight_value#9075 / weight_kg#803) END AS freight_per_kg#832]
```

```
+-- Project [seller_zip_code_prefix#9090, customer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, purchase_quarter#9116, (cast(product_weight_g#9098 as double) / cast(1000.0 as double)) AS weight_kg#803]
```

```
+-- Project [seller_zip_code_prefix#9090, customer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, lane#748, concat_ws(-, cast(year(cast(order_purchase_timestamp#9079 as date)) as string), concat(0, cast(quarter(cast(order_purchase_timestamp#9079 as date)) as string))) AS purchase_quarter#9116]
```

```
+-- Project [seller_zip_code_prefix#9090, customer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690, concat_ws(->, seller_state#9092, customer_state#9088) AS lane#748]
```

```
+-- Project [seller_zip_code_prefix#9090, customer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date#9083, customer_state#9088, seller_state#9092, product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684, sell_lat#689, sell_lng#690]
```

```
+-- Join LeftOuter, (seller_zip_code_prefix#9090 = seller_zip_code_prefix#688)
```

```
:- Project [customer_zip_code_prefix#9086, product_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery_date#9083, customer_state#9088, seller_state#9092, seller_zip_code_prefix#9090, product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101, product_category_name_english#9103, cust_lat#683, cust_lng#684]
```

```

: +- Join LeftOuter,
(customer_zip_code_prefix#9086 = customer_zip_code_prefix#682)
: :- Project [produ
ct_category_name#9094, product_id#9071, seller_id#9072, customer_id#9077,
order_id#9069, order_item_id#9070, shipping_limit_date#9073, price#9074, f
reight_value#9075, order_purchase_timestamp#9079, order_delivered_customer
_date#9082, order_estimated_delivery_date#9083, customer_state#9088, custo
mer_zip_code_prefix#9086, seller_state#9092, seller_zip_code_prefix#9090,
product_weight_g#9098, product_length_cm#9099, product_height_cm#9100, pro
duct_width_cm#9101, product_category_name_english#9103]
: +- Join LeftOu
ter, (product_category_name#9094 = product_category_name#9102)
: :- Project
[product_id#9071, seller_id#9072, customer_id#9077, order_id#9069, order_i
tem_id#9070, shipping_limit_date#9073, price#9074, freight_value#9075, ord
er_purchase_timestamp#9079, order_delivered_customer_date#9082, order_esti
mated_delivery_date#9083, customer_state#9088, customer_zip_code_prefix#90
86, seller_state#9092, seller_zip_code_prefix#9090, product_category_name#
9094, product_weight_g#9098, product_length_cm#9099, product_height_cm#910
0, product_width_cm#9101]
: +- Join
LeftOuter, (product_id#9071 = product_id#9093)
: :- Pr
oject [seller_id#9072, customer_id#9077, order_id#9069, order_item_id#907
0, product_id#9071, shipping_limit_date#9073, price#9074, freight_value#90
75, order_purchase_timestamp#9079, order_delivered_customer_date#9082, ord
er_estimated_delivery_date#9083, customer_state#9088, customer_zip_code_pr
efix#9086, seller_state#9092, seller_zip_code_prefix#9090]
: +-
Join Inner, (seller_id#9072 = seller_id#9089)
: :- Project [customer_id#9077, order_id#9069, order_item_id#9070, product_i
d#9071, seller_id#9072, shipping_limit_date#9073, price#9074, freight_valu
e#9075, order_purchase_timestamp#9079, order_delivered_customer_date#9082,
order_estimated_delivery_date#9083, customer_state#9088, customer_zip_code
_prefix#9086]
: +- Join Inner, (customer_id#9077 = customer_id#9084)
: :- Project [order_id#9069, order_item_id#9070, product_id#9071, sell
er_id#9072, shipping_limit_date#9073, price#9074, freight_value#9075, cust
omer_id#9077, order_purchase_timestamp#9079, order_delivered_customer_date
#9082, order_estimated_delivery_date#9083]
: +- Join Inner, (order_id#9069 = order_id#9076)
: :- Relation [order_id#9069,order_item_id#9070,product_id#9071,
seller_id#9072,shipping_limit_date#9073,price#9074,freight_value#9075] csv
: +- Project [order_id#9076, customer_id#9077, order_purchase_ti
mestamp#9079, order_delivered_customer_date#9082, order_estimated_delivery
_date#9083]
: +- Filter ((order_status#9078 = delivered) AND isnotnull(or
der_delivered_customer_date#9082))
: +- Relation [order_id#9076,customer_id#9077,order_status
#9078,order_purchase_timestamp#9079,order_approved_at#9080,order_delivered
_carrier_date#9081,order_delivered_customer_date#9082,order_estimated_deli
very_date#9083] csv

```

```

:      +- Project [customer_id#9084, customer_state#9088, customer_zip_code
_prefix#9086]
:      :      :      :
:      +- Relation [customer_id#9084,customer_unique_id#9085,customer_zi
p_code_prefix#9086,customer_city#9087,customer_state#9088] csv
:      :      :      :
+- ResolvedHint (strategy=broadcast)
:      :      :      :
+- Project [seller_id#9089, seller_state#9092, seller_zip_code_prefix#909
0]
:      :      :      :
+- Relation [seller_id#9089,seller_zip_code_prefix#9090,seller_city#9091,s
eller_state#9092] csv
:      :      :      +- Re
solvedHint (strategy=broadcast)
:      :      :      +-
Project [product_id#9093, product_category_name#9094, product_weight_g#909
8, product_length_cm#9099, product_height_cm#9100, product_width_cm#9101]
:      :      :
+- Project [product_id#9093, product_category_name#9094, product_name_leng
th#88, product_description_lenght#9096 AS product_description_length#99, p
roduct_photos_qty#9097, product_weight_g#9098, product_length_cm#9099, pro
duct_height_cm#9100, product_width_cm#9101]
:      :      :
+- Project [product_id#9093, product_category_name#9094, product_name_leng
ht#9095 AS product_name_length#88, product_description_lenght#9096, produc
t_photos_qty#9097, product_weight_g#9098, product_length_cm#9099, product_
height_cm#9100, product_width_cm#9101]
:      :      :
+- Relation [product_id#9093,product_category_name#9094,product_name_lengh
t#9095,product_description_lenght#9096,product_photos_qty#9097,product wei
ght_g#9098,product_length_cm#9099,product_height_cm#9100,product_width_cm#
9101] csv
:      :      +- Resolved
Hint (strategy=broadcast)
:      :      +- Proje
ct [product_category_name#9102, product_category_name_english#9103]
:      :      +- Un
ion false, false
:      :      :-
Relation [product_category_name#9102,product_category_name_english#9103] c
sv
:      :      +-
Project [product_category_name#9104, product_category_name_english#9105]
:      :
+- LogicalRDD [product_category_name#9104, product_category_name_english#9
105], false
:      +- ResolvedHint
(strategy=broadcast)
:      +- Project [ge
olocation_zip_code_prefix#9106 AS customer_zip_code_prefix#682, lat#403 AS
cust_lat#683, lng#405 AS cust_lng#684]
:      +- Aggregat
e [geolocation_zip_code_prefix#9106], [geolocation_zip_code_prefix#9106, a
vg(geolocation_lat#9107) AS lat#403, avg(geolocation_lng#9108) AS lng#405]
:      +- Relat
ion [geolocation_zip_code_prefix#9106,geolocation_lat#9107,geolocation_lng
#9108,geolocation_city#9109,geolocation_state#9110] csv
+- ResolvedHint (strate

```

```
gy=broadcast)
```

```

                                +- Project [geolocat
ion_zip_code_prefix#9111 AS seller_zip_code_prefix#688, lat#403 AS sell_la
t#689, lng#405 AS sell_lng#690]
```

```

                                +- Aggregate [geo
location_zip_code_prefix#9111], [geolocation_zip_code_prefix#9111, avg(geo
location_lat#9112) AS lat#403, avg(geolocation_lng#9113) AS lng#405]
```

```

                                +- Relation [g
eolocation_zip_code_prefix#9111,geolocation_lat#9112,geolocation_lng#9113,
geolocation_city#9114,geolocation_state#9115] csv
```

```
== Optimized Logical Plan ==
```

```

Project [purchase_quarter#775, lane#748, seller_state#51, customer_state#4
2, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_k
g#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_di
stance_km#8005, freight_per_kg#8018, quarter_baseline_fpk#8548, (freight_p
er_kg#8018 / quarter_baseline_fpk#8548) AS relative_to_quarter#9137]
+- Join LeftOuter, (purchase_quarter#775 = purchase_quarter#9116), rightHi
nt=(strategy=broadcast)
```

```

    :- Project [lane#748, seller_state#51, customer_state#42, purchase_quar
ter#775, item_count#7994L, order_count#7995L, total_freight#7997, total_we
ight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, m
ean_distance_km#8005, (total_freight#7997 / total_weight_kg#7999) AS freig
ht_per_kg#8018]
```

```
      : +- Filter (item_count#7994L >= 30)
```

```

      :       +- Aggregate [lane#748, seller_state#51, customer_state#42, purch
ase_quarter#775], [lane#748, seller_state#51, customer_state#42, purchase_
quarter#775, count(1) AS item_count#7994L, count(distinct order_id#8) AS o
rder_count#7995L, sum(freight_value#14) AS total_freight#7997, sum(weight_
kg#803) AS total_weight_kg#7999, avg(freight_per_kg#832) AS mean_item_frei
ght_per_kg#8001, avg(freight_to_price_ratio#862) AS mean_freight_to_price#
8003, avg(distance_km#925) AS mean_distance_km#8005]
```

```

      :       +- Project [order_id#8, lane#748, seller_state#51, customer_st
ate#42, purchase_quarter#775, freight_value#14, weight_kg#803, freight_per
_kg#832, freight_to_price_ratio#862, distance_km#925]
```

```

      :       +- Filter (isnotnull(weight_kg#803) AND (weight_kg#803 > 0.
0))
```

```

      :       +- InMemoryRelation [order_id#8, order_item_id#9, lane#7
48, seller_state#51, customer_state#42, customer_zip_code_prefix#40, selle
r_zip_code_prefix#49, purchase_quarter#775, order_purchase_timestamp#25, p
rice#13, freight_value#14, weight_kg#803, freight_per_kg#832, freight_to_p
rice_ratio#862, package_volume_cm3#893, distance_km#925, is_intrastate#95
8, product_category_name#71, product_category_name_english#67], StorageLev
el(disk, memory, deserialized, 1 replicas)
```

```

      :       +- *(9) Project [order_id#8, order_item_id#9, lane
#748, seller_state#51, customer_state#42, customer_zip_code_prefix#40, sel
ler_zip_code_prefix#49, purchase_quarter#775, order_purchase_timestamp#25,
price#13, freight_value#14, weight_kg#803, CASE WHEN (weight_kg#803 > 0.0)
THEN (freight_value#14 / weight_kg#803) END AS freight_per_kg#832, CASE WH
EN (price#13 > 0.0) THEN (freight_value#14 / price#13) END AS freight_to_p
rice_ratio#862, ((product_length_cm#76 * product_height_cm#77) * product_w
idth_cm#78) AS package_volume_cm3#893, (12742.0 * ASIN(SQRT((POWER(SIN((RA
DIANS((cust_lat#683 - sell_lat#689)) / 2.0)), 2.0) + ((COS(RADIANS(sell_la
t#689)) * COS(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS((cust_lng#684 -
sell_lng#690)) / 2.0)), 2.0)))))) AS distance_km#925, (seller_state#51 = c
ustomer_state#42) AS is_intrastate#958, product_category_name#71, product_
category_name_english#67]
```

```

      :       +- *(9) Project [seller_zip_code_prefix#49, cus
tomer_zip_code_prefix#40, product_category_name#71, order_id#8, order_item
_id#9, price#13, freight_value#14, order_purchase_timestamp#25, customer_s
```

```

tate#42, seller_state#51, product_length_cm#76, product_height_cm#77, prod
uct_width_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#
684, sell_lat#689, sell_lng#690, concat_ws(->, seller_state#51, customer
_state#42) AS lane#748, concat_ws(-, cast(year(cast(order_purchase_timesta
mp#25 as date)) as string), concat(0, cast(quarter(cast(order_purchase_tim
estamp#25 as date)) as string))) AS purchase_quarter#775, (cast(product_we
ight_g#75 as double) / 1000.0) AS weight_kg#803]
:                                     +- *(9) BroadcastHashJoin [seller_zip_code_p
refix#49], [seller_zip_code_prefix#688], LeftOuter, BuildRight, false
:                                     :- *(9) Project [customer_zip_code_prefix
#40, product_category_name#71, order_id#8, order_item_id#9, price#13, frei
ght_value#14, order_purchase_timestamp#25, customer_state#42, seller_state
#51, seller_zip_code_prefix#49, product_weight_g#75, product_length_cm#76,
product_height_cm#77, product_width_cm#78, product_category_name_english#6
7, cust_lat#683, cust_lng#684]
:                                     : +- *(9) BroadcastHashJoin [customer_zi
p_code_prefix#40], [customer_zip_code_prefix#682], LeftOuter, BuildRight,
false
:                                     : :- *(9) Project [product_category_n
ame#71, order_id#8, order_item_id#9, price#13, freight_value#14, order_pur
chase_timestamp#25, customer_state#42, customer_zip_code_prefix#40, seller
_state#51, seller_zip_code_prefix#49, product_weight_g#75, product_length_
cm#76, product_height_cm#77, product_width_cm#78, product_category_name_en
glish#67]
:                                     : +- *(9) BroadcastHashJoin [produ
ct_category_name#71], [product_category_name#66], LeftOuter, BuildRight, f
alse
:                                     : :- *(9) Project [order_id#8,
order_item_id#9, price#13, freight_value#14, order_purchase_timestamp#25,
customer_state#42, customer_zip_code_prefix#40, seller_state#51, seller_zi
p_code_prefix#49, product_category_name#71, product_weight_g#75, product_l
ength_cm#76, product_height_cm#77, product_width_cm#78]
:                                     : +- *(9) BroadcastHashJoin
[product_id#10], [product_id#70], LeftOuter, BuildRight, false
:                                     : :- *(9) Project [order_
id#8, order_item_id#9, product_id#10, price#13, freight_value#14, order_pu
rchase_timestamp#25, customer_state#42, customer_zip_code_prefix#40, selle
r_state#51, seller_zip_code_prefix#49]
:                                     : +- *(9) BroadcastHas
hJoin [seller_id#11], [seller_id#48], Inner, BuildRight, false
:                                     : :- *(9) Project
[order_id#8, order_item_id#9, product_id#10, seller_id#11, price#13, freig
ht_value#14, order_purchase_timestamp#25, customer_state#42, customer_zip_
code_prefix#40]
:                                     : +- *(9) Broadc
astHashJoin [customer_id#23], [customer_id#38], Inner, BuildRight, false
:                                     : :- *(9) Pro
ject [order_id#8, order_item_id#9, product_id#10, seller_id#11, price#13,
freight_value#14, customer_id#23, order_purchase_timestamp#25]
:                                     : +- *(9)
BroadcastHashJoin [order_id#8], [order_id#22], Inner, BuildRight, false
:                                     : :- *
(9) Filter (isnotnull(order_id#8) AND isnotnull(seller_id#11))
:                                     : +-
FileScan csv [order_id#8,order_item_id#9,product_id#10,seller_id#11,price#
13,freight_value#14] Batched: false, DataFilters: [isnotnull(order_id#8),
isnotnull(seller_id#11)], Format: CSV, Location: InMemoryFileIndex(1 path
s)[file:/Users/mmmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_o
r..., PartitionFilters: [], PushedFilters: [IsNotNull(order_id), IsNotNull
(seller_id)], ReadSchema: struct<order_id:string,order_item_id:int,product

```

```

_id:string,seller_id:string,price:double,freight_...
: : : : : : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]),false), [plan_id=9818]
: : : : : : +-
*(1) Project [order_id#22, customer_id#23, order_purchase_timestamp#25]
: : : : : :
+- *(1) Filter (((isnotnull(order_status#24) AND (order_status#24 = delivered)) AND isnotnull(order_delivered_customer_date#28)) AND isnotnull(order_id#22)) AND isnotnull(customer_id#23))
: : : : : :
+- FileScan csv [order_id#22,customer_id#23,order_status#24,order_purchase_timestamp#25,order_delivered_customer_date#28] Batched: false, DataFilters: [isnotnull(order_status#24), (order_status#24 = delivered), isnotnull(order_delivered_customer_date#28)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_or..., PartitionFilters: [], PushedFilters: [IsNotNull(order_status), EqualTo(order_status,delivered), IsNotNull(order_delivered_customer_date#28)], ReadSchema: struct<order_id:string,customer_id:string,order_status:string,order_purchase_timestamp:timestamp,...
: : : : : : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=9826]
: : : : : : +- *(2) Filter isnotnull(customer_id#38)
: : : : : : +- FileScan csv [customer_id#38,customer_zip_code_prefix#40,customer_state#42] Batched: false, DataFilters: [isnotnull(customer_id#38)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_customer..., PartitionFilters: [], PushedFilters: [IsNotNull(customer_id)], ReadSchema: struct<customer_id:string,customer_zip_code_prefix:int,customer_state:string>
: : : : : : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=9834]
: : : : : : +- *(3) Filter isnotnull(seller_id#48)
: : : : : : +- FileScan csv [seller_id#48,seller_zip_code_prefix#49,seller_state#51] Batched: false, DataFilters: [isnotnull(seller_id#48)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_seller..., PartitionFilters: [], PushedFilters: [IsNotNull(seller_id)], ReadSchema: struct<seller_id:string,seller_zip_code_prefix:int,seller_state:string>
: : : : : : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=9842]
: : : : : : +- *(4) Filter isnotnull(product_id#70)
: : : : : : +- FileScan csv [product_id#70,product_category_name#71,product_weight_g#75,product_length_cm#76,product_height_cm#77,product_width_cm#78] Batched: false, DataFilters: [isnotnull(product_id#70)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_product..., PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSchema: struct<product_id:string,product_category_name:string,product_weight_g:int,product_length_cm:int,...
: : : : : : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=9854]
: : : : : : +- Union

```

```

      :                               :                               :- *(5) Filter isnotnul
l(product_category_name#66)
      :                               :                               : +- FileScan csv [pro
duct_category_name#66,product_category_name_english#67] Batched: false, Da
taFilters: [isNotNull(product_category_name#66)], Format: CSV, Location: I
nMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessmen
t1-spark/data/product_..., PartitionFilters: [], PushedFilters: [IsNotNull
(product_category_name)], ReadSchema: struct<product_category_name:string,
product_category_name_english:string>
      :                               :                               +- *(6) Filter isnotnul
l(product_category_name#537)
      :                               :                               +- *(6) Scan Existin
gRDD[product_category_name#537,product_category_name_english#538]
      :                               :                               +- BroadcastExchange HashedRelation
BroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=9
863]
      :                               :                               +- *(7) Project [geolocation_zip
_code_prefix#56 AS customer_zip_code_prefix#682, lat#403 AS cust_lat#683,
lng#405 AS cust_lng#684]
      :                               :                               +- *(7) Filter isnotnull(geol
ocation_zip_code_prefix#56)
      :                               :                               +- InMemoryTableScan [geol
ocation_zip_code_prefix#56, lat#403, lng#405], [isNotNull(geolocation_zip_
code_prefix#56)]
      :                               :                               +- InMemoryRelation
[geolocation_zip_code_prefix#56, lat#403, lng#405], StorageLevel(disk, mem
ory, deserialized, 1 replicas)
      :                               :                               +- *(2) HashAg
gregate(keys=[geolocation_zip_code_prefix#56], functions=[avg(geolocation_
lat#57), avg(geolocation_lng#58)], output=[geolocation_zip_code_prefix#56,
lat#403, lng#405])
      :                               :                               +- Exchange
hashpartitioning(geolocation_zip_code_prefix#56, 4), ENSURE_REQUIREMENTS,
[plan_id=1685]
      :                               :                               +- *(1)
HashAggregate(keys=[geolocation_zip_code_prefix#56], functions=[partial_av
g(geolocation_lat#57), partial_avg(geolocation_lng#58)], output=[geolocati
on_zip_code_prefix#56, sum#413, count#414L, sum#415, count#416L])
      :                               :                               +- Fi
leScan csv [geolocation_zip_code_prefix#56,geolocation_lat#57,geolocation_
lng#58] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFi
leIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/
data/olist_ge..., PartitionFilters: [], PushedFilters: [], ReadSchema: str
uct<geolocation_zip_code_prefix:int,geolocation_lat:double,geolocation_ln
g:double>
      :                               :                               +- ReusedExchange [seller_zip_code_prefix
#688, sell_lat#689, sell_lng#690], BroadcastExchange HashedRelationBroadca
stMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=1608]
      +- Aggregate [purchase_quarter#9116], [purchase_quarter#9116, (sum(frei
ght_value#9075) / sum(weight_kg#803)) AS quarter_baseline_fpk#8548]
      +- Project [purchase_quarter#9116, freight_value#9075, weight_kg#80
3]
      +- Filter (isNotNull(weight_kg#803) AND (weight_kg#803 > 0.0))
      +- InMemoryRelation [order_id#9069, order_item_id#9070, lane#7
48, seller_state#9092, customer_state#9088, customer_zip_code_prefix#9086,
seller_zip_code_prefix#9090, purchase_quarter#9116, order_purchase_timesta
mp#9079, price#9074, freight_value#9075, weight_kg#803, freight_per_kg#83
2, freight_to_price_ratio#862, package_volume_cm3#893, distance_km#925, is
_intrastate#958, product_category_name#9094, product_category_name_english
#9103], StorageLevel(disk, memory, deserialized, 1 replicas)

```

```

+- *(9) Project [order_id#8, order_item_id#9, lane#748,
seller_state#51, customer_state#42, customer_zip_code_prefix#40, seller_zi
p_code_prefix#49, purchase_quarter#775, order_purchase_timestamp#25, price
#13, freight_value#14, weight_kg#803, CASE WHEN (weight_kg#803 > 0.0) THEN
(freight_value#14 / weight_kg#803) END AS freight_per_kg#832, CASE WHEN (p
rice#13 > 0.0) THEN (freight_value#14 / price#13) END AS freight_to_price
ratio#862, ((product_length_cm#76 * product_height_cm#77) * product_width_
cm#78) AS package_volume_cm3#893, (12742.0 * ASIN(SQRT((POWER(SIN((RADIANS
((cust_lat#683 - sell_lat#689)) / 2.0)), 2.0) + ((COS(RADIANS(sell_lat#68
9)) * COS(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS((cust_lng#684 - sel
l_lng#690)) / 2.0)), 2.0)))))) AS distance_km#925, (seller_state#51 = cust
omer_state#42) AS is_intrastate#958, product_category_name#71, product_cat
egory_name_english#67]

+- *(9) Project [seller_zip_code_prefix#49, customer_
zip_code_prefix#40, product_category_name#71, order_id#8, order_item_id#9,
price#13, freight_value#14, order_purchase_timestamp#25, customer_state#4
2, seller_state#51, product_length_cm#76, product_height_cm#77, product_wi
dth_cm#78, product_category_name_english#67, cust_lat#683, cust_lng#684, s
ell_lat#689, sell_lng#690, concat_ws(->, seller_state#51, customer_state
#42) AS lane#748, concat_ws(-, cast(year(cast(order_purchase_timestamp#25
as date)) as string), concat(0, cast(quarter(cast(order_purchase_timestamp
#25 as date)) as string))) AS purchase_quarter#775, (cast(product_weight_g
#75 as double) / 1000.0) AS weight_kg#803]

+- *(9) BroadcastHashJoin [seller_zip_code_prefix#
49], [seller_zip_code_prefix#688], LeftOuter, BuildRight, false
: - *(9) Project [customer_zip_code_prefix#40, p
roduct_category_name#71, order_id#8, order_item_id#9, price#13, freight_va
lue#14, order_purchase_timestamp#25, customer_state#42, seller_state#51, s
eller_zip_code_prefix#49, product_weight_g#75, product_length_cm#76, produ
ct_height_cm#77, product_width_cm#78, product_category_name_english#67, cu
st_lat#683, cust_lng#684]

: +- *(9) BroadcastHashJoin [customer_zip_code
_prefix#40], [customer_zip_code_prefix#682], LeftOuter, BuildRight, false
: - *(9) Project [product_category_name#7
1, order_id#8, order_item_id#9, price#13, freight_value#14, order_purchase
_timestamp#25, customer_state#42, customer_zip_code_prefix#40, seller_stat
e#51, seller_zip_code_prefix#49, product_weight_g#75, product_length_cm#7
6, product_height_cm#77, product_width_cm#78, product_category_name_englis
h#67]

: +- *(9) BroadcastHashJoin [product_cat
egory_name#71], [product_category_name#66], LeftOuter, BuildRight, false
: - *(9) Project [order_id#8, order_
item_id#9, price#13, freight_value#14, order_purchase_timestamp#25, custom
er_state#42, customer_zip_code_prefix#40, seller_state#51, seller_zip_code
_prefix#49, product_category_name#71, product_weight_g#75, product_length_
cm#76, product_height_cm#77, product_width_cm#78]

: +- *(9) BroadcastHashJoin [produ
ct_id#10], [product_id#70], LeftOuter, BuildRight, false
: - *(9) Project [order_id#8,
order_item_id#9, product_id#10, price#13, freight_value#14, order_purchase
_timestamp#25, customer_state#42, customer_zip_code_prefix#40, seller_stat
e#51, seller_zip_code_prefix#49]

: +- *(9) BroadcastHashJoin
[seller_id#11], [seller_id#48], Inner, BuildRight, false
: - *(9) Project [order_
id#8, order_item_id#9, product_id#10, seller_id#11, price#13, freight_valu
e#14, order_purchase_timestamp#25, customer_state#42, customer_zip_code_pr
efix#40]

: +- *(9) BroadcastHas
hJoin [customer_id#23], [customer_id#38], Inner, BuildRight, false

```

```

:      :      :      :      :      :- *(9) Project
[order_id#8, order_item_id#9, product_id#10, seller_id#11, price#13, freig
ht_value#14, customer_id#23, order_purchase_timestamp#25]
:      :      :      :      :      : +- *(9) Broadc
astHashJoin [order_id#8], [order_id#22], Inner, BuildRight, false
:      :      :      :      :      : :- *(9) Fil
ter (isnotnull(order_id#8) AND isnotnull(seller_id#11))
:      :      :      :      :      : +- FileS
can csv [order_id#8,order_item_id#9,product_id#10,seller_id#11,price#13,fr
eight_value#14] Batched: false, DataFilters: [isnotnull(order_id#8), isnot
null(seller_id#11)], Format: CSV, Location: InMemoryFileIndex(1 paths)[fil
e:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/data/olist_or..., Pa
rtitionFilters: [], PushedFilters: [IsNotNull(order_id), IsNotNull(seller_
id)], ReadSchema: struct<order_id:string,order_item_id:int,product_id:stri
ng,seller_id:string,price:double,freight...
:      :      :      :      :      : +- Broadcas
tExchange HashedRelationBroadcastMode(List(input[0, string, true]),false),
[plan_id=9881]
:      :      :      :      :      : +- *(1)
Project [order_id#22, customer_id#23, order_purchase_timestamp#25]
:      :      :      :      :      : +- *
(1) Filter (((isnotnull(order_status#24) AND (order_status#24 = delivere
d)) AND isnotnull(order_delivered_customer_date#28)) AND isnotnull(order_i
d#22)) AND isnotnull(customer_id#23))
:      :      :      :      :      : +-
FileScan csv [order_id#22,customer_id#23,order_status#24,order_purchase_ti
mestamp#25,order_delivered_customer_date#28] Batched: false, DataFilters:
[isnotnull(order_status#24), (order_status#24 = delivered), isnotnull(orde
r_delivered_customer_da..., Format: CSV, Location: InMemoryFileIndex(1 pat
hs)[file:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/data/olist_o
r..., PartitionFilters: [], PushedFilters: [IsNotNull(order_status), Equal
To(order_status,delivered), IsNotNull(order_delivered_customer_dat..., Rea
dSchema: struct<order_id:string,customer_id:string,order_status:string,ord
er_purchase_timestamp:timestamp,...
:      :      :      :      :      : +- BroadcastExcha
nge HashedRelationBroadcastMode(List(input[0, string, false]),false), [pla
n_id=9889]
:      :      :      :      :      : +- *(2) Filter
isnotnull(customer_id#38)
:      :      :      :      :      : +- FileScan
csv [customer_id#38,customer_zip_code_prefix#40,customer_state#42] Batche
d: false, DataFilters: [isnotnull(customer_id#38)], Format: CSV, Location:
InMemoryFileIndex(1 paths)[file:/Users/mmomeddu9/Desktop/ito5202-assessme
nt1-spark/data/olist_cu..., PartitionFilters: [], PushedFilters: [IsNotNul
l(customer_id)], ReadSchema: struct<customer_id:string,customer_zip_code_p
refix:int,customer_state:string>
:      :      :      :      :      : +- BroadcastExchange Ha
shedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=9
897]
:      :      :      :      :      : +- *(3) Filter isnot
null(seller_id#48)
:      :      :      :      :      : +- FileScan csv
[seller_id#48,seller_zip_code_prefix#49,seller_state#51] Batched: false, D
ataFilters: [isnotnull(seller_id#48)], Format: CSV, Location: InMemoryFile
Index(1 paths)[file:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/da
ta/olist_se..., PartitionFilters: [], PushedFilters: [IsNotNull(seller_i
d)], ReadSchema: struct<seller_id:string,seller_zip_code_prefix:int,seller
_state:string>
:      :      :      :      :      : +- BroadcastExchange HashedRe
lationBroadcastMode(List(input[0, string, false]),false), [plan_id=9905]

```

```

:      :      :      +- *(4) Filter isnotnull(p
roduct_id#70)
:      :      :      +- FileScan csv [produc
t_id#70,product_category_name#71,product_weight_g#75,product_length_cm#76,
product_height_cm#77,product_width_cm#78] Batched: false, DataFilters: [is
notnull(product_id#70)], Format: CSV, Location: InMemoryFileIndex(1 paths)
[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_p
r..., PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSc
hema: struct<product_id:string,product_category_name:string,product_weight
_g:int,product_length_cm:int,...
:      :      +- BroadcastExchange HashedRelation
BroadcastMode(List(input[0, string, false]),false), [plan_id=9917]
:      :      +- Union
:      :      :- *(5) Filter isnotnull(prod
uct_category_name#66)
:      :      :      +- FileScan csv [product_c
ategory_name#66,product_category_name_english#67] Batched: false, DataFilt
ers: [isnotnull(product_category_name#66)], Format: CSV, Location: InMemor
yFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spa
rk/data/product..., PartitionFilters: [], PushedFilters: [IsNotNull(produ
ct_category_name)], ReadSchema: struct<product_category_name:string,produc
t_category_name_english:string>
:      :      +- *(6) Filter isnotnull(prod
uct_category_name#537)
:      :      +- *(6) Scan ExistingRDD[p
roduct_category_name#537,product_category_name_english#538]
:      +- BroadcastExchange HashedRelationBroadc
astMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=9926]
:      +- *(7) Project [geolocation_zip_code_
prefix#56 AS customer_zip_code_prefix#682, lat#403 AS cust_lat#683, lng#40
5 AS cust_lng#684]
:      +- *(7) Filter isnotnull(geolatio
n_zip_code_prefix#56)
:      +- InMemoryTableScan [geolatio
n_zip_code_prefix#56, lat#403, lng#405], [isnotnull(geolocation_zip_code_p
refix#56)]
:      +- InMemoryRelation [geolo
cation_zip_code_prefix#56, lat#403, lng#405], StorageLevel(disk, memory, d
eserialized, 1 replicas)
:      +- *(2) HashAggregat
e(keys=[geolocation_zip_code_prefix#56], functions=[avg(geolocation_lat#5
7), avg(geolocation_lng#58)], output=[geolocation_zip_code_prefix#56, lat#
403, lng#405])
:      +- Exchange hashp
artitioning(geolocation_zip_code_prefix#56, 4), ENSURE_REQUIREMENTS, [plan
_id=1685]
:      +- *(1) HashAg
gregate(keys=[geolocation_zip_code_prefix#56], functions=[partial_avg(geol
ocation_lat#57), partial_avg(geolocation_lng#58)], output=[geolocation_zip
_code_prefix#56, sum#413, count#414L, sum#415, count#416L])
:      +- FileScan
csv [geolocation_zip_code_prefix#56,geolocation_lat#57,geolocation_lng#58]
Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex
(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/ol
ist_ge..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<geo
location_zip_code_prefix:int,geolocation_lat:double,geolocation_lng:double
>
:      +- ReusedExchange [seller_zip_code_prefix#688,
sell_lat#689, sell_lng#690], BroadcastExchange HashedRelationBroadcastMode
(List(cast(input[0, int, true] as bigint)),false), [plan_id=1608]

```

```

== Physical Plan ==
*(5) Project [purchase_quarter#775, lane#748, seller_state#51, customer_state#42, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_distance_km#8005, freight_per_kg#8018, quarter_baseline_fpk#8548, (freight_per_kg#8018 / quarter_baseline_fpk#8548) AS relative_to_quarter#9137]
+- *(5) BroadcastHashJoin [purchase_quarter#775], [purchase_quarter#9116], LeftOuter, BuildRight, false
    :- *(5) Project [lane#748, seller_state#51, customer_state#42, purchase_quarter#775, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_distance_km#8005, (total_freight#7997 / total_weight_kg#7999) AS freight_per_kg#8018]
        : +- *(5) Filter (item_count#7994L >= 30)
            : +- *(5) HashAggregate(keys=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775], functions=[count(1), sum(freight_value#14), sum(weight_kg#803), avg(freight_per_kg#832), avg(freight_to_price_ratio#862), avg(distance_km#925), count(distinct order_id#8)], output=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775, item_count#7994L, order_count#7995L, total_freight#7997, total_weight_kg#7999, mean_item_freight_per_kg#8001, mean_freight_to_price#8003, mean_distance_km#8005])
                : +- Exchange hashpartitioning(lane#748, seller_state#51, customer_state#42, purchase_quarter#775, 4), ENSURE_REQUIREMENTS, [plan_id=10006]
                    : +- *(2) HashAggregate(keys=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775], functions=[merge_count(1), merge_sum(freight_value#14), merge_sum(weight_kg#803), merge_avg(freight_per_kg#832), merge_avg(freight_to_price_ratio#862), merge_avg(distance_km#925), partial_count(distinct order_id#8)], output=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775, count#8334L, sum#9877, sum#9879, sum#9882, count#9883L, sum#9886, count#9887L, sum#9890, count#9891L, count#9894L])
                        : +- *(2) HashAggregate(keys=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775, order_id#8], functions=[merge_count(1), merge_sum(freight_value#14), merge_sum(weight_kg#803), merge_avg(freight_per_kg#832), merge_avg(freight_to_price_ratio#862), merge_avg(distance_km#925)], output=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775, order_id#8, count#8334L, sum#9877, sum#9879, sum#9882, count#9883L, sum#9886, count#9887L, sum#9890, count#9891L])
                            : +- Exchange hashpartitioning(lane#748, seller_state#51, customer_state#42, purchase_quarter#775, order_id#8, 4), ENSURE_REQUIREMENTS, [plan_id=10001]
                                : +- *(1) HashAggregate(keys=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775, order_id#8], functions=[partial_count(1), partial_sum(freight_value#14), partial_sum(weight_kg#803), partial_avg(freight_per_kg#832), partial_avg(freight_to_price_ratio#862), partial_avg(distance_km#925)], output=[lane#748, seller_state#51, customer_state#42, purchase_quarter#775, order_id#8, count#8334L, sum#9877, sum#9879, sum#9882, count#9883L, sum#9886, count#9887L, sum#9890, count#9891L])
                                    : +- *(1) Filter (isnotnull(weight_kg#803) AND (weight_kg#803 > 0.0))
                                        : +- InMemoryTableScan [order_id#8, lane#748, seller_state#51, customer_state#42, purchase_quarter#775, freight_value#14, weight_kg#803, freight_per_kg#832, freight_to_price_ratio#862, distance_km#925], [isnotnull(weight_kg#803), (weight_kg#803 > 0.0)]
                                            : +- InMemoryRelation [order_id#8, order_item_id#9, lane#748, seller_state#51, customer_state#42, customer_zip_code_prefix#40, seller_zip_code_prefix#49, purchase_quarter#775, order_purchase_timestamp#25, price#13, freight_value#14, weight_kg#803, freight_per_kg

```

```

#832, freight_to_price_ratio#862, package_volume_cm3#893, distance_km#925,
is_intrastate#958, product_category_name#71, product_category_name_english
#67], StorageLevel(disk, memory, deserialized, 1 replicas)
:                                     +- *(9) Project [order_id#8, ord
er_item_id#9, lane#748, seller_state#51, customer_state#42, customer_zip_c
ode_prefix#40, seller_zip_code_prefix#49, purchase_quarter#775, order_purc
hase_timestamp#25, price#13, freight_value#14, weight_kg#803, CASE WHEN (w
eight_kg#803 > 0.0) THEN (freight_value#14 / weight_kg#803) END AS freight
_per_kg#832, CASE WHEN (price#13 > 0.0) THEN (freight_value#14 / price#13)
END AS freight_to_price_ratio#862, ((product_length_cm#76 * product_height
_cm#77) * product_width_cm#78) AS package_volume_cm3#893, (12742.0 * ASIN
(SQRT((POWER(SIN((RADIANS((cust_lat#683 - sell_lat#689)) / 2.0)), 2.0) +
((COS(RADIANS(sell_lat#689)) * COS(RADIANS(cust_lat#683))) * POWER(SIN((RA
DIANS((cust_lng#684 - sell_lng#690)) / 2.0)), 2.0)))))) AS distance_km#92
5, (seller_state#51 = customer_state#42) AS is_intrastate#958, product_cat
egory_name#71, product_category_name_english#67]
:                                     +- *(9) Project [seller_zip_c
ode_prefix#49, customer_zip_code_prefix#40, product_category_name#71, orde
r_id#8, order_item_id#9, price#13, freight_value#14, order_purchase_timest
amp#25, customer_state#42, seller_state#51, product_length_cm#76, product_
height_cm#77, product_width_cm#78, product_category_name_english#67, cust_
lat#683, cust_lng#684, sell_lat#689, sell_lng#690, concat_ws(->, seller_
state#51, customer_state#42) AS lane#748, concat_ws(-, cast(year(cast(orde
r_purchase_timestamp#25 as date)) as string), concat(Q, cast(quarter(cast
(order_purchase_timestamp#25 as date)) as string))) AS purchase_quarter#77
5, (cast(product_weight_g#75 as double) / 1000.0) AS weight_kg#803]
:                                     +- *(9) BroadcastHashJoin
[seller_zip_code_prefix#49], [seller_zip_code_prefix#688], LeftOuter, Buil
dRight, false
:                                     :- *(9) Project [custom
er_zip_code_prefix#40, product_category_name#71, order_id#8, order_item_id
#9, price#13, freight_value#14, order_purchase_timestamp#25, customer_stat
e#42, seller_state#51, seller_zip_code_prefix#49, product_weight_g#75, pro
duct_length_cm#76, product_height_cm#77, product_width_cm#78, product_cate
gory_name_english#67, cust_lat#683, cust_lng#684]
:                                     : +- *(9) BroadcastHas
hJoin [customer_zip_code_prefix#40], [customer_zip_code_prefix#682], LeftO
uter, BuildRight, false
:                                     : :- *(9) Project
[product_category_name#71, order_id#8, order_item_id#9, price#13, freight_
value#14, order_purchase_timestamp#25, customer_state#42, customer_zip_cod
e_prefix#40, seller_state#51, seller_zip_code_prefix#49, product_weight_g#
75, product_length_cm#76, product_height_cm#77, product_width_cm#78, produ
ct_category_name_english#67]
:                                     : : +- *(9) Broadc
astHashJoin [product_category_name#71], [product_category_name#66], LeftOu
ter, BuildRight, false
:                                     : : :- *(9) Pro
ject [order_id#8, order_item_id#9, price#13, freight_value#14, order_purch
ase_timestamp#25, customer_state#42, customer_zip_code_prefix#40, seller_s
tate#51, seller_zip_code_prefix#49, product_category_name#71, product_weig
ht_g#75, product_length_cm#76, product_height_cm#77, product_width_cm#78]
:                                     : : : +- *(9)
BroadcastHashJoin [product_id#10], [product_id#70], LeftOuter, BuildRight,
false
:                                     : : : :- *
(9) Project [order_id#8, order_item_id#9, product_id#10, price#13, freight
_value#14, order_purchase_timestamp#25, customer_state#42, customer_zip_co
de_prefix#40, seller_state#51, seller_zip_code_prefix#49]
:                                     : : : : +-

```

```

*(9) BroadcastHashJoin [seller_id#11], [seller_id#48], Inner, BuildRight,
false
:
:
:
:
:- *(9) Project [order_id#8, order_item_id#9, product_id#10, seller_id#11,
price#13, freight_value#14, order_purchase_timestamp#25, customer_state#4
2, customer_zip_code_prefix#40]
:
:
:
:
+- *(9) BroadcastHashJoin [customer_id#23], [customer_id#38], Inner, Bu
ildRight, false
:
:
:
:
:- *(9) Project [order_id#8, order_item_id#9, product_id#10, seller_
id#11, price#13, freight_value#14, customer_id#23, order_purchase_timestam
p#25]
:
:
:
:
+- *(9) BroadcastHashJoin [order_id#8], [order_id#22], Inner, Bui
ldRight, false
:
:
:
:
:- *(9) Filter (isnotnull(order_id#8) AND isnotnull(seller_id#
11))
:
:
:
:
+- FileScan csv [order_id#8,order_item_id#9,product_id#10,s
eller_id#11,price#13,freight_value#14] Batched: false, DataFilters: [isnot
null(order_id#8), isnotnull(seller_id#11)], Format: CSV, Location: InMemor
yFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spa
rk/data/olist_or..., PartitionFilters: [], PushedFilters: [IsNotNull(order
_id), IsNotNull(seller_id)], ReadSchema: struct<order_id:string,order_item
_id:int,product_id:string,seller_id:string,price:double,freight...
:
:
:
:
+- BroadcastExchange HashedRelationBroadcastMode(List(input[0,
string, true]),false), [plan_id=10063]
:
:
:
:
+- *(1) Project [order_id#22, customer_id#23, order_purchas
e_timestamp#25]
:
:
:
:
+- *(1) Filter (((isnotnull(order_status#24) AND (order
_status#24 = delivered)) AND isnotnull(order_delivered_customer_date#28))
AND isnotnull(order_id#22)) AND isnotnull(customer_id#23))
:
:
:
:
+- FileScan csv [order_id#22,customer_id#23,order_sta
tus#24,order_purchase_timestamp#25,order_delivered_customer_date#28] Batch
ed: false, DataFilters: [isnotnull(order_status#24), (order_status#24 = de
livered), isnotnull(order_delivered_customer_da..., Format: CSV, Location:
InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessme
nt1-spark/data/olist_or..., PartitionFilters: [], PushedFilters: [IsNotNul
l(order_status), EqualTo(order_status,delivered), IsNotNull(order_delivere
d_customer_dat..., ReadSchema: struct<order_id:string,customer_id:string,o
rder_status:string,order_purchase_timestamp:timestamp,...
:
:
:
:
+- BroadcastExchange HashedRelationBroadcastMode(List(input[0, strin
g, false]),false), [plan_id=10071]
:
:
:
:
+- *(2) Filter isnotnull(customer_id#38)
:
:
:
:
+- FileScan csv [customer_id#38,customer_zip_code_prefix#40,cu
stomer_state#42] Batched: false, DataFilters: [isnotnull(customer_id#38)],
Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/D
esktop/ito5202-assessment1-spark/data/olist_cu..., PartitionFilters: [], P
ushedFilters: [IsNotNull(customer_id)], ReadSchema: struct<customer_id:str
ing,customer_zip_code_prefix:int,customer_state:string>
:
:
:
:

```

```

+- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10079]
      :           :           :           :
+- *(3) Filter isnotnull(seller_id#48)
      :           :           :           :
+- FileScan csv [seller_id#48,seller_zip_code_prefix#49,seller_state#51] Batched: false, DataFilters: [isnotnull(seller_id#48)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_se..., PartitionFilters: [], PushedFilters: [IsNotNull(seller_id)], ReadSchema: struct<seller_id:string,seller_zip_code_prefix:int,seller_state:string>
      :           :           :           : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10087]
      :           :           :           : +-
*(4) Filter isnotnull(product_id#70)
      :           :           :           :
+- FileScan csv [product_id#70,product_category_name#71,product_weight_g#75,product_length_cm#76,product_height_cm#77,product_width_cm#78] Batched: false, DataFilters: [isnotnull(product_id#70)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_pr..., PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSchema: struct<product_id:string,product_category_name:string,product_weight_g:int,product_length_cm:int,...
      :           :           :           : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10099]
      :           :           :           : +- Union
      :           :           :           : :- *
(5) Filter isnotnull(product_category_name#66)
      :           :           :           : +-
FileScan csv [product_category_name#66,product_category_name_english#67] Batched: false, DataFilters: [isnotnull(product_category_name#66)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/product_..., PartitionFilters: [], PushedFilters: [IsNotNull(product_category_name)], ReadSchema: struct<product_category_name:string,product_category_name_english:string>
      :           :           :           : +- *
(6) Filter isnotnull(product_category_name#537)
      :           :           :           : +-
*(6) Scan ExistingRDD[product_category_name#537,product_category_name_english#538]
      :           :           :           : +- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=10108]
      :           :           :           : +- *(7) Project [geolocation_zip_code_prefix#56 AS customer_zip_code_prefix#682, lat#403 AS cust_lat#683, lng#405 AS cust_lng#684]
      :           :           :           : +- *(7) Filter isnotnull(geolocation_zip_code_prefix#56)
      :           :           :           : +- InMemoryTableScan [geolocation_zip_code_prefix#56, lat#403, lng#405], [isnotnull(geolocation_zip_code_prefix#56)]
      :           :           :           : +-
InMemoryRelation [geolocation_zip_code_prefix#56, lat#403, lng#405], StorageLevel(disk, memory, deserialized, 1 replicas)
      :           :           :           :
+- *(2) HashAggregate(keys=[geolocation_zip_code_prefix#56], functions=[avg(geolocation_lat#57), avg(geolocation_lng#58)], output=[geolocation_zip_code_prefix#56, lat#403, lng#405])

```

```

:
+- Exchange hashpartitioning(geolocation_zip_code_prefix#56, 4), ENSURE_RE
QUIREMENTS, [plan_id=1685]
:
+- *(1) HashAggregate(keys=[geolocation_zip_code_prefix#56], functions=[pa
rtial_avg(geolocation_lat#57), partial_avg(geolocation_lng#58)], output=[g
eolocation_zip_code_prefix#56, sum#413, count#414L, sum#415, count#416L])
:
+- FileScan csv [geolocation_zip_code_prefix#56,geolocation_lat#57,geoloca
tion_lng#58] Batched: false, DataFilters: [], Format: CSV, Location: InMem
oryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-s
park/data/olist_ge..., PartitionFilters: [], PushedFilters: [], ReadSchem
a: struct<geolocation_zip_code_prefix:int,geolocation_lat:double,geoloca
tion_lng:double>
:
+- ReusedExchange [sell
er_zip_code_prefix#688, sell_lat#689, sell_lng#690], BroadcastExchange Has
hedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false),
[plan_id=1608]
+- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string,
false]),false), [plan_id=10019]
+- *(4) HashAggregate(keys=[purchase_quarter#9116], functions=[sum(f
reight_value#9075), sum(weight_kg#803)], output=[purchase_quarter#9116, qu
arter_baseline_fpk#8548])
+- Exchange hashpartitioning(purchase_quarter#9116, 4), ENSURE_RE
QUIREMENTS, [plan_id=10015]
+- *(3) HashAggregate(keys=[purchase_quarter#9116], functions=
[partial_sum(freight_value#9075), partial_sum(weight_kg#803)], output=[pur
chase_quarter#9116, sum#9897, sum#8865])
+- *(3) Filter (isnotnull(weight_kg#803) AND (weight_kg#803
> 0.0))
+- InMemoryTableScan [purchase_quarter#9116, freight_val
ue#9075, weight_kg#803], [isnotnull(weight_kg#803), (weight_kg#803 > 0.0)]
+- InMemoryRelation [order_id#9069, order_item_id#
9070, lane#748, seller_state#9092, customer_state#9088, customer_zip_code_
prefix#9086, seller_zip_code_prefix#9090, purchase_quarter#9116, order_pur
chase_timestamp#9079, price#9074, freight_value#9075, weight_kg#803, freig
ht_per_kg#832, freight_to_price_ratio#862, package_volume_cm3#893, distanc
e_km#925, is_intrastate#958, product_category_name#9094, product_category_
name_english#9103], StorageLevel(disk, memory, deserialized, 1 replicas)
+- *(9) Project [order_id#8, order_item_id#
9, lane#748, seller_state#51, customer_state#42, customer_zip_code_prefix#
40, seller_zip_code_prefix#49, purchase_quarter#775, order_purchase_timest
amp#25, price#13, freight_value#14, weight_kg#803, CASE WHEN (weight_kg#80
3 > 0.0) THEN (freight_value#14 / weight_kg#803) END AS freight_per_kg#83
2, CASE WHEN (price#13 > 0.0) THEN (freight_value#14 / price#13) END AS fr
eight_to_price_ratio#862, ((product_length_cm#76 * product_height_cm#77) *
product_width_cm#78) AS package_volume_cm3#893, (12742.0 * ASIN(SQRT((POWE
R(SIN((RADIANS((cust_lat#683 - sell_lat#689)) / 2.0)), 2.0) + ((COS(RADIAN
S(sell_lat#689)) * COS(RADIANS(cust_lat#683))) * POWER(SIN((RADIANS((cust_
lng#684 - sell_lng#690)) / 2.0)), 2.0)))))) AS distance_km#925, (seller_st
ate#51 = customer_state#42) AS is_intrastate#958, product_category_name#7
1, product_category_name_english#67]
+- *(9) Project [seller_zip_code_prefix#4
9, customer_zip_code_prefix#40, product_category_name#71, order_id#8, orde
r_item_id#9, price#13, freight_value#14, order_purchase_timestamp#25, cust
omer_state#42, seller_state#51, product_length_cm#76, product_height_cm#7
7, product_width_cm#78, product_category_name_english#67, cust_lat#683, cu
st_lng#684, sell_lat#689, sell_lng#690, concat_ws( -> , seller_state#51, c
ustomer_state#42) AS lane#748, concat_ws(-, cast(year(cast(order_purchase_
timestamp#25 as date)) as string), concat(0, cast(quarter(cast(order_purch

```

```

ase_timestamp#25 as date)) as string))) AS purchase_quarter#775, (cast(pro
duct_weight_g#75 as double) / 1000.0) AS weight_kg#803]
      +- *(9) BroadcastHashJoin [seller_zip_
code_prefix#49], [seller_zip_code_prefix#688], LeftOuter, BuildRight, fals
e
      :- *(9) Project [customer_zip_code_
prefix#40, product_category_name#71, order_id#8, order_item_id#9, price#1
3, freight_value#14, order_purchase_timestamp#25, customer_state#42, selle
r_state#51, seller_zip_code_prefix#49, product_weight_g#75, product_length
_cm#76, product_height_cm#77, product_width_cm#78, product_category_name_e
nglish#67, cust_lat#683, cust_lng#684]
      : +- *(9) BroadcastHashJoin [custo
mer_zip_code_prefix#40], [customer_zip_code_prefix#682], LeftOuter, BuildR
ight, false
      : :- *(9) Project [product_cate
gory_name#71, order_id#8, order_item_id#9, price#13, freight_value#14, ord
er_purchase_timestamp#25, customer_state#42, customer_zip_code_prefix#40,
seller_state#51, seller_zip_code_prefix#49, product_weight_g#75, product_l
ength_cm#76, product_height_cm#77, product_width_cm#78, product_category_n
ame_english#67]
      : : +- *(9) BroadcastHashJoin
[product_category_name#71], [product_category_name#66], LeftOuter, BuildRi
ght, false
      : : :- *(9) Project [order_
id#8, order_item_id#9, price#13, freight_value#14, order_purchase_timestam
p#25, customer_state#42, customer_zip_code_prefix#40, seller_state#51, sel
ler_zip_code_prefix#49, product_category_name#71, product_weight_g#75, pro
duct_length_cm#76, product_height_cm#77, product_width_cm#78]
      : : : +- *(9) BroadcastHas
hJoin [product_id#10], [product_id#70], LeftOuter, BuildRight, false
      : : : :- *(9) Project
[order_id#8, order_item_id#9, product_id#10, price#13, freight_value#14, o
rder_purchase_timestamp#25, customer_state#42, customer_zip_code_prefix#4
0, seller_state#51, seller_zip_code_prefix#49]
      : : : : +- *(9) Broadc
astHashJoin [seller_id#11], [seller_id#48], Inner, BuildRight, false
      : : : : :- *(9) Pro
ject [order_id#8, order_item_id#9, product_id#10, seller_id#11, price#13,
freight_value#14, order_purchase_timestamp#25, customer_state#42, customer
_zip_code_prefix#40]
      : : : : : +- *(9)
BroadcastHashJoin [customer_id#23], [customer_id#38], Inner, BuildRight, f
alse
      : : : : : :- *
(9) Project [order_id#8, order_item_id#9, product_id#10, seller_id#11, pri
ce#13, freight_value#14, customer_id#23, order_purchase_timestamp#25]
      : : : : : : +-
*(9) BroadcastHashJoin [order_id#8], [order_id#22], Inner, BuildRight, fal
se
      : : : : : :
      :- *(9) Filter (isnotnull(order_id#8) AND isnotnull(seller_id#11))
      : : : : : :
      : +- FileScan csv [order_id#8,order_item_id#9,product_id#10,seller_id#11,
price#13,freight_value#14] Batched: false, DataFilters: [isnotnull(order_i
d#8), isnotnull(seller_id#11)], Format: CSV, Location: InMemoryFileIndex(1
paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist
_or..., PartitionFilters: [], PushedFilters: [IsNotNull(order_id), IsNotNu
ll(seller_id)], ReadSchema: struct<order_id:string,order_item_id:int,produ
ct_id:string,seller_id:string,price:double,freight_...
      : : : : : :

```

```

+- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, true]),false), [plan_id=10126]
      :      :      :      :      :
+- *(1) Project [order_id#22, customer_id#23, order_purchase_timestamp#25]
      :      :      :      :      :
+- *(1) Filter (((isnotnull(order_status#24) AND (order_status#24 = delivered)) AND isnotnull(order_delivered_customer_date#28)) AND isnotnull(order_id#22)) AND isnotnull(customer_id#23))
      :      :      :      :      :
+- FileScan csv [order_id#22,customer_id#23,order_status#24,order_purchase_timestamp#25,order_delivered_customer_date#28] Batched: false, DataFilters: [isnotnull(order_status#24), (order_status#24 = delivered), isnotnull(order_delivered_customer_date#28)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/data/olist_order_status.csv], PartitionFilters: [], PushedFilters: [IsNotNull(order_status), EqualTo(order_status,delivered), IsNotNull(order_delivered_customer_date)], ReadSchema: struct<order_id:string,customer_id:string,order_status:string,order_purchase_timestamp:timestamp,...
      :      :      :      :      : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10134]
      :      :      :      :      : +- *(2) Filter isnotnull(customer_id#38)
      :      :      :      :      :
+- FileScan csv [customer_id#38,customer_zip_code_prefix#40,customer_state#42] Batched: false, DataFilters: [isnotnull(customer_id#38)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/data/olist_customer_state.csv], PartitionFilters: [], PushedFilters: [IsNotNull(customer_id)], ReadSchema: struct<customer_id:string,customer_zip_code_prefix:int,customer_state:string>
      :      :      :      :      : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10142]
      :      :      :      :      : +- *(3) Filter isnotnull(seller_id#48)
      :      :      :      :      : +- FileScan csv [seller_id#48,seller_zip_code_prefix#49,seller_state#51] Batched: false, DataFilters: [isnotnull(seller_id#48)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/data/olist_seller_state.csv], PartitionFilters: [], PushedFilters: [IsNotNull(seller_id)], ReadSchema: struct<seller_id:string,seller_zip_code_prefix:int,seller_state:string>
      :      :      :      :      : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10150]
      :      :      :      :      : +- *(4) Filter isnotnull(product_id#70)
      :      :      :      :      : +- FileScan csv [product_id#70,product_category_name#71,product_weight_g#75,product_length_cm#76,product_height_cm#77,product_width_cm#78] Batched: false, DataFilters: [isnotnull(product_id#70)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmomeddu9/Desktop/ito5202-assessment1-spark/data/olist_product.csv], PartitionFilters: [], PushedFilters: [IsNotNull(product_id)], ReadSchema: struct<product_id:string,product_category_name:string,product_weight_g:int,product_length_cm:int,...
      :      :      :      :      : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),false), [plan_id=10162]
      :      :      :      :      : +- Union
      :      :      :      :      : :- *(5) Filter is

```

```

notnull(product_category_name#66)
      :      :      : +- FileScan csv [product_category_name#66,product_category_name_english#67] Batched: false, DataFilters: [isnotnull(product_category_name#66)], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/product_..., PartitionFilters: [], PushedFilters: [IsNotNull(product_category_name)], ReadSchema: struct<product_category_name:string,product_category_name_english:string>
      :      :      +- *(6) Filter is notnull(product_category_name#537)
      :      :      +- *(6) Scan ExistingRDD[product_category_name#537,product_category_name_english#538]
      :      +- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=10171]
      :      +- *(7) Project [geolocation_zip_code_prefix#56 AS customer_zip_code_prefix#682, lat#403 AS cust_lat#683, lng#405 AS cust_lng#684]
      :      +- *(7) Filter isnotnull(geolocation_zip_code_prefix#56)
      :      +- InMemoryTableScan [geolocation_zip_code_prefix#56, lat#403, lng#405], [isnotnull(geolocation_zip_code_prefix#56)]
      :      +- InMemoryRelation [geolocation_zip_code_prefix#56, lat#403, lng#405], StorageLevel(disk, memory, serialized, 1 replicas)
      :      +- *(2) HashAggregate(keys=[geolocation_zip_code_prefix#56], functions=[avg(geolocation_lat#57), avg(geolocation_lng#58)], output=[geolocation_zip_code_prefix#56, lat#403, lng#405])
      :      +- Exchange hashpartitioning(geolocation_zip_code_prefix#56, 4), ENSURE_REQUIREMENTS, [plan_id=1685]
      :      +- *(1) HashAggregate(keys=[geolocation_zip_code_prefix#56], functions=[partial_avg(geolocation_lat#57), partial_avg(geolocation_lng#58)], output=[geolocation_zip_code_prefix#56, sum#413, count#414L, sum#415, count#416L])
      :      +- FileScan csv [geolocation_zip_code_prefix#56,geolocation_lat#57,geolocation_lng#58] Batched: false, DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1 paths)[file:/Users/mmondeddu9/Desktop/ito5202-assessment1-spark/data/olist_ge..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<geolocation_zip_code_prefix:int,geolocation_lat:double,geolocation_lng:double>
      +- ReusedExchange [seller_zip_code_prefix#688, sell_lat#689, sell_lng#690], BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int, true] as bigint)),false), [plan_id=1608]

```

```

In [66]: # Pull out just the join and shuffle operators from the uncached base view
base_plan = base._jdf.queryExecution().executedPlan().toString()

for line in base_plan.split("\n"):
    if "Join" in line or "Exchange" in line:
        print(line.strip())

```

```

+- *(12) BroadcastHashJoin [seller_zip_code_prefix#49], [seller_zip_code_p
refix#688], LeftOuter, BuildRight, false
: +- *(12) BroadcastHashJoin [customer_zip_code_prefix#40], [customer_zip
_code_prefix#682], LeftOuter, BuildRight, false
:   : +- *(12) BroadcastHashJoin [product_category_name#71], [product_c
ategory_name#66], LeftOuter, BuildRight, false
:   :   : +- *(12) BroadcastHashJoin [product_id#10], [product_id#7
0], LeftOuter, BuildRight, false
:   :   :   : +- *(12) BroadcastHashJoin [seller_id#11], [seller_id
#48], Inner, BuildRight, false
:   :   :   :   : +- *(12) BroadcastHashJoin [customer_id#23], [c
ustomer_id#38], Inner, BuildRight, false
:   :   :   :   :   : +- *(12) SortMergeJoin [order_id#8], [ord
er_id#22], Inner
:   :   :   :   :   :   : +- Exchange hashpartitioning(order_
id#8, 4), ENSURE_REQUIREMENTS, [plan_id=10361]
:   :   :   :   :   :   :   : +- Exchange hashpartitioning(order_
id#22, 4), ENSURE_REQUIREMENTS, [plan_id=10370]
:   :   :   :   :   :   :   : +- BroadcastExchange HashedRelationBroadcast
Mode(List(input[0, string, false]),false), [plan_id=10380]
:   :   :   :   :   :   :   : +- BroadcastExchange HashedRelationBroadcastMode(L
ist(input[0, string, false]),false), [plan_id=10387]
:   :   :   :   :   :   :   : +- BroadcastExchange HashedRelationBroadcastMode(List(in
put[0, string, false]),false), [plan_id=10394]
:   :   :   :   :   :   :   : +- BroadcastExchange HashedRelationBroadcastMode(List(input[0,
string, false]),false), [plan_id=10403]
:   :   :   :   :   :   :   : +- BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0,
int, true] as bigint)),false), [plan_id=10411]
:   :   :   :   :   :   :   : +- Exchange hashpartitioning(geolocation_zip
_code_prefix#56, 4), ENSURE_REQUIREMENTS, [plan_id=10494]
+- ReusedExchange [seller_zip_code_prefix#688, sell_lat#689, sell_lng#69
0], BroadcastExchange HashedRelationBroadcastMode(List(cast(input[0, int,
true] as bigint)),false), [plan_id=10411]

```

```

In [67]: # Check Spark's estimated size for the larger join inputs
threshold_bytes = int(
    spark.conf.get("spark.sql.autoBroadcastJoinThreshold").rstrip("b")
)

threshold_mb = threshold_bytes / (1024 ** 2)

print("Broadcast threshold:", round(threshold_mb, 1), "MB")

```

Broadcast threshold: 10.0 MB

```

In [68]: # only the columns needed from orders
orders_3cols = orders_delivered.select(
    "order_id",
    "customer_id",
    "order_purchase_timestamp"
)

```

```

In [69]: customers_size = customers_join._jdf.queryExecution() \
        .optimizedPlan().stats().sizeInBytes()

orders_3_size = orders_3cols._jdf.queryExecution() \
        .optimizedPlan().stats().sizeInBytes()

orders_5_size = orders_join._jdf.queryExecution() \
        .optimizedPlan().stats().sizeInBytes()

In [70]: customers_mb = customers_size / (1024 ** 2)
orders_3_mb = orders_3_size / (1024 ** 2)
orders_5_mb = orders_5_size / (1024 ** 2)

In [71]: print(
        "Customers (3 columns):",
        round(customers_mb, 1), "MB",
        "| Broadcast eligible:", customers_mb <= threshold_mb
    )

print(
    "Orders (3 columns):",
    round(orders_3_mb, 1), "MB",
    "| Broadcast eligible:", orders_3_mb <= threshold_mb
)

print(
    "Orders (5 columns):",
    round(orders_5_mb, 1), "MB",
    "| Broadcast eligible:", orders_5_mb <= threshold_mb
)

```

```

Customers (3 columns): 4.9 MB | Broadcast eligible: True
Orders (3 columns): 8.7 MB | Broadcast eligible: True
Orders (5 columns): 11.2 MB | Broadcast eligible: False

```

Annotated physical plan

Since the physical plan above is quite long, we have extracted points where the Shuffle Exchanges occur, as well as the operators immediately preceding the shuffles. Annotations have been added as comment lines below.

```

*(5) Project [purchase_quarter, lane, ...,
              (freight_per_kg / quarter_baseline_fpk) AS
relative_to_quarter]

+- *(5) BroadcastHashJoin [purchase_quarter], [purchase_quarter],
LeftOuter, BuildRight

      :- *(5) Project [..., (total_freight / total_weight_kg) AS
freight_per_kg]

      :   +- *(5) Filter (item_count >= 30)

      :       +- *(5) HashAggregate(
      :           keys=[lane, seller_state, customer_state,
purchase_quarter],
      :           functions=[count(1), sum(...), avg(...),

```

```

count(distinct order_id)]
:      )
:
:      +- Exchange hashpartitioning(
:          lane,
:          seller_state,
:          customer_state,
:          purchase_quarter,
:          4
:      )
:      # SHUFFLE EXCHANGE 2
:      # Partitioning: hash partitioning on lane, seller
state,
:      # customer state and purchase quarter
:      # Number of partitions: 4
:      # Operator before exchange: HashAggregate
:      # Operator after exchange: HashAggregate
:
:      +- *(2) HashAggregate(
:          keys=[lane, seller_state, customer_state,
purchase_quarter],
:          functions=[merge_count(1), merge_sum(...),
:              ..., partial_count(distinct
order_id)]
:      )
:
:      +- *(2) HashAggregate(
:          keys=[lane, seller_state, customer_state,
:              purchase_quarter, order_id],
:          functions=[merge_count(1),
merge_sum(...), ...]
:      )
:
:      +- Exchange hashpartitioning(
:          lane,
:          seller_state,
:          customer_state,
:          purchase_quarter,
:          order_id,
:          4
:      )
:      # SHUFFLE EXCHANGE 1
:      # Partitioning: hash partitioning on the
four grouping
:      # keys plus order_id
:      # Number of partitions: 4
:      # Operator before exchange: HashAggregate
:      # Operator after exchange: HashAggregate
:
:      +- *(1) HashAggregate(
:          keys=[lane, seller_state,
customer_state,
:              purchase_quarter, order_id],
:          functions=[partial_count(1),
:              partial_sum(...),
:              partial_avg(...)]

```


Analysis

Why the shuffles occur

When looking at the physical plan, the key question that arises is 'why do the shuffles occur?'.

To answer this, we first need to consider the fact that rows which belong to the same lane-quarter are initially spread across multiple partitions, so Spark cannot finish the aggregation locally. So, to do the aggregation, Spark needs to first shuffle by the lane-quarter keys plus `order_id` in order to bring duplicate order IDs together (*Shuffle 1*), then it needs to repartition by the lane-quarter keys alone to complete the aggregation (*Shuffle 2*). Finally, the quarterly baseline requires its own smaller shuffle by `purchase_quarter` (*Shuffle 3*).

Impact on execution

It is important to consider the fact that shuffle exchanges create stage boundaries and require intermediate data to be written and read between tasks. Shuffle 1 is the most expensive due to the fact that including `order_id` in the key limits how by how much the partial aggregation can reduce the input rows before data moves. Shuffle 2 however, is much smaller because much of the data has already been combined. While in local mode there is no network transfer between cluster machines, Spark still incurs serialisation, local shuffle I/O and task-scheduling overhead.

Reducing the shuffles

Broadcasting does not offer us with any avenue to reduce the number of shuffles, since the expensive operation is actually an aggregation rather than a large-small join. Instead, our clearest option would be to replace `countDistinct` with `approx_count_distinct`. In doing so, we would be able to merge the intermediate state across partitions, thereby avoiding the distinct-specific shuffle pattern while making `order_count` approximate. We argue that this would be a reasonable trade-off in this instance since order count is included for context and does not actually affect the freight ranking itself.

Join strategies in the base view

The first thing we should note is that because `base_view` is cached, the main physical plan reads from an `InMemoryTableScan` rather than showing all of the joins used to build it. When looking at the uncached join plan instead, it shows much more detail about the strategies Spark chose earlier in the pipeline.

We can see that the main difference appears around the `orders` join. In the uncached `base` plan, Spark uses a `SortMergeJoin` on `order_id`, with an `Exchange hashpartitioning` on both sides of the join. This means both inputs need to be redistributed so that matching `order_id` values are placed in the same

partitions. In contrast, the smaller `customers` input is handled using a `BroadcastHashJoin`, thereby avoiding the shuffle.

To understand why, we should look to Spark's estimated input size. We can see that the broadcast threshold is **10 MB**. After projection pruning, the three-column `customers` input is estimated at about **4.9 MB**, making it small enough to broadcast. The estimate for `orders` changes depending on which columns are retained, with the narrower version being **8.7 MB**, while the wider version is **11.2 MB**.

From the above, we can see the importance of **projection pruning**, where Catalyst can remove columns that are not needed later in the query. Here, reducing estimated size also changed the join that was needed from `SortMergeJoin` to a `BroadcastHashJoin`, removing the two hash-partitioning exchanges that would otherwise be needed.

Importantly, we can see that this plan actually contradicts our earlier expectations from Part A.2, where we had expected that both joins would be too large to broadcast, and that the geolocation lookup would be broadcasted twice.

Part B.4: DAG analysis via the Spark Web UI

Now, we want to look at examining the DAG that is produced when our full DataFrame query runs, using the Spark Web UI.

While earlier, in Part B.3, we focused on the physical execution plan, the Web UI will now be able to show us how that plan was carried out as Spark jobs and stages. Through this, we will then be able to better identify where shuffle boundaries occur, how tasks are distributed across stages, and where most of the execution time is spent.

Here we want to run the complete DataFrame query, including the window functions, so the DAG represents the full analysis rather than only the lane-quarter aggregation examined in Part B.3.

```
In [72]: # Label the job so it is easy to find in the Web UI
spark.sparkContext.setJobDescription("Part B.4: full DataFrame implementa

dag_rows = build_dataframe_query().collect()
print("Rows returned:", len(dag_rows))

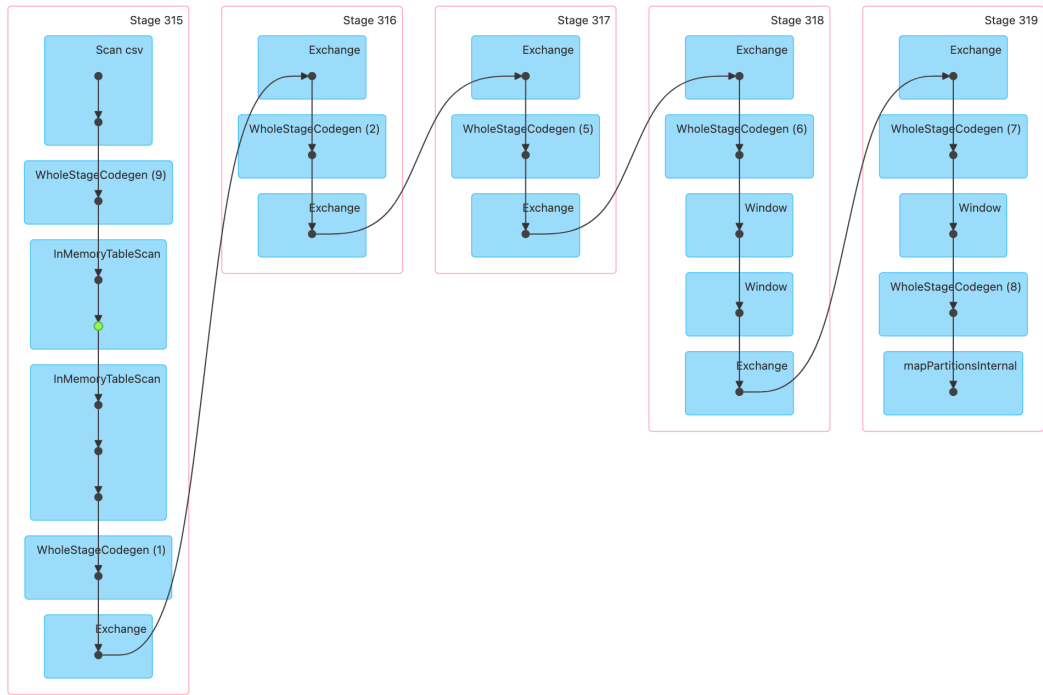
spark.sparkContext.setJobDescription(None)
print("Web UI:", spark.sparkContext.uiWebUrl)
```

Rows returned: 348

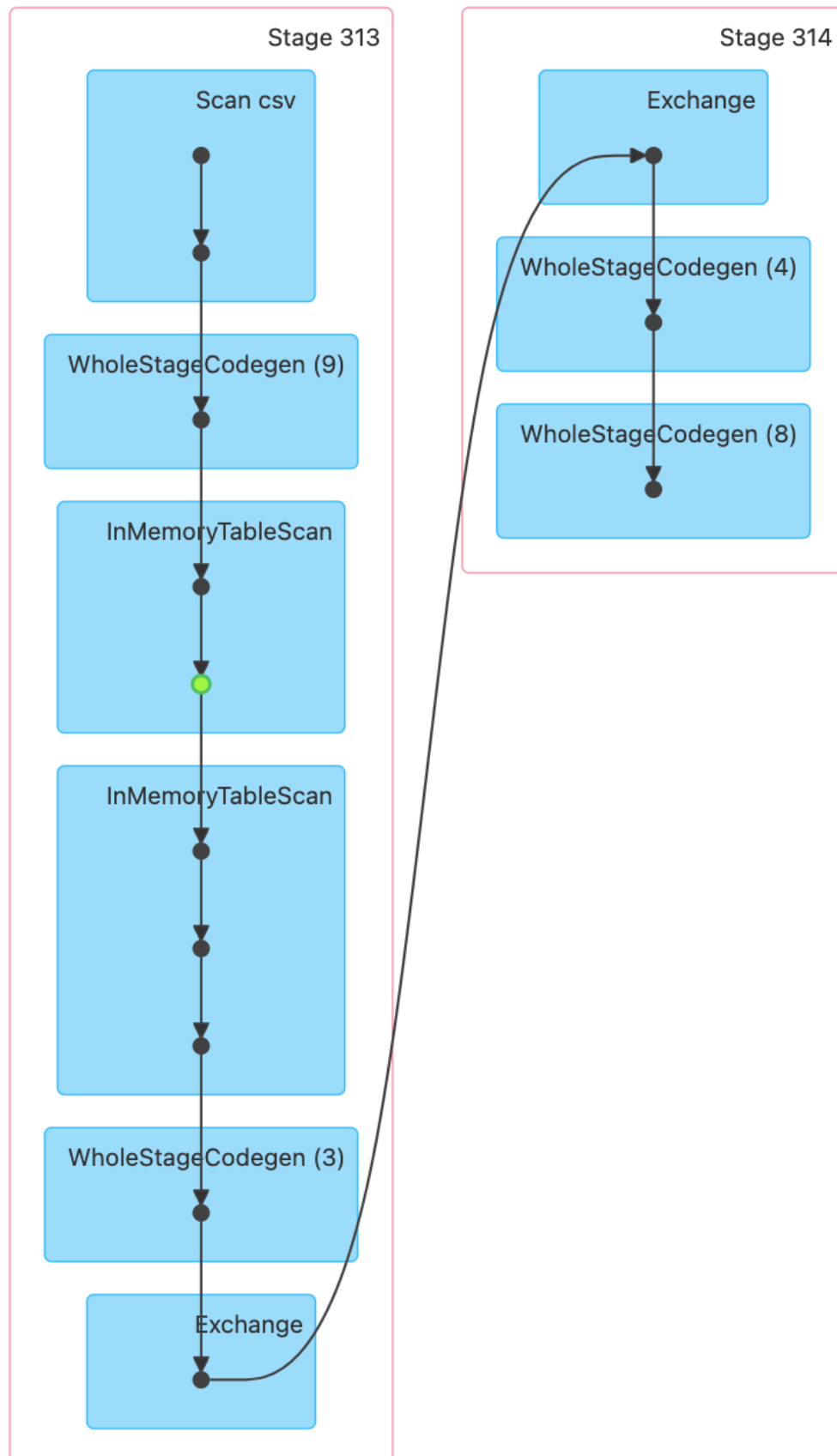
Web UI: http://192.168.0.137:4040

DAG visualisation

Main job (stages 315 to 319):



Baseline job (tages 313 and 314):



DAG analysis

We can see from the DAG visualisation that when the query ran, Spark divided the work into two jobs and seven stages.

Job	Stage	Duration	Shuffle read	Shuffle write	Work performed
Baseline	313	61 ms		1,386 B	Reads cached data and calculates partial quarterly baseline sums
Baseline	314	21 ms	1,386 B		Completes the baseline aggregation for broadcast
Main	315	0.3 s		7.6 MiB	Partial aggregation by lane-quarter and <code>order_id</code>
Main	316	0.3 s	7.6 MiB	294.1 KiB	Merges partial results and processes the distinct order count
Main	317	55ms	294.1 KiB	29.0 KiB	Final aggregation, volume filter and baseline join
Main	318	63 ms	29.0 KiB	29.3 KiB	Ranking and lane count within each quarter
Main	319	82 ms	29.3 KiB		<code>LAG()</code> calculations and result collection

Stage boundaries

We can see from the above that the stage boundaries line up with the shuffle exchanges that we identified in the physical plan. More specifically, we see that stages 315 - 316 redistribute data using the grouping keys plus `order_id` for `countDistinct`, while 316 - 317 repartitions by the lane-quarter grouping keys to complete the aggregation.

Additionally, the window calculations then require two different partitionings. 317 - 318 repartitions by `purchase_quarter` for the ranking and lane count, while 318 - 319 repartitions by `lane` for the `LAG()` calculations. The baseline has its own small shuffle between stages 313 - 314 before the result is broadcast to the main query.

Furthermore, we see that stage 318 contains both quarter-based Window operators after one exchange, showing that Spark can use the same partitioning for those calculations. The `LAG()` window needs another exchange because it uses a different partition key.

Where the cost sits

We note that the largest movement is between stages 315 and 316, where *7.6 miB* is shuffled. This is much larger than the *294.1 kiB* in the following shuffle and the roughly *29 kiB* moved by each window stage.

Stages 315 and 316 also account for most of the main job's execution time. Because `order_id` is included in the first aggregation key, the partial aggregation cannot reduce the data as strongly before that shuffle. By the time the window functions run, only 348 lane-quarter rows remain, so their shuffle cost is comparatively small.

Skew

Based on our analysis, there is very little evidence of task skew. Stage 316's four tasks took about *0.2–0.4 seconds* and each processed roughly 24,000 records.

Possible optimisation

Based on the above, we would argue that the clearest optimisation target is the exact distinct order count. This is due to the fact that if an approximate value were acceptable, `approx_count_distinct` could avoid the extra distinct-specific aggregation pattern and reduce shuffle work. Spark's default relative standard deviation is about 5%, which could be acceptable here because `order_count` is descriptive and thus does not affect the lane ranking.